



AniLuCa
PUBLISHING



Python



SNMP



PYTHON PARA REDES

PROGRAMACIÓN Y AUTOMATIZACIÓN
CON SNMP, SSH, APIS Y MÁS DESDE CERO

AUTOR

DANIEL LLIVIPUMA



SERIE

PYTHON APLICADO

Dedicatoria

A mis hijas **Ana Lucía** y **Ana Carolina**,
mis mayores tesoros y fuente constante de inspiración.

Y a mi esposa,
por regalarme los milagros más hermosos de mi vida
y acompañarme con amor en cada paso de este camino.

Prólogo

Aprender programación puede parecer intimidante al inicio, especialmente si vienes del mundo de redes, infraestructura u operación y nunca has escrito código antes.

La buena noticia es que **Python fue diseñado para ser uno de los lenguajes de programación más claros, amigables y accesibles que existen.**

Actualmente, Python se utiliza en áreas como:

- Automatización.
- Redes y sistemas.
- Análisis de datos.
- Inteligencia artificial.
- Ciberseguridad.
- Desarrollo de aplicaciones.

Pero más importante aún: **Python puede convertirse en una herramienta extremadamente poderosa para tu crecimiento profesional.**

Este libro fue escrito pensando en personas que desean aprender Python de forma práctica, utilizando ejemplos y escenarios reales de automatización desde el inicio.

No necesitas experiencia previa en programación. Solo curiosidad, práctica y disposición para aprender algo nuevo.

Probablemente cometerás errores, tendrás dudas y encontrarás conceptos nuevos constantemente. Eso es completamente normal. Programar es una habilidad que se desarrolla paso a paso.

Lo importante no es memorizar todo, sino aprender a pensar, experimentar y resolver problemas utilizando código.

Si decidiste abrir este libro y dar este primer paso, ya comenzaste algo importante.

Bienvenido al Mundo Python.

Sobre el autor

Daniel Llivipuma es Ingeniero en Electrónica y Telecomunicaciones con más de 17 años de experiencia en operación, monitoreo y automatización de infraestructuras de red.

A lo largo de su carrera ha trabajado directamente con tecnologías Alcatel, Cisco, Juniper, Huawei, MikroTik y Nokia, desarrollando soluciones de observabilidad, automatización y gestión operativa utilizando Python, Linux, Grafana, SNMP, APIs, bases de datos y desarrollo web aplicado a plataformas de monitoreo y automatización.

Ha trabajado en diseño e implementación de plataformas de monitoreo y automatización capaces de supervisar miles de dispositivos de red, integrando métricas, alarmas, APIs y herramientas operativas desarrolladas en Python.

Este libro fue escrito pensando especialmente en profesionales de redes que necesitan aprender Python como una herramienta práctica para automatizar tareas reales, construir soluciones reutilizables y simplificar su trabajo diario, utilizando ejemplos y escenarios inspirados en problemas reales de operación y automatización de redes.

Este es el libro que al autor le hubiera gustado tener hace 10 años.

Sobre esta muestra extendida

Este documento corresponde a una muestra gratuita del libro:

Python para Redes: Programación y automatización con SNMP, SSH, APIs y más desde cero.

Esta edición incluye los primeros 4 capítulos completos del libro, permitiéndote conocer el enfoque práctico, la estructura y el estilo del contenido antes de adquirir la versión completa.

 **Hotmart:**

<https://hotmart.com/es/club/daniel-francisco-llivipuma-pozo/products>

 **Repositorio oficial:**

https://github.com/aniluca-publishing/python_para_redes

 **LinkedIn:**

<https://www.linkedin.com/in/dllivipuma>

¡Gracias por formar parte de este proyecto!

Contenido

Cómo pensar como programador.....	1
Cómo utilizar este libro	5
Convenciones utilizadas en este libro	7
Nota sobre seguridad y manejo de credenciales.....	9
PARTE I: Fundamentos de Python aplicados a redes.....	10
Capítulo 1 - Introducción a Python	1
1.1 ¿Qué es Python?.....	2
1.2 Instalando Python en Linux/macOS.....	5
1.3 Instalando Python en Windows	7
1.4 Instalando un IDE en Windows	10
1.5 Estructura de directorios para trabajar con este libro	17
1.6 Tu primer programa en Python.....	18
1.7 Módulos e <code>import</code>	21
1.8 Librerías externas: qué son y para qué sirven.....	24
1.9 Recomendación: Crear archivos <code>test.py</code>	25
1.10 Resumen	26
Capítulo 2 - Conceptos fundamentales de Python	27
2.1 Sintaxis básica	28
2.2 Introducción a variables.....	36
2.3 Palabras reservadas.....	38
2.4 Introducción a funciones	39
2.5 Flujo básico de ejecución en Python	42
2.6 Introducción a estructuras de control	44
2.7 Introducción a errores y excepciones.....	46
2.8 Introducción a clases y objetos	48
2.9 Buenas prácticas.....	49
2.10 Resumen	51
Capítulo 3 - Variables y tipos de datos.....	52
3.1 Variables	53
3.2 Tipos de datos básicos.....	54
3.3 Colecciones	60

3.4 Operaciones sobre variables	67
3.5 Secuencias.....	73
3.6 Iterables	84
3.7 Comparación y valores booleanos	86
3.8 Mutabilidad e inmutabilidad.....	87
3.9 Objetos hashables	90
3.10 Resumen	92
Capítulo 4 - Operaciones y manipulación de datos	93
4.1 Funciones integradas más utilizadas	94
4.2 Usando métodos	96
4.3 Métodos de strings.....	97
4.4 Métodos de listas	106
4.5 Métodos de sets.....	111
4.6 Métodos de diccionarios.....	114
4.7 Encadenando métodos	119
4.8 Trabajar con direcciones IP usando el módulo <code>ipaddress</code>	120
4.9 Ejercicio práctico: Analizando un inventario básico de dispositivos	123
4.10 Resumen	128
Capítulo 5 - Estructuras de control	130
5.1 Condicionales: <code>if</code> , <code>else</code> , <code>elif</code>	131
5.2 Operadores lógicos: <code>and</code> , <code>or</code> , <code>not</code>	132
5.3 Lazos: <code>for</code> y <code>while</code>	133
5.4 Comprehensions (comprensiones).....	134
5.5 Controlando el flujo del programa	135
5.6 Ejercicio práctico: Validando VLANs y detectando inconsistencias	136
5.7 Resumen	137
Capítulo 6 - Funciones.....	138
6.1 ¿Qué es una función?	139
6.2 ¿Cómo definir una función?.....	140
6.3 Pasando argumentos a las funciones	141
6.4 Valores de retorno.....	142
6.5 Argumentos variables: <code>*args</code> y <code>**kwargs</code>	143

6.6 Funciones recursivas.....	144
6.7 Docstrings: documentando tus funciones	145
6.8 Alcance de variables (<code>scope</code>).....	146
6.9 El bloque <code>__name__ == '__main__'</code>	147
6.10 Ejercicio práctico: Analizando un inventario básico de dispositivos, utilizando funciones	148
6.11 Ejercicio práctico: Validando VLANs y detectando inconsistencias, utilizando funciones	149
6.12 Resumen	150
Capítulo 7 - Herramientas esenciales en Python	151
7.1 Gestionando paquetes con <code>pip</code>	152
7.2 Procesamiento de datos con Python	153
7.3 Introducción a las expresiones regulares (<code>regex</code>).....	154
7.4 Interactuar con archivos y el sistema operativo	155
7.5 Manejo básico de errores (<code>try/except</code>).....	156
7.6 Trabajar con archivos CSV (datos estructurados)	157
7.7 Manejo de tiempo y fechas.....	158
7.8 Ejecutar comandos y comunicarse por la red	159
7.9 Ejercicio práctico: Analizador simple de interfaces.....	160
7.10 Resumen	161
Capítulo 8 - Manejo de Errores.....	162
8.1 ¿Por qué es importante manejar los errores?	163
8.2 Cómo interpretar los errores en la consola (<code>tracebacks</code>)	164
8.3 Debugging de <code>tracebacks</code> complejos	165
8.4 Estructura de manejo de errores en Python.....	166
8.5 La instrucción <code>raise</code>	167
8.6 Creación de excepciones personalizadas.....	168
8.7 Ejercicio práctico: Procesador robusto de inventarios	169
8.8 Resumen	170
Capítulo 9 - Clases y Objetos en Python.....	171
9.1 Programación orientada a objetos (POO).....	172
9.2 Clases y objetos	173
9.2.3 El constructor	¡Error! Marcador no definido.

9.3 Herencia.....	174
9.4 La función <code>isinstance()</code>	175
9.5 ¿Qué es importante entender en este capítulo?	176
9.6 Ejercicio práctico: Aplicando POO a redes.....	177
9.7 Resumen	178
Capítulo 10 - Leer y escribir archivos	179
10.1 Introducción al manejo de archivos	180
10.2 Rutas de archivos: relativas y absolutas	181
10.3 Leer archivos	182
10.4 Escribir en archivos.....	183
10.5 Codificación de caracteres (encoding).....	¡Error! Marcador no definido.
10.6 Leer y escribir archivos binarios	184
10.7 Manejo de errores al trabajar con archivos.....	185
10.8 Buenas prácticas al trabajar con archivos.....	186
10.9 Ejercicio práctico: Simulador de respaldo SSH.....	187
10.10 Resumen	188
Resumen de la PARTE I.....	189
Antes de continuar.....	190
1. Ejercicios de Strings y métodos básicos.....	191
2. Ejercicios de Listas y comprensión de listas.....	192
3. Ejercicios de Diccionarios y comprensión de diccionarios.....	193
4. Ejercicios de Iteración.....	194
5. Ejercicios de Condicionales	195
6. Ejercicios de Funciones.....	196
7. Ejercicios de Slicing.....	197
8. Ejercicios de Manejo básico de errores.....	198
PARTE II: Aplicando Python a problemas reales de redes.....	199
Antes de continuar.....	200
Capítulo 11 - Analizando configuraciones de red con expresiones regulares (<code>regex</code>)	201
11.1 Comprendiendo expresiones regulares (<code>regex</code>)	202
11.2 Metacaracteres esenciales en <code>regex</code>	203
11.3 La clase <code>Match</code>	204

11.4 Herramientas esenciales de <code>regex</code> en Python	205
11.5 Uso de flags	206
11.6 Grupos sin captura (?:)	207
11.7 Expresiones regulares comunes en redes.....	208
11.8 Aplicando <code>regex</code> a redes.....	209
11.9 Ejercicio práctico: Extraer VLANs de un comando "show vlan"	210
11.10 Ejercicio práctico: Extraer estado de interfaces de un show ip interface brief.....	211
11.11 Nota avanzada: compilar patrones con <code>re.compile</code> (eficiencia y diseño avanzado)	212
11.12 Resumen.....	213
Capítulo 12 - Pruebas de conectividad: ping y traceroute	214
12.1 Ping con Python.....	215
12.2 Traceroute con Python	216
12.3 Resumen	217
Capítulo 13 - Auditorías de seguridad: escaneo de puertos.....	218
13.1 Escáner de puertos con Python.....	219
13.2 Resumen	220
Capítulo 14 - SSH y SFTP: conectando a dispositivos de red	221
14.1 SSH con Python.....	222
14.2 SFTP con Python	223
14.3 Resumen	224
Capítulo 15 - Obtención y modificación de datos con SNMP	225
15.1 OIDs numéricos y OIDs con nombre.....	226
15.2 Concepto de índices en SNMP	227
15.3 Relación de índices entre distintos OIDs.....	228
15.4 Formato de comandos SNMP.....	229
15.5 SNMP GET con Python.....	230
15.6 SNMP WALK / BULKWALK con Python	231
15.7 SNMP SET con Python.....	232
15.8 Decisiones de diseño en este capítulo.....	233
15.9 Resumen	234
Capítulo 16 - Introducción práctica a APIs	235

16.1 ¿Qué es una API?	236
16.2 ¿Qué es JSON y por qué ya sabes usarlo?	237
16.3 Consumir una API desde Python (lo más básico)	238
16.4 APIs útiles para administración de redes	239
16.5 NETCONF, RESTCONF y las APIs que configuran dispositivos de red	240
16.6 Ejercicio práctico: Analizador de URLs e IPs públicas	241
16.7 Ejercicio práctico con token: Consulta básica a AbuseIPDB	242
16.8 Resumen	243
Capítulo 17 - Notificaciones automáticas: Correo (SMTP) y Slack (Webhooks)	244
17.1 Enviar correos con SMTP	245
17.2 Enviar mensajes por Slack usando Webhooks	246
17.3 Resumen	247
Capítulo 18 - Usando concurrencia en Python	248
18.1 Qué es concurrencia, y por qué importa en redes	249
18.2 ¿Qué es un thread?	250
18.3 Conceptos clave antes de continuar	251
18.4 Creando un módulo para concurrencia	252
18.5 Por qué a veces necesitamos una función <code>worker</code>	253
18.6 Ping concurrente	254
18.7 SSH concurrente	255
18.8 SNMP concurrente	256
18.9 Qué tomar en cuenta al escoger <code>max_workers</code>	257
18.10 Resumen	258
PARTE III: Automatización y proyectos reales de redes con Python	259
Introducción	260
Capítulo 19 - De script funcional a herramienta robusta	262
19.1 Gestión segura de credenciales usando variables de entorno	263
19.2 Tipado y contratos explícitos en funciones	264
19.3 Logging estructurado en lugar de <code>print()</code>	265
19.4 Estructura y organización de proyectos	266
19.5 Resumen	267
Capítulo 20 - Respaldo automático de configuraciones de red	268

20.1 Objetivos	269
20.2 Lo que tenemos	270
20.3 Paso a paso.....	271
20.4 Integración final del código.....	272
20.5 Posibles mejoras y consideraciones	273
20.6 Resumen	274
Capítulo 21 - Inventario automático de dispositivos de red	275
21.1 Objetivos	276
21.2 Lo que tenemos.....	277
21.3 Paso a paso.....	278
21.4 Integración final del código.....	279
21.5 Resumen	280
PARTE IV: Anexos técnicos	281
Introducción	282
Anexo A - Documentación Técnica de las Funciones.....	283
A.1 Funciones de conectividad	284
A.2 Funciones de seguridad.....	285
A.3 Funciones SSH/SFTP	286
A.4 Funciones SNMP	287
A.5 Funciones para interactuar con APIs.....	288
A.6 Funciones para notificaciones.....	289
A.7 Funciones para registrar logs.....	290
A.8 Funciones para concurrencia	291
Anexo B - Criterios y Buenas Prácticas.....	292
B.1 Estilo y legibilidad (PEP 8 mínimo viable)	293
B.2 Gestión segura de credenciales y variables de entorno.....	294
B.3 Estandarización de retornos de funciones.....	295
B.4 Logging y trazabilidad.....	296
B.5 Separación de responsabilidades y estructura de proyecto.....	297
B.6 Timeouts, reintentos y fallos transitorios.....	298
B.7 Validación de argumentos y errores previsibles	299
Anexo C - Ideas para proyectos futuros	300
C.1 Automatizaciones basadas en <code>execute_ssh()</code>	301

C.2 Automatizaciones basadas en <code>snmp_walk()</code> y <code>snmp_get()</code>	302
C.3 Automatizaciones basadas en <code>scan_port()</code> y <code>scan_ports()</code>	303
C.4 Automatizaciones con <code>execute_ping()</code> y <code>execute_traceroute()</code>	304
C.5 Automatizaciones basadas en <code>api_tools.py</code>	305
C.6 Automatizaciones basadas en <code>notification_tools.py</code>	306
Anexo D - Glosario Técnico	307
D.1 Redes y automatización	308
D.2 Python y programación	309
Anexo E - Soluciones a los ejercicios de la PARTE I	310
E.1 Soluciones a ejercicios de Strings y métodos básicos	311
E.2 Soluciones a ejercicios de Listas y comprensión de listas	312
E.3 Soluciones a ejercicios de Diccionarios y comprensión de diccionarios	313
E.4 Soluciones a ejercicios de Iteración	314
E.5 Soluciones a ejercicios de Condicionales	315
E.6 Soluciones a ejercicios de Funciones	316
E.7 Soluciones a ejercicios de Slicing	317
E.8 Soluciones a ejercicios de Manejo básico de errores	318
Anexo F - Introducción rápida a PyCharm	319
F.1 Interfaz básica de PyCharm	320
F.2 Trabajar con archivos en PyCharm	321
F.3 Autocompletado y ayuda inteligente	322
F.4 Advertencias y detección de errores	323
F.5 Navegación y edición rápida	324
F.6 Funciones útiles adicionales de PyCharm	325

Cómo pensar como programador

Antes de aprender a escribir código, es necesario aprender a **pensar correctamente sobre los problemas**.

Uno de los errores más comunes al comenzar a programar es creer que programar consiste en abrir un editor y empezar a escribir líneas de código hasta que algo **funcione**.

En entornos reales, ese enfoque casi siempre termina en frustración, soluciones frágiles y scripts imposibles de mantener.

Programar no es escribir código.

Programar es **resolver problemas de forma estructurada**.

En este libro no solo aprenderás Python. Aprenderás el proceso mental que usaremos de forma consistente para construir soluciones reales en automatización de redes.

Programar es resolver problemas

Un lenguaje de programación no entiende intenciones, ideas ni contextos. Solo ejecuta instrucciones muy concretas, en un orden preciso.

Por eso, el verdadero trabajo del programador ocurre **antes** de escribir la primera línea de código.

Cuando un problema parece difícil, casi nunca se debe a la sintaxis. Normalmente ocurre porque:

- El objetivo no está claramente definido.
- No se tiene claridad sobre la información disponible.
- El problema no ha sido dividido en pasos simples.

Python no resuelve eso por ti.

Ese trabajo te corresponde a ti como programador.

Qué significa pensar como programador

Pensar como programador significa adoptar hábitos muy concretos:

- Definir claramente qué se quiere lograr.
- Identificar con qué información y recursos se cuenta.
- Describir los pasos necesarios para llegar al resultado.
- Solo entonces, traducir esos pasos a código.

Este enfoque no es exclusivo de Python. Es la misma forma de pensar que utilizan quienes automatizan redes, sistemas, procesos de negocio o análisis de datos en el mundo real.

Con el tiempo descubrirás que los programadores más efectivos no son necesariamente quienes escriben código más rápido, sino quienes entienden mejor el problema antes de empezar.

Muchas veces, dedicar unos minutos a pensar correctamente la lógica ahorra horas de errores, reescrituras y depuración innecesaria.

Precisamente por eso, antes de profundizar en Python, primero vamos a entender cómo descomponer problemas y construir soluciones paso a paso.

El flujo que usaremos en todo el libro

Los ejemplos importantes del libro seguirán siempre el mismo esquema:

- **Objetivos**
Qué queremos lograr exactamente.
- **Lo que tenemos**
Qué información, recursos y limitaciones existen.
- **Paso a paso**
Qué pasos son necesarios para alcanzar el objetivo.
- **Código final y resultados**
La implementación completa en Python.

Este flujo no es teórico.

Es una herramienta práctica para evitar improvisación y construir soluciones claras y mantenibles.

Objetivos: Definir bien las metas

Un buen objetivo debe ser **claro y verificable**.

Mal objetivo:

- “Hacer un script de respaldos”.

Buen objetivo:

- “Crear un script que se conecte por SSH a una lista de dispositivos y guarde su configuración en archivos separados”.

Si no puedes describir el objetivo en una o dos frases claras, todavía no estás listo para escribir código.

Lo que tenemos: Entender con qué contamos

Antes de pensar en soluciones, es fundamental listar lo que realmente tenemos:

- Archivos (CSV, JSON, texto).
- Direcciones IP.
- Credenciales.
- Comandos disponibles.
- Restricciones reales: tiempo, permisos, posibles errores.

Este paso evita suposiciones falsas.

En automatización real, asumir que *todo va a salir bien* es una de las causas más comunes de fallas.

Paso a paso: Descomponer el problema

Por ejemplo:

- Leer el archivo con la lista de dispositivos.
- Conectarse a cada dispositivo.
- Ejecutar los comandos necesarios.
- Guardar la salida en archivos.
- Manejar errores de conexión.

Si no puedes escribir estos pasos en lenguaje natural, el código tampoco va a salir bien.

A partir de estos pasos, puedes desarrollar el código de forma progresiva.

Código final y resultados

Primero definimos el objetivo y los pasos. Luego construimos el código en partes. Al final, presentamos la versión completa y ordenada.

El código es simplemente la traducción de los pasos anteriores a instrucciones que el computador puede ejecutar.

En este punto final ya tienes todo completo y puedes verificar que los resultados sean los mismos definidos como objetivos.

Un error común (y cómo evitarlo)

Un error típico es empezar escribiendo código y luego ajustar el objetivo sobre la marcha. Eso produce scripts difíciles de entender, modificar o reutilizar.

La solución es simple: respeta el flujo.

Si algo no funciona, vuelve atrás:

- ¿El objetivo estaba bien definido?
- ¿Faltaba información?
- ¿Algún paso no estaba claro?

Depurar no es solo corregir errores en el código. Es corregir errores en el razonamiento.

Qué debes recordar

No necesitas memorizar este capítulo. Necesitas **usarlo**.

Cada vez que empieces un nuevo ejemplo o proyecto en este libro, pregúntate:

- ¿Cuál es el objetivo?
- ¿Qué es lo que realmente tengo?
- ¿Cuáles son los pasos necesarios?
- ¿Recién ahora escribo el código?

Si adoptas esta forma de pensar, no solo aprenderás Python.

Aprenderás a resolver problemas de forma sistemática, que es la habilidad más valiosa en automatización.

Cómo utilizar este libro

Todos los ejemplos de este libro han sido probados y ejecutados.

Dependiendo del visor de PDF que utilices, es posible que la sangría del código no se muestre exactamente igual a como fue escrita originalmente. Esto es una limitación conocida de algunos lectores PDF y no afecta el funcionamiento del código.

Para una mejor experiencia, así como para acceder a versiones actualizadas de los ejemplos, se recomienda utilizar el repositorio oficial del libro en GitHub:

https://github.com/aniluca-publishing/python_para_redes

Este libro fue diseñado para ser trabajado de forma progresiva.

PARTE I – Fundamentos de Python

Aprenderás las bases necesarias para programar en Python desde cero:

- Instalación y entorno de trabajo.
- Variables y tipos de datos.
- Strings, listas, diccionarios y métodos.
- Condicionales y lazos.
- Funciones.
- Manejo de errores.
- Expresiones regulares (**regex**).
- Clases y objetos.
- Lectura y escritura de archivos.

PARTE II – Automatización y redes

Aplicarás Python a tareas reales de automatización y operación de redes:

- Análisis de configuraciones con **regex**.
- Ping y traceroute.
- Escaneo de puertos.
- SSH y SFTP.
- Obtención y modificación de datos con SNMP.
- APIs y procesamiento de datos externos.
- Notificaciones automáticas.
- Concurrencia y automatización masiva.

PARTE III – Proyectos integrados

Integrarás lo aprendido en herramientas más completas y estructuradas:

- Gestión segura de credenciales.
- Logging y organización de proyectos.
- Respaldo automático de configuraciones e inventario automático de dispositivos.

Además, el libro incluye anexos técnicos que pueden utilizarse como material de consulta rápida durante el aprendizaje:

- [ANEXO A](#): Documentación técnica resumida de las funciones desarrolladas en el libro.
- [ANEXO B](#): Buenas prácticas y criterios utilizados en los ejemplos y proyectos.
- [ANEXO C](#): Ideas para extender lo aprendido hacia automatizaciones más avanzadas.
- [ANEXO D](#): Glosario técnico de Python, redes y automatización.
- [ANEXO E](#): Soluciones de ejercicios de la **PARTE I**.
- [ANEXO F](#): Introducción rápida a PyCharm y herramientas básicas del entorno de desarrollo.

No es necesario memorizar todo en la primera lectura. Python y la automatización se aprenden principalmente practicando, ejecutando ejemplos y construyendo proyectos progresivamente.

Convenciones utilizadas en este libro

Para mantener los ejemplos simples y consistentes, este libro utiliza algunas convenciones que es importante entender desde el inicio.

✚ Nombres de dispositivos (hostnames)

Se utilizan nombres como:

```
R1-GYE, R2-UIO, R1-CUE
```

Estos nombres siguen una estructura simple:

- **R1, R2:** Identificador del equipo (por ejemplo, Router 1, Router 2).
- **GYE, UIO, CUE:** Código de la ciudad en la que se encuentra el equipo.

Algunos códigos de ciudad utilizados:

- **GYE:** Guayaquil
- **UIO:** Quito
- **CUE:** Cuenca
- **LOJ:** Loja
- **PVJ:** Portoviejo

Algunos tipos de dispositivos utilizados:

- **FW:** Firewall
- **R:** Router
- **SW:** Switch
- **SRV:** Servidor

Estos nombres son únicamente ejemplos didácticos y no afectan la lógica del código.

✚ Direcciones IP utilizadas

Las direcciones IP utilizadas en los ejemplos pertenecen a rangos privados (por ejemplo: 10.x.x.x) y se utilizan únicamente con fines de laboratorio y aprendizaje.

En entornos reales, los nombres de dispositivos y los esquemas de direccionamiento pueden variar dependiendo de la organización y el diseño de la red.

✚ Datos simplificados

En los ejemplos:

- Los inventarios son pequeños.
- Los datos están controlados.
- Las estructuras son simples.

En entornos reales, los datos pueden ser más complejos, inconsistentes o incompletos. A lo largo del libro aprenderás a manejar estos escenarios.

🔗 Idioma utilizado en el código

Aunque el contenido del libro está escrito en español, los ejemplos de código utilizan nombres de variables, funciones, archivos y módulos en inglés.

Esto se hace por varias razones:

- Python utiliza palabras clave en inglés, como `if`, `else`, `for`, `while`, `def`, `return`, `try` y `except`.
- La mayoría de librerías, documentación técnica y ejemplos profesionales también utilizan inglés.
- Usar nombres en inglés facilita la lectura del código junto con herramientas, documentación y buenas prácticas del ecosistema Python.
- Mantiene el código más alineado con estándares comunes de desarrollo.

Por ejemplo, en lugar de escribir:

```
nombre_dispositivo = 'R1-GYE'  
estado_interfaz = 'UP'
```

En este libro se prefiere:

```
hostname = 'R1-GYE'  
interface_status = 'UP'
```

Esto no implica que sea incorrecto utilizar nombres en español. Sin embargo, para mantener consistencia, claridad técnica y alineación con las prácticas habituales del ecosistema Python, en este libro se utilizarán nombres en inglés.

🔗 Valores de estado utilizados en los ejemplos

En varios ejercicios se utilizan valores simples para representar estados:

```
1: Operativo  
0: Caído o inactivo  
-1: Desconocido, inválido o no disponible
```

Estos valores son convenciones didácticas usadas para simplificar los ejemplos. En entornos reales, los sistemas pueden usar otros códigos, textos o estructuras más complejas.

Nota sobre seguridad y manejo de credenciales

A lo largo de este libro encontrarás ejemplos que incluyen usuarios, contraseñas, comunidades SNMP, tokens o claves directamente dentro del código.

Esto se hace únicamente con fines didácticos.

Durante las primeras partes nos enfocamos en comprender la lógica de programación, la estructura de los scripts y el funcionamiento de los protocolos, sin añadir complejidad adicional que distraiga de los conceptos fundamentales.

Sin embargo, es importante dejar algo claro:

No es una práctica segura almacenar credenciales en texto plano dentro del código en entornos reales.

En escenarios profesionales, la configuración y las credenciales deben separarse del código, comúnmente mediante el uso de variables de entorno u otros mecanismos de configuración externa.

En la **PARTE III** del libro aprenderás cómo implementar correctamente este enfoque.

Por ahora, el uso de credenciales en texto plano tiene un propósito pedagógico: facilitar el aprendizaje progresivo sin introducir complejidad innecesaria.

A medida que avances, el enfoque evolucionará hacia prácticas más profesionales.

PARTE I:

Fundamentos de Python aplicados a redes

Capítulo 1 - Introducción a Python

Python es uno de los lenguajes más utilizados actualmente en automatización, desarrollo de software, análisis de datos, inteligencia artificial y redes.

Su sintaxis clara y su enorme ecosistema de librerías lo convierten en una herramienta ideal para automatizar tareas repetitivas y construir soluciones escalables de forma relativamente simple.

En este capítulo prepararemos el entorno de trabajo que utilizaremos a lo largo del libro y comenzaremos a familiarizarnos con la forma en que Python ejecuta programas.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Comprender qué es Python y por qué se utiliza tanto en automatización.
- Instalar Python y preparar tu entorno de trabajo.
- Ejecutar tus primeros programas en Python.
- Importar y utilizar módulos básicos.
- Diferenciar módulos estándar y librerías externas.

1.1 ¿Qué es Python?

Python es un lenguaje de programación de alto nivel, interpretado y multipropósito, diseñado para ser claro, legible y fácil de aprender. Fue creado con la idea de permitir que las personas se concentren más en resolver problemas reales y menos en la complejidad del lenguaje.

Esto significa que Python ejecuta el código mediante un intérprete, sin necesidad de compilar manualmente el programa antes de ejecutarlo.

En otros lenguajes, conocidos como lenguajes compilados, normalmente es necesario transformar primero el código fuente en un archivo ejecutable antes de poder ejecutar el programa. Python simplifica este proceso, permitiendo desarrollar y probar código de forma más rápida e interactiva.

Gracias a su sintaxis simple y expresiva, Python se ha convertido en uno de los lenguajes de programación más utilizados del mundo, tanto por personas que recién comienzan como por grandes empresas tecnológicas, ingenieros, científicos y especialistas en automatización.

Actualmente, Python se utiliza en áreas como:

- Automatización de redes y sistemas.
- Desarrollo web.
- Inteligencia artificial y machine learning.
- Análisis de datos.
- Ciberseguridad.
- Desarrollo de APIs y aplicaciones.

Una de las razones de su popularidad es que permite crear soluciones reales con relativamente poco código. Muchas tareas que en otros lenguajes requieren decenas de líneas complejas, en Python pueden resolverse de forma mucho más simple y legible.

Por ejemplo, más adelante en este libro podrás:

- Conectarte por SSH a routers y switches.
- Obtener métricas mediante SNMP.
- Consumir APIs REST.
- Procesar archivos y logs.
- Automatizar respaldos.
- Analizar configuraciones de red.
- Crear herramientas reutilizables para operación y monitoreo.

Todo esto utilizando un lenguaje cuya sintaxis es cercana al inglés y mucho más fácil de leer que la de muchos otros lenguajes tradicionales.

Python también se destaca por su enorme ecosistema de librerías. Existen miles de herramientas ya desarrolladas que permiten extender fácilmente las capacidades del lenguaje. Gracias a esto, Python no solo es fácil de aprender, sino también extremadamente práctico en entornos reales.

Otro punto importante es su legibilidad. Python fue diseñado con la idea de que el código debe ser claro y fácil de entender. Por eso utiliza sangría (espacios en blanco) para estructurar bloques de código, promoviendo automáticamente un estilo más limpio y mantenible.

Esto permite que el programador se enfoque más en resolver problemas reales y menos en lidiar con complejidades innecesarias del lenguaje.

1.1.1 Versiones de Python

Este libro está basado en **Python 3**.

Python evoluciona constantemente, y dentro de **Python 3** existen múltiples subversiones, por ejemplo:

- **Python 3.9**
- **Python 3.10**
- **Python 3.11**

Estas versiones pueden introducir mejoras, optimizaciones o nuevas funcionalidades. Sin embargo, los conceptos fundamentales que aprenderás en este libro funcionan prácticamente igual entre versiones modernas de **Python 3**.

1.1.2 Archivos Python y extensión `.py`

Los archivos de Python normalmente utilizan la extensión:

```
.py
```

Por ejemplo:

```
example.py
backup_routers.py
snmp_monitor.py
```

La extensión `.py` le indica al sistema operativo y a las herramientas de desarrollo que el archivo contiene código escrito en Python.

Aunque técnicamente un archivo Python podría tener otro nombre o extensión, utilizar `.py` es el estándar utilizado prácticamente en todos los proyectos y herramientas del ecosistema Python.

1.1.3 Ejecutar programas en Python

Un programa es simplemente un conjunto de instrucciones escritas para que el computador realice una tarea específica. Por ejemplo, un programa puede:

- Mostrar información en pantalla.
- Conectarse a un router.
- Analizar un archivo.
- Automatizar respaldos.
- Monitorear dispositivos de red.

En Python, estas instrucciones normalmente se guardan dentro de archivos de texto con extensión `.py`.

También es común que estos archivos sean llamados **scripts Python**. En la práctica, para este libro puedes pensar que un script es simplemente un programa escrito en Python.

Muchas veces el término script se utiliza para programas pequeños o tareas de automatización, aunque técnicamente sigue siendo un programa.

Cuando ejecutas un programa en Python, lo que realmente ocurre es que Python lee el archivo `.py`, interpreta el código y ejecuta las instrucciones contenidas dentro del script.

La forma general de ejecutar un archivo Python es:

```
python3 nombre_del_archivo.py
```

Por ejemplo:

```
python3 example.py
```

En algunas instalaciones de Windows también puedes utilizar:

```
py example.py
```

Para que este comando funcione, el archivo `example.py` debe encontrarse dentro del directorio actual desde donde estás ejecutando el comando.

Caso contrario, será necesario indicar la ruta completa hacia el archivo.

Por ejemplo, en Linux:

```
python3 /home/user/python_networking/example.py
```

O en Windows:

```
python3 C:\Users\usuario\python_networking\example.py
```

Más adelante aprenderemos a trabajar con rutas y directorios con mayor detalle y veremos que los IDEs como PyCharm también ejecutan internamente comandos similares, pero de forma gráfica y automatizada.

1.2 Instalando Python en Linux/macOS

Antes de comenzar a escribir y ejecutar programas en Python, necesitamos verificar si Python ya está disponible en el sistema o instalarlo en caso contrario.

Python es gratuito, de código abierto y compatible con Linux, macOS y Windows.

En muchos sistemas Linux y macOS, Python ya viene instalado o parcialmente integrado con el sistema operativo, aunque la versión puede variar dependiendo de la distribución o versión utilizada.

En esta sección verificaremos si Python ya se encuentra disponible y, de ser necesario, veremos cómo instalarlo de forma básica para comenzar a trabajar con los ejemplos del libro.

Si vas a utilizar Windows, puedes omitir esta sección y continuar con la siguiente.

1.2.1 Linux

En la mayoría de las distribuciones de Linux, Python ya viene instalado de forma predeterminada. Para verificar si lo tienes y qué versión es, abre la terminal y escribe:

```
python3 -V
```

Deberías ver una respuesta similar a:

```
Python 3.10.12
```

Instalación básica

Si el sistema te indica que el comando no existe, instálalo con el gestor de paquetes. Por ejemplo, en Ubuntu/Debian:

```
sudo apt update  
sudo apt install python3
```

 **Nota importante:** `apt install python3` instalará la versión que se considere estable y estándar para el sistema, la cual puede no ser la última versión lanzada oficialmente por Python.

Actualizaciones de seguridad

Para asegurarte de tener los últimos parches de seguridad de la versión que ya tienes instalada, utiliza:

```
sudo apt update  
sudo apt upgrade python3
```

Finalmente, confirma que todo esté en orden verificando la versión nuevamente:

```
python3 -V
```

1.2.2 macOS

Al igual que en Linux, macOS suele incluir una versión de Python instalada por el sistema. Sin embargo, para desarrollo profesional, lo más recomendable es instalar una versión gestionada por el usuario.

Para verificar si ya tienes Python instalado, abre la **Terminal** (puedes buscarla en **Spotlight** con **Cmd + Espacio**) y escribe:

```
python3 -V
```

Deberías ver una respuesta similar a:

```
Python 3.10.12
```

Instalación básica

Si necesitas instalarlo o quieres una versión más reciente, el método estándar en macOS es utilizar **Homebrew**, el gestor de paquetes más popular para Mac:

- Instala Homebrew (si no lo tienes) desde brew.sh o pegando su comando de instalación en la terminal.
- Instala Python:

```
brew install python
```

 **Nota importante:** A diferencia de Linux, al usar `brew install python`, Homebrew siempre intentará instalar la versión estable **más reciente** disponible oficialmente, no la del sistema operativo.

Actualizaciones

Para mantener Python y todos tus paquetes de Homebrew al día con las últimas mejoras y parches de seguridad, solo necesitas ejecutar:

```
brew update  
brew upgrade python
```

Finalmente, confirma la instalación verificando la versión:

```
python3 -V
```

1.3 Instalando Python en Windows

En Windows, Python puede instalarse mediante el **Python Install Manager**, que ahora se distribuye como **MSIX**.

MSIX es un formato moderno de instalación utilizado por Windows para distribuir aplicaciones de forma más segura, organizada y fácil de actualizar. En la práctica, funciona como un instalador, similar a un **.exe** o **.msi**, pero con un modelo más moderno de administración e integración con el sistema operativo.

Según fuentes oficiales, esta es la herramienta recomendada para instalar y administrar versiones de Python en Windows.

En equipos o versiones anteriores de Windows también puedes encontrar instaladores tradicionales. El objetivo práctico es el mismo: dejar Python correctamente instalado y disponible desde la terminal.

Puedes encontrar diferentes instaladores y versiones anteriores en:

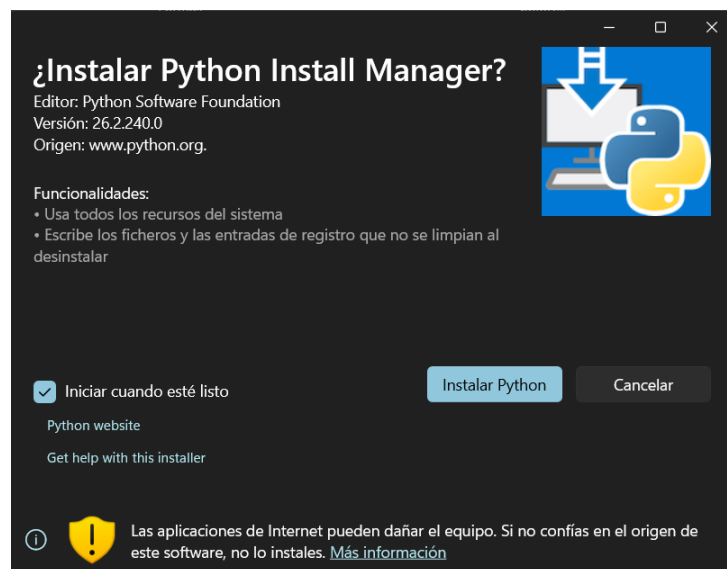
<https://www.python.org/downloads/windows>

Instalando con Python Install Manager

Descarga el archivo desde la URL oficial de Python, por ejemplo:

<https://www.python.org/ftp/python/pymanager/python-manager-26.2.msix>

Al ejecutarlo, deberás ver una pantalla similar a:



El instalador puede consultarte si deseas modificar configuraciones internas. Para este libro, puedes dejar las opciones por defecto y continuar presionando **ENTER** en cada paso:

```

C:\Program Files\WindowsAp x + v
Python and some other apps can exceed this limit, but it requires changing a
system-wide setting, which may need an administrator to approve, and will
require a reboot. Some packages may fail to install without long path support
enabled.
Update setting now? [y/N]

*****
The global shortcuts directory is not configured.

Configuring this enables commands like python3.14.exe to run from your
terminal, but is not needed for the python or py commands (for example, py
-V:3.14).

We can add the directory (C:\Users\analul\AppData\Local\Python\bin) to PATH now,
but you will need to restart your terminal to use it. The entry will be removed
if you run py uninstall --purge, or else you can remove it manually when
uninstalling Python.
Add commands directory to your PATH now? [y/N] y
PATH has been updated, and will take effect after opening a new terminal.

*****
You do not have the latest Python runtime.

Install the current latest version of CPython? If not, you can use 'py install
default' later to install.
Install CPython now? [Y/n]

```

Una vez finalizada la instalación, abre una terminal (**cmd** o **PowerShell**) y ejecuta:

```
python3 -V
```

Deberías ver una respuesta similar a:

```
Python 3.14.1
```

🔧 Instalando con instaladores tradicionales

Dependiendo de tu versión de Windows, también puedes encontrar instaladores tradicionales con extensión **.exe**.

Aunque visualmente son diferentes, el proceso general es prácticamente el mismo:

- Descargar el instalador.
- Ejecutarlo.
- Completar el asistente.
- Verificar que Python funcione correctamente desde la terminal.

Durante la instalación, es importante habilitar la opción **Add Python to PATH**:



Esto permitirá ejecutar comandos de Python desde cualquier directorio del sistema, sin necesidad de navegar hasta el directorio donde Python fue instalado:

```
python  
python3  
py
```

¿Qué es PATH?

PATH es una variable del sistema operativo que contiene una lista de directorios donde se buscan programas ejecutables.

Cuando escribes un comando como:

```
python3
```

Windows, Linux o macOS revisan automáticamente las rutas definidas en **PATH** para encontrar el ejecutable correspondiente.

Si Python no fue agregado al **PATH**, normalmente aparecerá un error indicando que el comando no existe o no fue encontrado.

Por eso, en instalaciones tradicionales, es importante habilitar la opción Add Python to **PATH** durante la instalación.

Verificando la instalación

Finalmente, confirma que Python quedó correctamente instalado ejecutando alguno de estos comandos:

```
python -V  
python3 -V  
py -V
```

Si todo está correcto, deberías ver la versión instalada de Python:

```
Python 3.X.Y
```


1.4 Instalando un IDE en Windows

Aunque puedes escribir código Python en cualquier editor de texto, lo más recomendable es usar un **IDE (Integrated Development Environment)**. Un IDE facilita el trabajo al ofrecer:

- Resaltado de sintaxis.
- Autocompletado inteligente.
- Detección de errores.
- Ejecutar scripts con un clic.
- Organización automática de archivos y directorios.
- Terminal integrada.

Para este libro, te recomiendo utilizar **PyCharm**, desarrollado por JetBrains. Se trata de un IDE muy completo, diseñado específicamente para trabajar con Python.

PyCharm facilita significativamente el proceso de escritura de código gracias a funcionalidades como el autocompletado inteligente, la detección de errores en tiempo real y herramientas integradas para depuración. Además, permite organizar proyectos de manera estructurada, lo cual resulta especialmente útil a medida que los scripts crecen en complejidad.

Otra ventaja importante es su integración con entornos virtuales, lo que te permitirá gestionar dependencias de forma ordenada y evitar conflictos entre proyectos. Asimismo, cuenta con soporte integrado para control de versiones, lo que facilita el seguimiento de cambios en el código.

Todo esto hace que escribir y ejecutar código sea mucho más cómodo, especialmente cuando recién estás comenzando.

Aunque existen otras alternativas válidas, PyCharm ofrece una experiencia sólida y consistente, especialmente adecuada para quienes están comenzando y desean enfocarse en aprender Python sin preocuparse por configuraciones adicionales.

i Nota editorial: PyCharm es una marca registrada de JetBrains s.r.o.

1.4.1 Descargar PyCharm

Puedes descargar PyCharm desde el sitio oficial de JetBrains:

<https://www.jetbrains.com/pycharm/download/other.html>

PyCharm es uno de los IDEs más utilizados para desarrollar en Python y está disponible para Windows, macOS y Linux.

Actualmente, JetBrains distribuye PyCharm como un producto unificado. Al instalarlo, obtendrás automáticamente una prueba gratuita de 30 días con funciones avanzadas (**PyCharm Pro**).

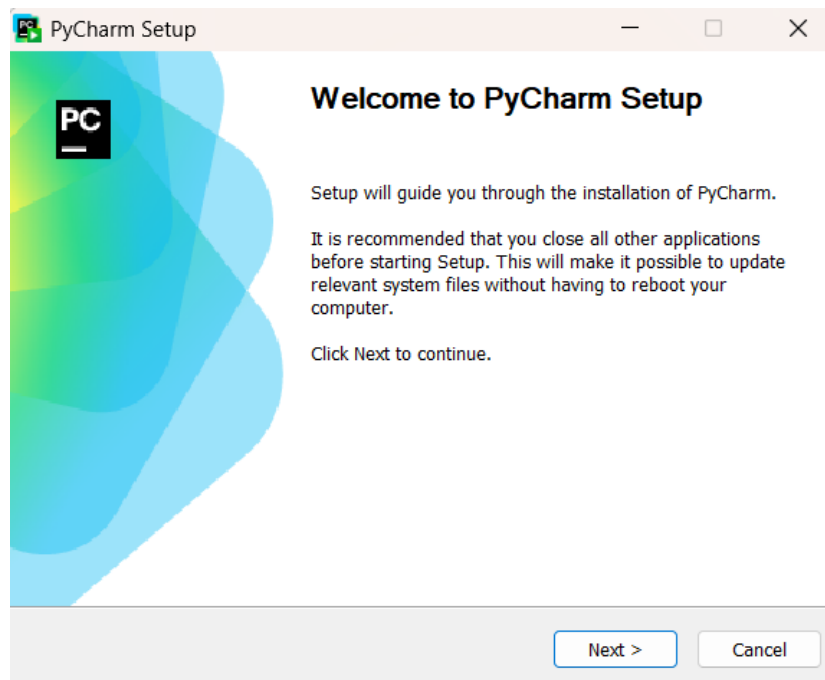
Una vez finalizada la prueba, puedes:

- Continuar usando gratuitamente las funciones principales de **PyCharm**.
- O adquirir una suscripción **Pro** si necesitas herramientas más avanzadas.

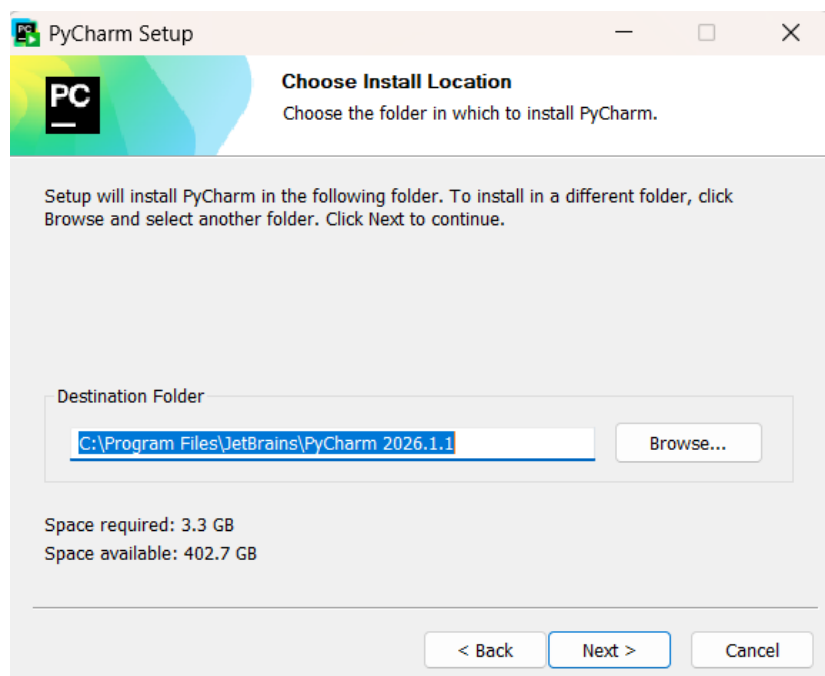
Para este libro, las funciones gratuitas son más que suficientes.

1.4.2 Instalar PyCharm

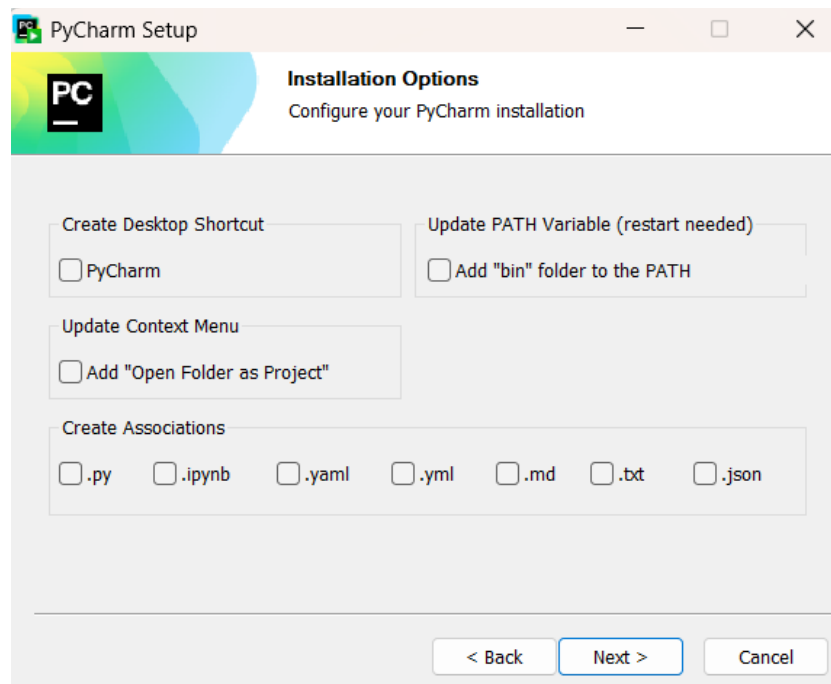
Al ejecutar el instalador deberás ver una pantalla similar a la siguiente. Presiona **Next** para continuar con el proceso de instalación:



En la pantalla de **Destination Folder** puedes cambiar el directorio donde se va a instalar **PyCharm**. Puedes cambiar el directorio según tu preferencia. Para continuar, presiona **Next**:

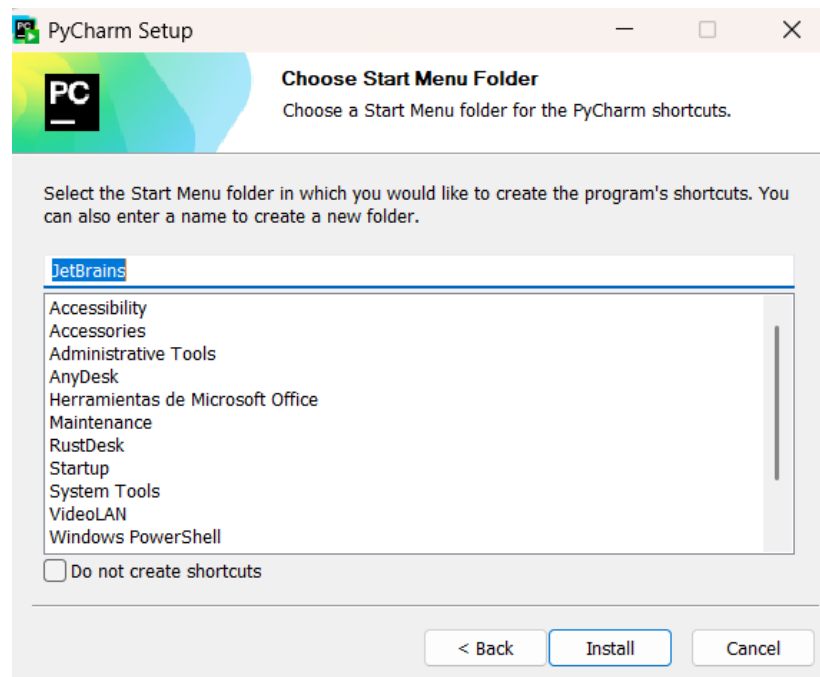


En **Installation Options** puedes dejar todo por defecto, pero te recomiendo agregar el ícono en Escritorio (selecciona **Create Desktop Shortcut**). La opción **Add "bin" folder to the PATH** no es indispensable para este libro, porque ejecutaremos **PyCharm** desde su interfaz gráfica:



En la opción de **Choose Start Menu Folder** puedes dejar por defecto o escoger el directorio donde quieras que se muestren las aplicaciones de JetBrains en el **Start Menu** de Windows.

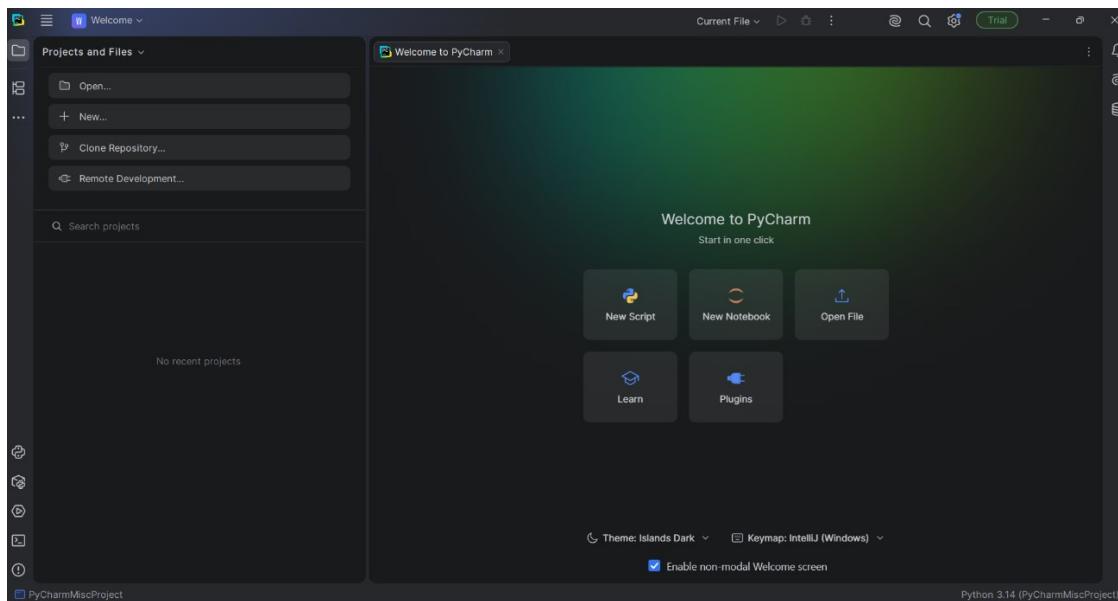
Para empezar con la instalación, presiona **Install**:



Al completar el proceso, deberás ver la pantalla de **Completing PyCharm Setup**. Selecciona **Run PyCharm** y presiona **Finish**.

Al iniciar PyCharm por primera vez, acepta los términos de uso. Si aparece la pantalla de **Data Sharing**, puedes seleccionar **Don't Send** si prefieres no enviar estadísticas anónimas. Esto no afecta el funcionamiento de PyCharm.

Finalmente, después de todo el proceso podrás ver la siguiente pantalla:



1.4.3 Crear el proyecto `python_networking`

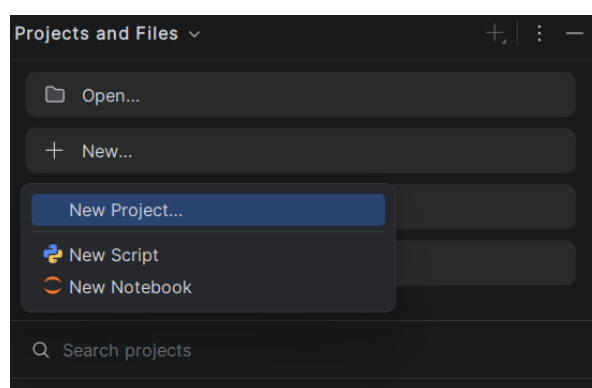
La mayoría de IDEs modernos trabajan utilizando proyectos. Un proyecto es simplemente un directorio organizado que contiene archivos, configuraciones y recursos relacionados con un trabajo específico.

Por ejemplo, podrías tener un proyecto para automatización de una red corporativa y otro completamente distinto para desarrollar herramientas personales o laboratorios de prueba, manteniendo separados y organizados los archivos, librerías y configuraciones de cada proyecto.

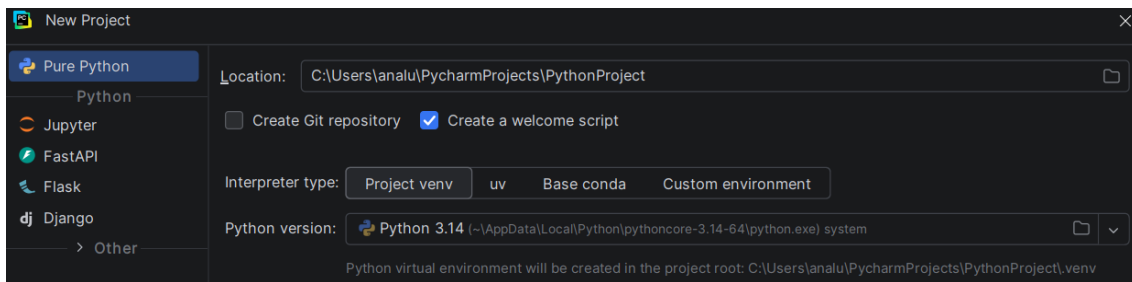
Para este libro utilizaremos un proyecto llamado:

```
python_networking
```

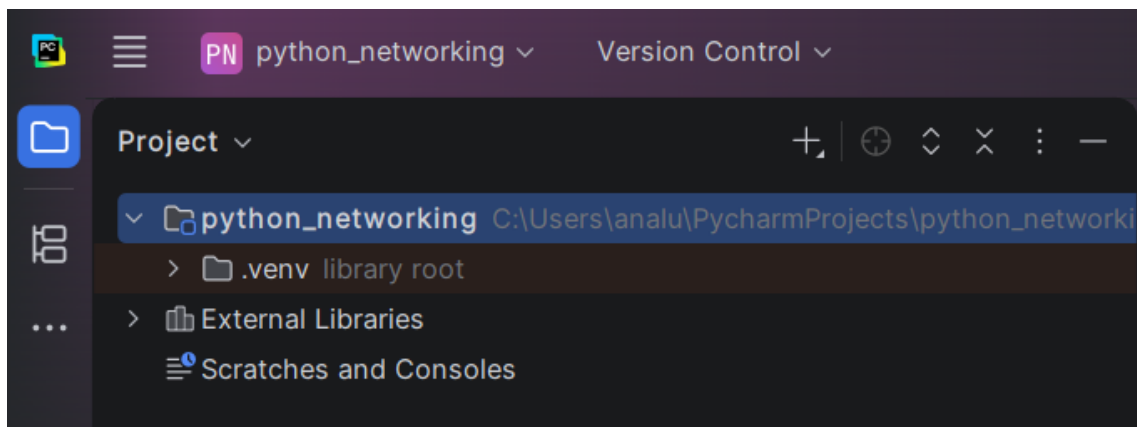
En la sección de **Projects and Files** (esquina superior izquierda) haz clic en **New** y luego en **New Project**:



Crea el proyecto con el nombre `python_networking` y presiona **Create**:

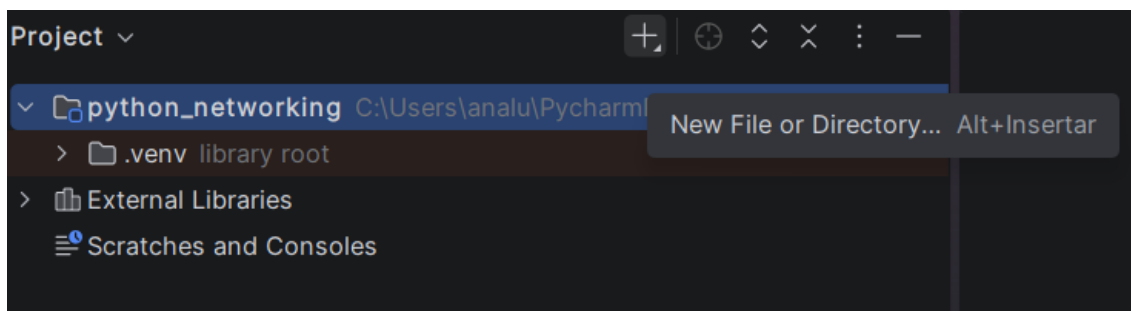


Ahora debería aparecer el nombre del proyecto en la sección **Project**:

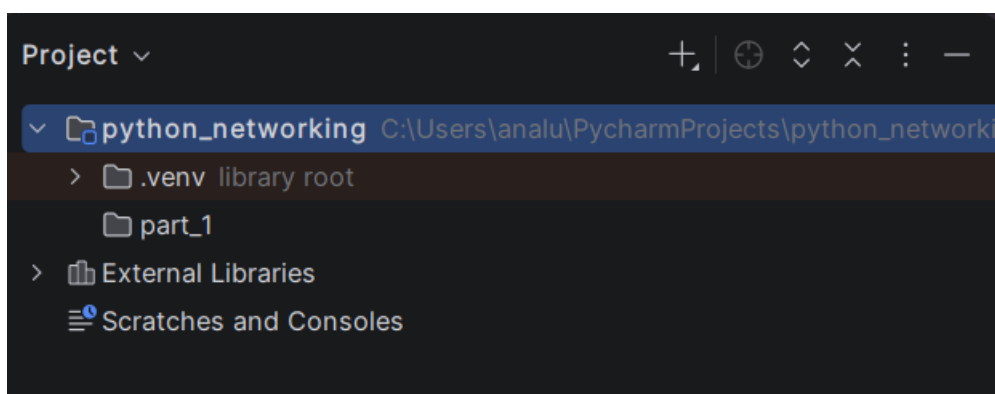


⚠ Nota importante: Como puedes observar, PyCharm crea automáticamente un **environment** o **entorno virtual** (directorio **.venv**). Esto permite mantener las librerías y configuraciones de cada proyecto separadas y organizadas, sin afectar otros proyectos del sistema.

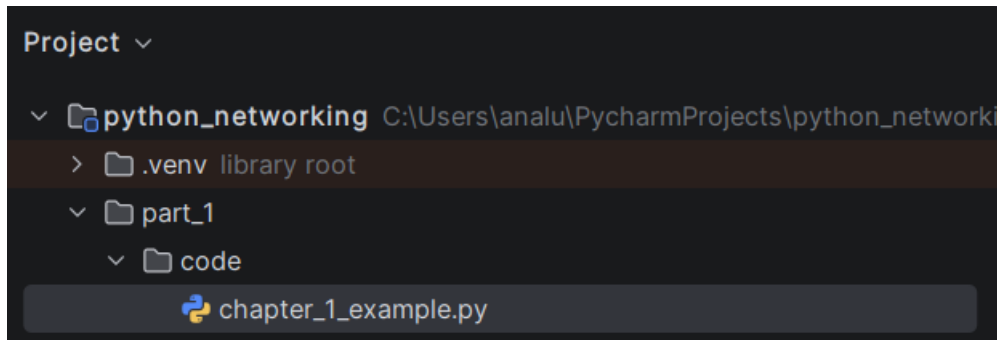
A continuación, haz clic en **python_networking** y en el signo + para crear un nuevo directorio, llamado **part_1**:



Vas a poder observar que se creó **part_1**, dentro de **python_networking**:



Vamos a crear ahora dentro de `part_1` otro directorio llamado `code`. Dentro de `code` vamos a crear un archivo llamado `chapter_1_example.py` para llegar a la siguiente estructura:



El archivo `chapter_1_example.py` es el que vas a utilizar en los ejemplos de este capítulo para que puedas empezar a utilizar Python.

1.4.4 Configurar español como lenguaje natural

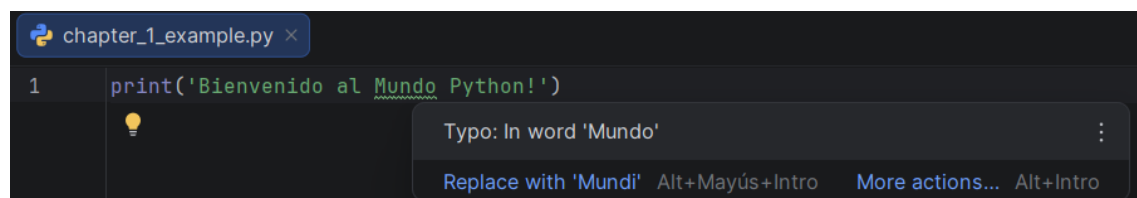
Una de las características de los IDEs modernos es que pueden validar automáticamente la ortografía y detectar posibles errores de escritura dentro del código y los comentarios.

Por defecto, PyCharm normalmente utiliza diccionarios en inglés. Debido a esto, algunas palabras en español pueden aparecer marcadas como errores ortográficos, incluso cuando están correctamente escritas.

Por ejemplo, si escribimos el siguiente código:

```
print('Bienvenido al Mundo Python!')
```

Es posible que PyCharm marque la palabra **Mundo** como un supuesto error ortográfico:



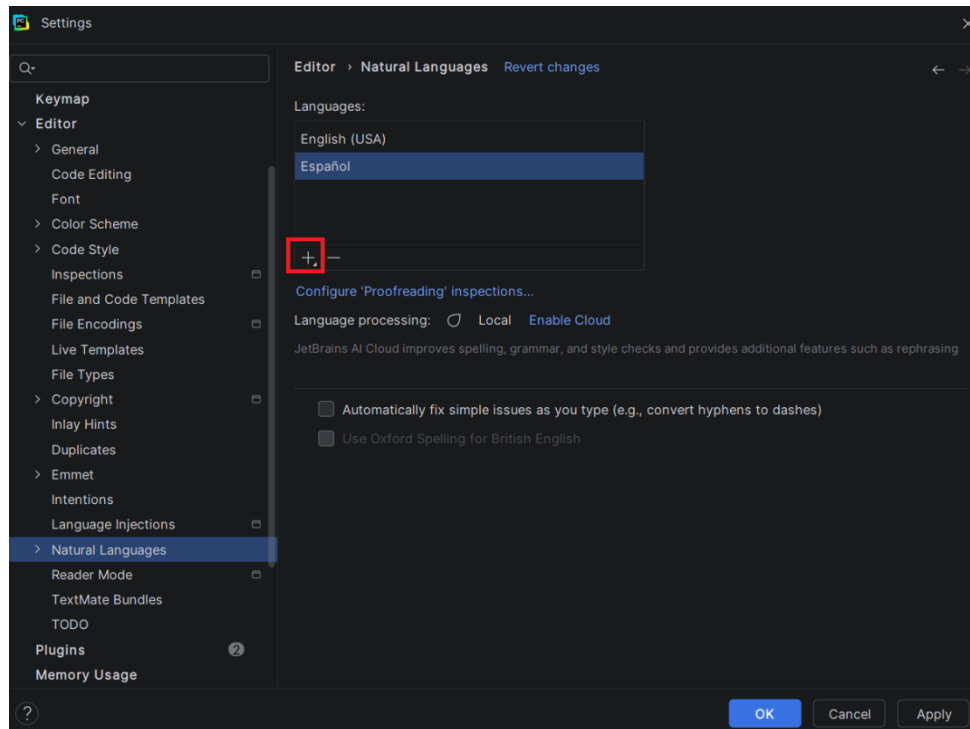
Para evitar estos falsos positivos, vamos a agregar el idioma español a la configuración de PyCharm.

En la esquina superior izquierda, haz clic en los tres puntos y luego selecciona **File -> Settings**.

Alternativamente, puedes abrir la configuración utilizando:

```
CTRL + ALT + S
```

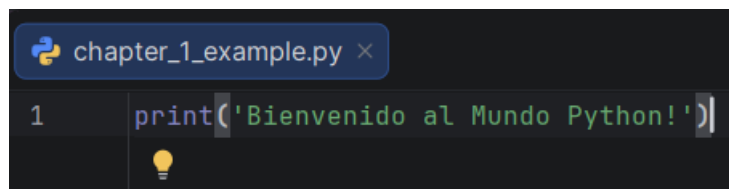
Dentro de la ventana de configuración, selecciona **Editor -> Natural Languages**:



Dentro del cuadro **Languages**, haz clic en el botón **+** y selecciona **Español**.

Finalmente, presiona **OK** para guardar los cambios.

A partir de este momento, PyCharm reconocerá palabras en español y dejará de mostrar advertencias ortográficas innecesarias para muchos términos utilizados en comentarios, strings o documentación del código.



1.4.5 Guía rápida de PyCharm

En esta sección solamente vimos la instalación y configuración inicial de PyCharm para comenzar a trabajar con Python.

El objetivo principal de este capítulo es aprender Python, no aprender todas las funciones del IDE. Por esta razón, se recomienda revisar también el [Anexo F](#), donde encontrarás una guía rápida con herramientas útiles de PyCharm, incluyendo:

- Autocompletado.
- Detección de errores y advertencias.
- Instalación gráfica de paquetes.
- Refactorización básica.
- Herramientas del editor y navegación entre archivos.

Estas herramientas no son obligatorias para aprender Python, pero sí pueden facilitar significativamente el desarrollo.

1.5 Estructura de directorios para trabajar con este libro

Aunque acabamos de crear un proyecto utilizando PyCharm, la estructura de directorios que utilizaremos en este libro es independiente del IDE y puede recrearse manualmente en Linux, macOS o Windows utilizando cualquier editor o terminal.

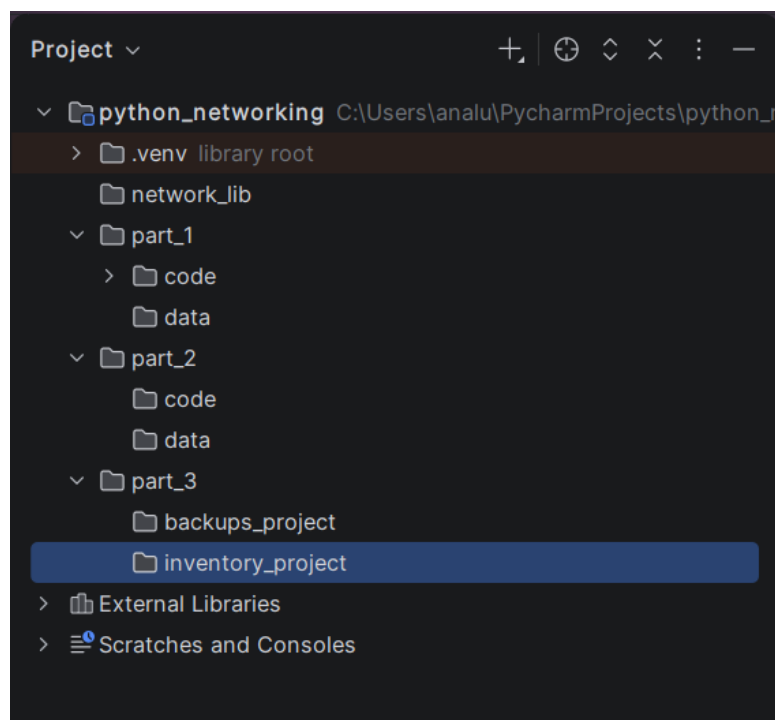
Si estás usando un IDE (recomendado), puedes crear directorios y archivos en la parte izquierda de la ventana, tal como acabamos de ver en la sección anterior.

Si estás usando la consola, tendrás que crear los directorios usando `mkdir` o su equivalente.

El directorio raíz es `python_networking`. Dentro de este directorio crea la siguiente estructura. Tienes que dar clic izquierdo y luego dar clic en **New -> Directory**:

```
python_networking/  
├── network_lib/  
├── part_1/  
│   ├── code/  
│   └── data/  
├── part_2/  
│   ├── code/  
│   └── data/  
└── part_3/  
    ├── backups_project/  
    └── inventory_project/
```

En PyCharm tendrías algo parecido a:



Esta estructura nos permitirá trabajar de forma ordenada al momento de crear todos nuestros scripts.

1.6 Tu primer programa en Python

Ahora que Python y tu entorno de desarrollo están listos, es momento de ejecutar tu primer programa. Crearemos un programa extremadamente simple para verificar rápidamente que todo funciona correctamente.

Aunque en teoría puedes escribir código en cualquier editor de texto, para este libro **recomendamos usar un IDE**, como PyCharm, ya que facilitan enormemente el proceso de escribir y ejecutar programas.

Para este primer ejemplo, utilizaremos la función `print()`, que permite mostrar la información en pantalla (salida estándar o `stdout`).

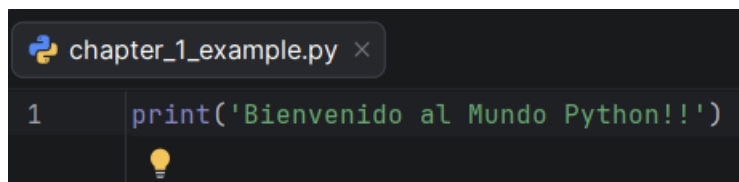
1.6.1 Opción recomendada: Usar un IDE

- Abre tu IDE.
- Haz clic en el archivo `python_networking/part_1/code/chapter_1_example.py`. En el editor que se abrirá en la parte derecha, escribe el siguiente código:

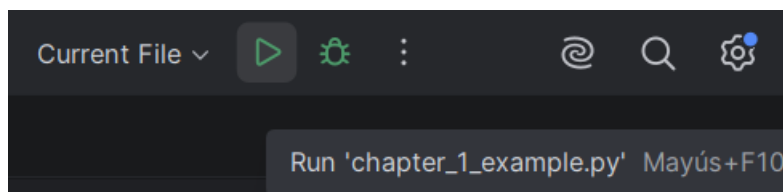
```
print('Bienvenido al Mundo Python!')
```

La función `print()` es una de las funciones más utilizadas en Python y permite mostrar información en pantalla. Más adelante aprenderemos a utilizarla con distintos tipos de datos y variables.

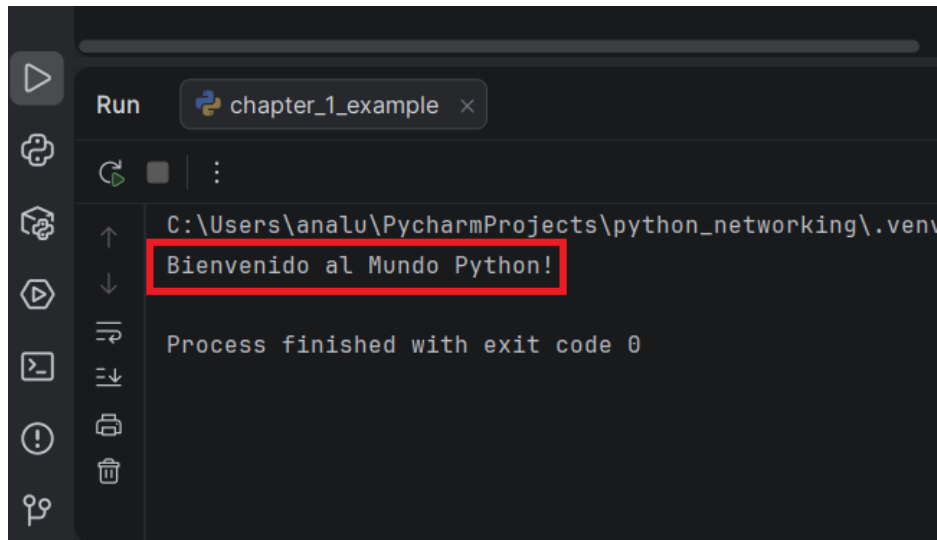
En PyCharm deberías poder ver algo similar a:



- No es necesario que guardes el archivo, pues el IDE guarda automáticamente tus cambios.
- Asegúrate de que en la barra de herramientas esté seleccionada la opción **Current File**, para asegurarte de ejecutar el archivo que estás editando. A continuación, ejecuta el script usando el botón  **Run** del IDE que se encuentra junto a **Current File**:



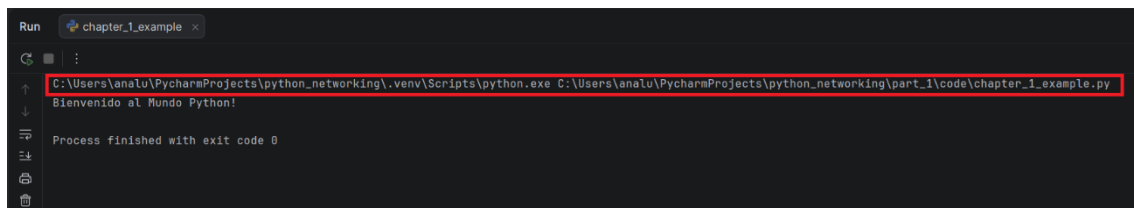
En la parte inferior del IDE, deberías ver el mensaje que se muestra en la consola:



⚠ **Nota importante:** El IDE también está ejecutando el programa tal cual vimos al inicio del capítulo, es decir:

```
python3 nombre_del_archivo.py
```

Pero lo ejecuta con las rutas completas hacia Python y hacia el archivo, tal como puedes ver en la consola:



La salida va a cambiar dependiendo de tu implementación, pero básicamente, debería ser algo similar a:

```
C:\Users\analuca\PycharmProjects\python_networking\.venv\Scripts\python.exe
C:\Users\analuca\PycharmProjects\python_networking\part_1\code\chapter_1_example.py
```

Observa que las rutas corresponden al proyecto y al entorno virtual creados por PyCharm.

1.6.2 Opción alternativa: Usar un editor básico

También es posible utilizar un editor básico, como **Notepad** (Windows), **TextEdit** (macOS) o cualquier editor de texto en Linux.

Los pasos son similares a los que ya ejecutamos:

- Abre el editor de texto escogido.
- Escribe la siguiente línea de código

```
print('Bienvenido al Mundo Python!')
```

- Guarda el archivo con el nombre `chapter_1_example.py` dentro de `python_networking/part_1/code`.
- Abre una terminal (cmd, PowerShell, Terminal).
- Navega hacia el directorio donde guardaste el archivo:

```
cd ruta/de/python_networking
```

- Ejecuta el programa

```
python3 chapter_1_example.py
```

1.6.3 Opción rápida: usar una herramienta online

Si quieres probar Python sin instalar nada, existen muchas páginas que permiten escribir y ejecutar código directamente desde el navegador. Funcionan bien para ejemplos simples y pruebas rápidas, aunque normalmente tienen limitaciones respecto a librerías externas o acceso al sistema operativo.

Busca en internet: ***Python online editor***.

Solo escribes tu código en la página y haces clic en **Run**.

```
print('Bienvenido al Mundo Python!')
```

Resultado:

```
Bienvenido al Mundo Python!
```

¡Felicidades! Has ejecutado tu primer programa en Python.

1.7 Módulos e `import`

Python incluye muchas funciones básicas listas para usar, como imprimir en pantalla, trabajar con datos o realizar operaciones matemáticas. Sin embargo, muchas herramientas avanzadas se encuentran dentro de **módulos**, que amplían las capacidades del lenguaje.

Un **módulo** es simplemente un archivo con extensión `.py` que contiene funciones y clases ya preparadas para ser utilizadas.

Python trae cientos de módulos preinstalados (la **librería estándar**), como:

- `math`
- `os`
- `datetime`
- `ipaddress`

Estos módulos permiten trabajar con números, archivos, fechas, direcciones IP y muchas otras tareas.

Para usar un módulo, primero debes importarlo utilizando la palabra clave `import`. La forma general es:

```
import módulo
```

Aquí:

- `import` permite importar el `módulo` para poder usar sus funciones y clases.
- `módulo` corresponde al nombre del módulo que vamos a importar.

Después de importar un módulo, normalmente accedemos a sus funciones utilizando la sintaxis:

```
módulo.función()
```

Por ejemplo, podemos importar `math` para usar la función `sqrt()`, la cual calcula la raíz cuadrada de un número:

```
import math

print(math.sqrt(25))
```

Resultado:

```
5.0
```

En este caso:

- `math` es el módulo importado.
- `sqrt()` es una función definida dentro del módulo `math`.

También puedes importar solo una parte específica de un módulo. La forma general es:

```
from módulo import elemento
```

Esto permite importar directamente una función o clase específica, sin necesidad de escribir el nombre completo del módulo cada vez.

Por ejemplo:

```
from datetime import datetime # Importa solo datetime
print(datetime.now())
```

Resultado:

```
2026-05-20 08:39:04.191691
```

También es posible importar un módulo utilizando un alias con la palabra clave **as**. Esto permite usar nombres más cortos o más cómodos dentro del código:

```
import math as m
print(m.sqrt(25))
```

Resultado:

```
5.0
```

En este caso:

- **m** es simplemente un alias del módulo **math**.
- Después del **import**, podemos utilizar **m** en lugar de escribir **math**.

Esto es muy común en Python, especialmente cuando:

- Los nombres de los módulos son largos.
- El módulo se utiliza frecuentemente dentro del programa.

En automatización de redes, el uso de módulos es fundamental. Por ejemplo, el módulo **ipaddress** facilita el manejo de redes y direcciones IP:

```
from ipaddress import ip_network
network = ip_network('10.0.0.0/24')
print(network.num_addresses)
```

Resultado:

256

A lo largo de este libro utilizaremos muchos módulos distintos.

Aprender a importar y utilizar módulos correctamente es una de las bases más importantes de la programación en Python.

1.8 Librerías externas: qué son y para qué sirven

Una de las características más poderosas de Python es su ecosistema de librerías. Las librerías son paquetes de código predefinido que puedes usar para realizar tareas específicas de manera más eficiente, sin tener que escribir todo desde cero. Esto es especialmente útil en la automatización de redes, ya que permite interactuar con dispositivos, enviar comandos y procesar datos de forma rápida y confiable.

Algunas de las librerías externas más importantes para la automatización de redes que exploraremos en este libro son:

- **requests**: Facilita la interacción con dispositivos web o APIs.
- **paramiko**: Es esencial para establecer conexiones SSH y ejecutar comandos en dispositivos remotos.

Estas herramientas son la base para llevar a cabo tareas más complejas en la gestión y automatización de redes.

A diferencia de módulos como **os**, **math** o **ipaddress**, estas librerías **NO** vienen incluidas con Python y deben instalarse con **pip**.

Más adelante aprenderás a instalarlas y utilizarlas dentro de proyectos reales.

 **Nota importante:** Python tiene miles de librerías adicionales para prácticamente cualquier área tecnológica. Aunque muchas de ellas están fuera del alcance de este libro, es útil conocer algunas de las más populares:

- **pandas**: Procesamiento y análisis de datos tabulares. Muy utilizada para trabajar con CSV, Excel y grandes volúmenes de información.
- **matplotlib**: Permite generar gráficos y visualizaciones de datos.
- **numpy**: Librería especializada en operaciones matemáticas y procesamiento numérico.
- **torch** y **tensorflow**: Utilizadas en inteligencia artificial y machine learning.
- **flask** y **fastapi**: Frameworks para desarrollar APIs y aplicaciones web.

El ecosistema de Python es enorme y continúa creciendo constantemente. Una de las mayores fortalezas del lenguaje es que existen herramientas listas para resolver prácticamente cualquier tipo de problema.

1.9 Recomendación: Crear archivos `test.py`

A lo largo de este libro encontrarás una gran cantidad de ejemplos pequeños y fragmentos de código.

Aunque muchos ejemplos muestran únicamente partes específicas del código para facilitar la explicación, es altamente recomendable que ejecutes y modifiques los ejemplos por tu cuenta mientras avanzas.

Una práctica muy útil es crear un archivo llamado `test.py` dentro de cada directorio `code` correspondiente al capítulo:

```
part_1/code/test.py  
part_2/code/test.py  
part_3/code/test.py
```

De esta manera puedes:

- Copiar ejemplos del libro.
- Modificar variables y valores.
- Experimentar con distintas entradas.
- Probar variantes del código.
- Entender mejor cómo funciona cada concepto.

Programar no se aprende únicamente leyendo código.

La mejor forma de aprender Python es ejecutando, probando y experimentando de forma constante.

1.10 Resumen

En este capítulo aprendiste:

- Qué es Python y por qué es una excelente opción para automatización de redes.
- Cómo instalar Python en Windows, macOS y Linux.
- Cómo configurar un entorno de desarrollo (IDE).
- Cómo ejecutar tu primer programa con diferentes métodos.
- Qué son los módulos y cómo funciona `import`.
- Qué son las librerías externas y para qué sirven.

Este capítulo estableció los fundamentos necesarios para comenzar a programar en Python: instalaste las herramientas esenciales, ejecutaste tu primer script y conociste los conceptos básicos que forman la base del desarrollo en este lenguaje. Con este punto de partida, ya estás listo para avanzar hacia temas más profundos y comenzar a aplicar Python en tareas reales de automatización de redes.

Capítulo 2 - Conceptos fundamentales de Python

A partir de este capítulo comenzaremos a trabajar directamente con la sintaxis y estructura del lenguaje.

Python se caracteriza por tener una sintaxis clara y relativamente simple, pero existen varias reglas y convenciones importantes que es necesario comprender desde el inicio para poder escribir código legible, consistente y fácil de mantener.

Muchos de los conceptos que veremos aquí aparecerán constantemente a lo largo del libro, ya que forman parte de la base sobre la que se construyen programas y automatizaciones más complejas.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Comprender los elementos fundamentales de la sintaxis de Python.
- Utilizar correctamente la sangría, espacios y saltos de línea.
- Escribir y documentar código usando comentarios.
- Trabajar con variables y distintos tipos de datos básicos.
- Comprender cómo Python ejecuta el código paso a paso.
- Utilizar funciones básicas como `print()` e `input()`.
- Reconocer estructuras de control como `if` y `for`.
- Interpretar errores y excepciones comunes en Python.
- Comprender qué son las clases y los objetos.
- Aplicar buenas prácticas y convenciones básicas de PEP 8.

2.1 Sintaxis básica

La sintaxis es el conjunto de reglas que indica cómo debe escribirse el código para que Python pueda interpretarlo correctamente. Cada lenguaje tiene sus propias normas, y en Python estas reglas son conocidas por ser simples, limpias y muy estrictas.

A diferencia de otros lenguajes, en Python, la sintaxis no solo organiza el código, sino que también define la estructura lógica del programa.

Por eso es tan importante respetarla: incluso pequeños detalles como los espacios, la sangría o el uso correcto de símbolos pueden cambiar completamente el significado del programa.

En Python, una buena sintaxis garantiza que el código sea:

- **Legible:** Fácil de entender para otros programadores (y para ti en el futuro).
- **Predecible:** Evita errores difíciles de detectar.
- **Ordenado:** Obliga a mantener un estilo limpio y consistente.

A lo largo de este capítulo verás los elementos más importantes de la sintaxis de Python: la sangría, los comentarios, las reglas para nombrar variables, el uso de espacios en blanco y otros conceptos básicos que forman la *gramática* del lenguaje.

2.1.1 Sangría

En Python, la sangría es esencial para definir bloques de código. A diferencia de otros lenguajes, no se utilizan llaves { }, sino espacios en blanco al inicio de cada línea. Esto hace que el código sea más limpio y fácil de leer, pero también implica que la sangría tiene un significado sintáctico. Un bloque de código corresponde a un conjunto de instrucciones que pertenecen a una misma estructura. La sangría indica exactamente dónde comienza y termina ese bloque.

La guía oficial de Python, **PEP 8**, recomienda usar **4 espacios por nivel de sangría**. Cuando utilizas un editor de código (como PyCharm), normalmente puedes presionar la tecla **Tab** para insertar automáticamente la sangría. La mayoría de los editores modernos convierten automáticamente la tabulación en 4 espacios, siguiendo esta recomendación.

Es importante mantener un formato consistente y no mezclar espacios y tabulaciones dentro del mismo bloque de código, ya que Python puede interpretarlos de forma distinta y generar errores de sangría.

Ejemplo de sangría correcta:

```
x = 1
if x > 0:
    print('El valor de x es positivo')
else:
    print('El valor de x es negativo o cero')
```

 **Nota importante:** En este ejemplo se utiliza una estructura `if/else`, que es un tipo de condicional. No es necesario entender todavía su funcionamiento; lo importante aquí es observar cómo las líneas con sangría pertenecen a un mismo bloque de código. Las estructuras condicionales se explicarán más adelante.

Ejemplo de sangría incorrecta:

```
x = 1
if x > 0:
print('El valor de x es positivo')
else:
print('El valor de x es negativo o cero')
```

Si no se sigue la sangría correcta, Python generará un error de tipo **IndentationError**.

2.1.2 Comentarios

Los comentarios sirven para hacer anotaciones sobre el código y no afectan su ejecución. En Python, existen dos tipos principales de comentarios: de una sola línea y multilínea.

Además, los comentarios forman parte de lo que se conoce como **documentación del código**. La documentación consiste en explicaciones escritas que describen qué hace el código, cómo funciona o por qué se tomó cierta decisión. Una buena documentación facilita la comprensión del programa, tanto para otras personas como para el propio desarrollador en el futuro.

Comentarios de una sola línea

Para un comentario de una sola línea, usa el símbolo **#**. Python ignorará el código a la derecha del símbolo:

```
# Este es un comentario
print('Bienvenido al Mundo Python!') # Esto imprime un mensaje
```

Comentarios multilínea

Para comentarios que ocupan varias líneas, usa el símbolo **#** al principio de cada línea:

```
# Este es un comentario que ocupa varias líneas.
# Puedes usar esta técnica para explicar en detalle
# lo que hace una sección en tu código.
```

También puedes encontrar cadenas de texto multilínea (usando comillas triples).

Técnicamente no son comentarios, sino **strings**. Cuando no se asignan a ninguna variable, Python no las utiliza durante la ejecución, por lo que a veces se usan con fines explicativos:

```
"""
Este es un comentario de varias líneas.
Puede utilizarse como documentación o como comentario extendido.
"""
```

2.1.3 Espacios

En Python, los espacios en blanco (espacios y tabulaciones) no solo se utilizan para la sangría, sino también para separar los operadores, variables y otros elementos dentro de una línea de código.

Ejemplo de buen uso de los espacios:

```
total_interfaces = 24 + 24 # Un espacio alrededor del operador
```

Ejemplo de mal uso de los espacios:

```
total_interfaces=24+24 # Sin espacios, es más difícil de leer
```

Usar espacios en blanco correctamente mejora la legibilidad del código y es una buena práctica. La mayoría de los IDEs advertirá automáticamente cuando los espacios o la sangría no sigan las convenciones recomendadas.

2.1.4 Saltos de línea y continuación implícita

En Python, los saltos de línea no siempre indican el final de una instrucción.

Python permite **dividir una misma expresión en varias líneas** para mejorar la legibilidad, siempre que la expresión esté dentro de paréntesis `()`, corchetes `[]` o llaves `{}`.

Continuación implícita con paréntesis, corchetes y llaves

Cuando una expresión está dentro de paréntesis `()`, corchetes `[]` o llaves `{}`, Python permite dividirla en varias líneas sin generar error.

Por ejemplo, podemos declarar una lista de hostnames en una sola línea:

```
devices = ['R1-GYE', 'SW1-GYE', 'R1-UIO']
```

Pero también podemos escribir la misma lista en varias líneas:

```
devices = [
    'R1-GYE',
    'SW1-GYE',
    'R1-UIO'
]
```

Ambas crean exactamente la misma lista.

La diferencia es únicamente **visual y de estilo**, no funcional.

Este comportamiento es muy útil cuando:

- La línea es larga.
- La estructura tiene muchos elementos.
- Quieres que el código sea más legible y mantenible.

Esto también aplica al contenido dentro de `()` y `{}`. Por ejemplo, estas dos funciones `print()` imprimen lo mismo en pantalla:

```
print('Los siguientes routers están alarmados: R1-AMB, R1-PVJ y R1-LOJ')
```

```
print('Los siguientes routers están alarmados: '
      'R1-AMB, '
      'R1-PVJ '
      'y R1-LOJ')
```

Resultado:

```
Los siguientes routers están alarmados: R1-AMB, R1-PVJ y R1-LOJ
Los siguientes routers están alarmados: R1-AMB, R1-PVJ y R1-LOJ
```

Python une automáticamente las cadenas de texto consecutivas dentro de los paréntesis, por lo que el resultado final es exactamente el mismo.

Uso de saltos de línea para mejorar la legibilidad

Además de dividir expresiones largas, en Python también es común utilizar saltos de línea simplemente para mejorar la legibilidad del código. Por ejemplo, es completamente válido escribir:

```
print('Iniciando proceso...')
print('Conectando a dispositivos...')
print('Proceso finalizado')
```

Y también es válido escribir:

```
print('Iniciando proceso...')
print('Conectando a dispositivos...')
print('Proceso finalizado')
```

Ambas formas son equivalentes y producen el mismo resultado:

```
Iniciando proceso...
Conectando a dispositivos...
Proceso finalizado
```

Estos espacios ayudan a:

- Separar visualmente bloques de código.
- Hacer el código más fácil de leer.
- Mejorar la organización lógica del programa.

En general, es recomendable utilizarlos cuando ayudan a entender mejor lo que hace el código.

Caso adicional: uso de la barra invertida (\) en cadenas de texto

Como acabamos de ver, podemos introducir saltos de línea dentro de un paréntesis, por lo que estas tres formas son equivalentes:

```
alarm_message = 'R1-GYE DOWN!'
alarm_message = ('R1-GYE DOWN!')
alarm_message = ('R1-GYE '
                 'DOWN!')
```

También es válido continuar una instrucción (que no está dentro de paréntesis) en la siguiente línea usando la barra invertida (**backslash**) \. Por ejemplo:

```
alarm_message = 'R1-GYE ' \
                'DOWN!'
```

Python concatena automáticamente las cadenas adyacentes, por lo que el resultado es el mismo:

```
print('R1-GYE DOWN!')
print(('R1-GYE DOWN!'))
print(('R1-GYE '
       'DOWN!'))
print('R1-GYE ' \
      'DOWN!')
```

En pantalla se muestra:

```
R1-GYE DOWN!
R1-GYE DOWN!
R1-GYE DOWN!
R1-GYE DOWN!
```

Este comportamiento es específico de literales de texto y no debe confundirse con el carácter \n, que representa un salto de línea dentro del contenido de la cadena.

Aunque funciona correctamente, en la práctica se suele preferir el uso de paréntesis para evitar errores y mejorar la legibilidad.

El carácter de salto de línea \n

El carácter especial \n no debe confundirse con escribir código en varias líneas.

\n representa un salto de línea dentro de una cadena de texto, no en la estructura del código. Es similar a presionar **Enter** dentro de un editor de texto.

Ejemplo:

```
message = 'Router R1-GYE\nEstado: UP\nCPU: 85%'
print(message)
```

Resultado:

```
Router R1-GYE
Estado: UP
CPU: 85%
```

En este caso, `\n` controla cómo se muestra el texto, no cómo se escribe el programa.

✦ Otros caracteres especiales comunes

Existen otros caracteres especiales similares a `\n` que también aparecen frecuentemente en programación:

- `\t` → Tabulación horizontal
Inserta un espacio amplio similar a presionar **TAB** en un editor de texto.
- `\r` → Retorno de carro (**carriage return**).
Mueve el cursor al inicio de la línea actual. Es común en protocolos antiguos, terminales y algunos formatos de texto.

Ejemplo:

```
print('Hostname:\tR1-GYE')
print('\r')
print('Estado:\t\tUP')
```

Resultado:

```
Hostname:  R1-GYE
Estado:      UP
```

⚠ **Nota importante:** En automatización de redes es común encontrar el carácter `\r` al procesar configuraciones, logs o salidas capturadas por SSH. Dependiendo del sistema o dispositivo, puede aparecer junto con `\n` como parte de los saltos de línea. En muchos casos, estos caracteres se eliminan o se reemplazan para normalizar el texto antes de procesarlo.

2.1.5 Uso de comillas

En Python, las comillas simples (`'`) o dobles (`"`) se utilizan para definir cadenas de texto. Estos dos tipos de comillas son equivalentes, pero se recomienda ser consistente en su uso, es decir, mantener el mismo estilo a lo largo del código para que sea más claro y fácil de leer.

```
router = 'R1-GYE'
switch = "SW1-UIO"
```

Para incluir comillas dentro de una cadena, utiliza el tipo opuesto o el carácter de escape **backslash** (`\`).

A continuación, vamos a ver algunos ejemplos de cómo utilizar correctamente las comillas:

▶ String con comillas dobles externas y comillas simples internas

Cuando las comillas internas son diferentes a las externas, no es necesario utilizar caracteres de escape:

```
message = "Alarmas: 'R1-GYE con temperatura elevada'"
print(message)
```

Resultado:

```
Alarmas: 'R1-GYE con temperatura elevada'
```

▶ String con comillas simples externas y escape de comillas simples

También es posible utilizar el carácter de escape backslash (\) para incluir comillas dentro del texto:

```
message = 'Alarmas: \'R1-GYE con temperatura elevada\''
print(message)
```

Resultado:

```
Alarmas: 'R1-GYE con temperatura elevada'
```

▶ String con comillas simples externas y comillas dobles internas

Al igual que con las comillas simples, puedes usar comillas dobles dentro del texto si las externas son diferentes:

```
message = 'Alarmas: "R1-GYE con temperatura elevada"'
print(message)
```

Resultado:

```
Alarmas: "R1-GYE con temperatura elevada"
```

▶ String con comillas dobles externas y escape de comillas dobles

Las comillas dobles también pueden escaparse utilizando el carácter backslash (\):

```
message = "Alarmas: \"R1-GYE con temperatura elevada\""
print(message)
```

Resultado:

```
Alarmas: "R1-GYE con temperatura elevada"
```

⚠ **Nota importante:** Si intentas ejecutar código sin escapar correctamente las comillas, vas a generar un error de tipo **SyntaxError** y el programa no se ejecutará exitosamente:

```
message = "Alarmas: "R1-GYE con temperatura elevada""  
print(message)
```

Resultado:

```
File "test.py", line 2  
    message = "Alarmas: "R1-GYE con temperatura elevada""  
                    ^^  
SyntaxError: invalid syntax
```

Más adelante, veremos que las cadenas también pueden formarse usando **f-strings**, una técnica para insertar variables dentro de un texto.

2.2 Introducción a variables

Más adelante veremos en mayor profundidad el uso de las variables. Sin embargo, en esta sección veremos lo mínimo necesario para poder comprender otros conceptos fundamentales.

Aunque todavía no las hemos explicado formalmente, ya hemos utilizado variables en ejemplos anteriores. Por ejemplo, cuando vimos el uso de comillas, almacenamos un texto dentro de una variable:

```
alarm_message = 'R1-GYE DOWN!'
```

Las variables permiten almacenar y trabajar con valores. En Python deben seguir ciertas reglas de nomenclatura. A continuación, se muestran las principales:

- Para asignar el nombre a una variable, se usa el operador `=`. El nombre de la variable es el de la izquierda y el valor asignado está a la derecha del `=`:

```
nombre_variable = 'valor de la variable'
```

- Los nombres de variables comienzan con una letra o guion bajo (`_`). No pueden comenzar con un número:

```
variable = 5      # Correcto (empieza con una letra)
_variable = 5     # Correcto (empieza con un guion bajo)
1variable = 10    # Incorrecto (no puede empezar con un número)
```

- Los nombres de variables pueden contener letras, números y guiones bajos:

```
variable_1 = 7      # Correcto
my_variable_1 = 10  # Correcto
```

- No pueden ser palabras reservadas. Algunas palabras están reservadas por Python, y no puedes usarlas como nombres de variables. Estas palabras son clave para el lenguaje y tienen un significado especial (más detalles en la siguiente sección).
- Por convención, los nombres de variables se escriben en minúsculas, y si se usan varias palabras, se separan con guiones bajos. Esta convención de nomenclatura se denomina **snake_case**.

```
router_hostname = 'R1-GYE' # Correcto
RouterHostname = 'R1-GYE'  # Se recomienda minúsculas en variables
```

- Evita usar nombres demasiado cortos. Usa nombres descriptivos para que tu código sea fácil de leer y entender.

```
response_time = 200 # Correcto
rt = 200           # Menos claro, pero válido
```

Sensibilidad a mayúsculas (case-sensitive)

Python distingue mayúsculas de minúsculas.

Estas dos variables son diferentes:

```
hostname = 'R1-GYE'  
Hostname = 'R2-UIO'
```

Aunque los nombres se parecen visualmente, Python los interpreta como variables completamente distintas. Por esta razón, es importante mantener un estilo consistente al nombrar variables para evitar errores difíciles de detectar.

Tipos de datos

Los valores que almacenamos en variables tienen un tipo de dato. El tipo de dato define qué clase de información contiene la variable y cómo Python puede trabajar con ella.

Python maneja muchos tipos de datos. Algunos de los más comunes son:

- **Tipo entero (int)**
Almacena valores enteros, como 1, 2 o 10.
- **Tipo texto (str)**
Almacena texto, por ejemplo: 'Gi0/1' o 'Equipo alarmado'.
- **Tipo lista (list)**
Almacena múltiples valores en una misma variable, por ejemplo: [1, 2, 10]

2.3 Palabras reservadas

En Python, algunas palabras tienen un significado especial y forman parte de la sintaxis oficial del lenguaje. Estas palabras se conocen como palabras reservadas (*keywords*), y no pueden utilizarse como nombres de variables, funciones o clases.

Esto ocurre porque Python ya utiliza estas palabras internamente como instrucciones dentro del código. Si estuviera permitido reutilizarlas como nombres de variables, habría confusión entre las instrucciones de Python y los nombres creados por el programador.

Python utiliza estas palabras para construir diferentes partes del código, por ejemplo:

- Condicionales (**if**, **else**)
- Bucles (**for**, **while**)
- Funciones (**def**, **return**)
- Manejo de errores (**try**, **except**)
- Operadores lógicos (**and**, **or**, **not**)

Algunas de las palabras reservadas más comunes son:

```
False await else import pass None break except in raise
True class finally is return and continue for lambda try
as def from nonlocal while assert del global not with
async elif if or yield
```

Si intentas utilizar una palabra reservada como nombre de variable, Python generará un error de tipo **SyntaxError** cuando ejecutes el código.

Por ejemplo:

```
class = 'Router'
```

Resultado:

```
File "test.py", line 1
  class = 'Router'
    ^
SyntaxError: invalid syntax
```

 **Nota importante:** La lista exacta de palabras reservadas puede variar ligeramente entre versiones de Python, aunque la mayoría se mantiene estable entre versiones modernas.

A medida que avances en el libro, iremos utilizando muchas de estas palabras reservadas de manera natural dentro del código.

2.4 Introducción a funciones

Las funciones en Python nos permiten ejecutar tareas específicas dentro de un programa.

[En un capítulo posterior](#) las estudiaremos en detalle, pero por ahora es suficiente comprender su uso básico.

En programación, utilizar una función se conoce como **llamar la función**. También es común utilizar términos como **invocarla** o **ejecutarla**.

Para utilizar una función en Python, simplemente **la llamamos** y le pasamos los **argumentos** que necesita.

Los argumentos son valores o información que la función utiliza para realizar su tarea.

La forma general de llamar una función es la siguiente:

```
nombre_funcion(argumento1, argumento2, ...)
```

Python incluye muchas funciones listas para usar. Algunas de las más fundamentales son:

- `print()` : Te permite imprimir mensajes en pantalla.
- `input()` : Te permite ingresar información por teclado para que tu script pueda utilizarla posteriormente.

2.4.1 La función `print()`

La función `print()` es una de las herramientas más utilizadas al comenzar con Python. Su uso básico es imprimir contenido en pantalla, pero también tiene varias opciones que vale la pena conocer.

El uso básico de `print()` es el siguiente:

```
print('El router R1-GYE presenta alto consumo de CPU!')
```

Sin embargo, también tenemos argumentos que nos permiten cambiar lo que se presenta en pantalla:

🔴 El argumento `sep`

`sep` es el carácter separador. Define qué se coloca entre los elementos impresos. Si no lo especificamos, se utiliza un espacio en blanco por defecto:

```
print('R1', 'UP', '10.0.0.1')
```

Resultado:

```
R1 UP 10.0.0.1
```

Observa que Python agregó automáticamente un espacio entre cada elemento.

Si especificamos el separador utilizando **sep**, el espacio por defecto es reemplazado por el valor indicado:

```
print('R1', 'UP', '10.0.0.1', sep=',')
```

Resultado:

```
R1,UP,10.0.0.1
```

En este caso, las comas reemplazan los espacios entre los elementos impresos.

📌 El argumento **end**

end define cómo termina lo que se imprime en pantalla. Por defecto, **print()** finaliza con un salto de línea (**\n**), por lo que el siguiente **print()** normalmente aparecería en una nueva línea:

```
print('R1-GYE')
print('R1-UIO')
```

Resultado:

```
R1-GYE
R1-UIO
```

Utilizando el argumento **end**:

```
print('R1-GYE', end=' --> ')
print('R2-UIO')
```

Resultado:

```
R1-GYE --> R2-UIO
```

En este caso, el salto de línea por defecto fue reemplazado por el texto **'--> '**.

2.4.2 La función **input()**

En capítulos posteriores trabajaremos principalmente con archivos, APIs y dispositivos reales; es decir, tu código obtendrá datos automáticamente.

Pero para practicar conceptos básicos, es útil poder pedir datos al usuario, y para eso existe la función **input()**.

input() siempre devuelve una cadena de caracteres, incluso si el usuario escribe un número:

```
name = input('Ingresa tu nombre: ')
print('Hola, ' + name)
```

Resultado:

```
Ingresar tu nombre: AniLuCa  
Hola, AniLuCa
```

Más adelante verás que `input()` no suele usarse en automatización real (los scripts no deberían depender de interacción humana), pero es una herramienta excelente para practicar lógica y estructura de programas.

2.5 Flujo básico de ejecución en Python

Uno de los conceptos más importantes (y a la vez más fáciles de pasar por alto) es entender **cómo se ejecuta un programa en Python**:

- Python ejecuta el código de forma **secuencial, línea por línea, de arriba hacia abajo**.
- No analiza todo el archivo primero ni adivina lo que vendrá después.
- Simplemente ejecuta cada línea en el orden en que aparece.

🔴 Ejecución secuencial: de arriba hacia abajo

Observa el siguiente ejemplo:

```
print('Inicio del programa')

x = 5
print(x)

x = x + 1
print(x)

print('Fin del programa')
```

La ejecución ocurre exactamente en este orden:

- Se imprime **Inicio del programa**.
- Se asigna el valor 5 a la variable **x**.
- Se imprime 5.
- Se actualiza el valor de **x** a 6.
- Se imprime 6.
- Se imprime **Fin del programa**.

Python **no salta líneas** ni ejecuta nada fuera de este orden.

🔴 Python *no adivina el futuro*

Este ejemplo suele causar confusión al inicio:

```
print(y)
y = 10
```

Este código produce un error, porque cuando Python llega a la primera línea `print(y)` la variable **y** todavía no existe.

Python no sabe que más abajo se le asignará un valor.

Solo conoce lo que ya se ejecutó.

📌 Las variables cambian con el flujo del programa

Una variable no conserva versiones anteriores. Solo tiene un valor en cada momento del flujo:

```
status = 'UP'
print(status)

status = 'DOWN'
print(status)
```

Resultado:

```
UP
DOWN
```

El valor anterior se reemplaza completamente.

📌 El flujo solo cambia con estructuras de control

El orden de arriba hacia abajo solo se modifica cuando usamos estructuras especiales (entraremos en detalle en futuros capítulos), como:

- Condicionales `if/else`
- Lazos `for/while`
- Funciones

Ejemplo con `if`:

```
x = 3
if x > 5:
    print('Mayor que 5')
print('Esto siempre se ejecuta')
```

Si la condición no se cumple, Python **simplemente salta ese bloque** y continúa con la siguiente línea válida.

En resumen:

- Python ejecuta el código en orden
- Cada línea se ejecuta una sola vez
- Las variables cambian conforme avanza el programa
- Python no anticipa líneas futuras
- El flujo solo cambia con estructuras de control

2.6 Introducción a estructuras de control

Cuando estamos desarrollando nuestros scripts, en muchos casos necesitamos que el programa pueda:

- Tomar decisiones.
- Repetir acciones varias veces.
- Ejecutar código solo cuando se cumpla una condición.

Para lograr esto, los lenguajes de programación utilizan **estructuras de control**.

Las estructuras de control permiten modificar el flujo normal de ejecución del programa, que normalmente ocurre **de arriba hacia abajo**.

En Python, dos de las estructuras de control más comunes son:

- `if`, que permite tomar decisiones.
- `for`, que permite repetir una acción sobre múltiples elementos.

En este punto no estudiaremos estas estructuras en detalle, pero veremos un ejemplo sencillo de cada una para que puedas reconocerlas cuando aparezcan en los próximos capítulos.

📌 Uso básico de `if`

La estructura `if` permite ejecutar código solo cuando se cumple una condición.

```
cpu_usage = 85
if cpu_usage > 80:
    print('Alto uso de CPU!')
```

En este ejemplo, el mensaje se imprime en pantalla únicamente si la condición `cpu_usage > 80` es verdadera.

📌 Uso básico de `for`

La estructura `for` permite recorrer una colección de elementos y ejecutar una acción para cada uno.

```
routers = ['R1-GYE', 'R1-UIO', 'R2-GYE']
for router in routers:
    print(router)
```

Resultado:

```
R1-GYE
R1-UIO
R2-GYE
```

Aquí Python toma cada elemento de la lista `routers` y lo asigna temporalmente a la variable `router`, ejecutando el bloque de código para cada uno.

En un [capítulo posterior](#) estudiaremos estas estructuras con mayor detalle y veremos cómo utilizarlas para construir scripts más completos.

Por ahora, lo importante es reconocer que las estructuras de control permiten que nuestros programas tomen decisiones y trabajen con múltiples datos.

2.7 Introducción a errores y excepciones

Al comenzar a programar, es completamente normal que el código produzca errores. Un error no significa que el programa esté **mal diseñado**. Significa que Python detectó algo que no puede ejecutar.

Cuando Python encuentra un problema durante la ejecución del programa, genera una excepción y muestra información detallada del error en pantalla. A esta salida se le conoce como **traceback**.

El traceback muestra:

- Dónde ocurrió el error.
- Qué línea produjo el problema.
- Qué tipo de excepción fue generada.

Por ejemplo:

```
print(variable_no_definida)
```

Resultado:

```
Traceback (most recent call last):
  File "test.py", line 1, in <module>
    print(variable_no_definida)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^
NameError: name 'variable_no_definida' is not defined
```

Se generó una excepción porque tratamos de imprimir una variable que no ha sido definida anteriormente.

O este otro caso:

```
print(5/0)
```

Resultado:

```
Traceback (most recent call last):
  File "test.py", line 1, in <module>
    print(5/0)
    ~^~
ZeroDivisionError: division by zero
```

La división por 0 no está definida en Matemáticas, por lo que también se genera un error.

Estos mensajes pueden parecer intimidantes al inicio, pero en realidad son información valiosa. Le indican al programador:

- Qué tipo de error ocurrió.
- En qué línea ocurrió.
- Qué parte del código lo causó.

En los próximos capítulos seguiremos viendo errores de forma natural.

[Más adelante](#) aprenderás cómo capturarlos y manejarlos correctamente usando estructuras como `try` y `except`.

Por ahora, lo importante es entender que:

- Los errores son parte normal del proceso.
- Leer el mensaje completo es fundamental.
- Cada excepción tiene un nombre y un significado específico.
- Cuando Python detecta un error, **el programa deja de ejecutarse en ese momento.**

Aprender a interpretar errores es una de las habilidades más importantes en programación.

2.8 Introducción a clases y objetos

En Python, casi todo lo que manipulamos es un **objeto**. Por ejemplo, al asignar un número a una variable:

```
total_interfaces = 24
```

El valor **24** es un **objeto** de tipo entero (`int`).

De la misma forma, cuando creamos una cadena de texto:

```
hostname = 'R1-GYE'
```

'R1-GYE' es un **objeto** de tipo string (`str`).

🔴 ¿Qué es una clase?

Una **clase** es un **molde** o **plantilla**. Los objetos se crean a partir de ese molde.

Por ejemplo:

Clase	Descripción	Objetos
<code>int</code>	Permite almacenar números enteros.	10, 20, 864, etc.
<code>str</code>	Permite almacenar cadenas de texto.	'R1-GYE', 'R2-GYE', 'R1-UIO', etc.
<code>list</code>	Permite almacenar múltiples elementos en una lista.	['R1-GYE', 'R2-GYE'], [10, 20, 864], etc.

Por ahora, lo importante es entender que los distintos tipos de datos en Python son objetos creados a partir de una clase. Más adelante aprenderemos cómo crear nuestras propias clases y utilizar objetos en programas más complejos.

2.9 Buenas prácticas

Además de seguir las reglas de sintaxis, es importante adoptar ciertos hábitos que ayuden a mantener el código claro, ordenado y fácil de entender. Estas prácticas mejoran la legibilidad y facilitan el mantenimiento de los programas a medida que crecen en complejidad:

- **Usar nombres descriptivos:**
Nombres como `packet_loss` o `cpu_usage` son más claros que `x` o `y`.
- **Consistencia en la sangría:**
Usa 4 espacios para cada nivel de sangría.
- **Mantener una estructura consistente en el código:**
Organiza el código de forma ordenada y mantén un estilo similar entre diferentes scripts y funciones.
- **Añadir comentarios:**
Usa comentarios cuando el propósito de una sección de código no sea claro de inmediato.
- **Mantener las líneas de código dentro de un límite razonable:**
Una línea de código larga puede ser difícil de leer.
- **Evitar duplicar código innecesariamente:**
Si una misma lógica se repite muchas veces, normalmente es mejor convertirla en una función.
- **Probar el código en partes pequeñas:**
Ejecutar y validar pequeñas secciones de código facilita detectar errores más rápidamente.

Seguir estas buenas prácticas desde el inicio facilita la lectura, el mantenimiento y la depuración del código. En proyectos reales, un código claro y consistente es tan importante como que el programa funcione correctamente.

2.9.1 PEP 8: Guía oficial de estilo en Python

Python tiene una guía de estilo oficial llamada **PEP 8**, que define cómo debe escribirse el código para que sea claro, uniforme y fácil de mantener.

No es obligatorio seguirla al 100%, pero sí es altamente recomendable, especialmente cuando varios desarrolladores trabajan en el mismo proyecto.

Aquí resumimos las reglas más importantes que usarás desde este libro en adelante:

- Usa 4 espacios por nivel de sangría. Python no utiliza llaves como otros lenguajes; la sangría define los bloques de código.

```
if interface_status == 'up':
    print('Interfaz operativa')
```

- Líneas de máximo (aproximado) 79 caracteres. Esto facilita la lectura en pantallas pequeñas y herramientas como `diffs` o revisores de código.
- Deja dos líneas en blanco entre funciones y clases a nivel superior. Esto ayuda a separar visualmente los bloques principales del archivo:


```
def function_1():
    pass

def function_2():
    pass
```

- Deja una línea en blanco entre métodos dentro de una clase. Esto diferencia claramente cada comportamiento del objeto.
- Usa nombres en **snake_case** para variables y funciones. El estándar **snake_case** especifica el uso de minúsculas, separados por guiones bajos, por ejemplo:

```
total_traffic, load_interfaces(), validate_vlan()
```

- Usa nombres en **PascalCase** para clases. El estándar **PascalCase** especifica el uso de mayúsculas, sin separadores, por ejemplo:

```
CiscoRouter, InventoryParser, TrafficReport
```

- Las importaciones deben estar siempre en la parte superior del archivo:

```
import time
import requests

# Resto del código
```

- Usa mayúsculas para valores o constantes que no deban cambiar:

```
DEFAULT_TIMEOUT = 5
MAX_HOPS = 30
IPV4_RE = re.compile(r'\b\d+\.\d+\.\d+\.\d+\b')
```

Python no impone constantes reales. Usar mayúsculas es una convención para comunicar intención.

Seguir estas reglas te permitirá escribir código más limpio, profesional y compatible con herramientas como **PyCharm**.

Conforme avances en tu camino de aprendizaje con Python, verás cómo estas buenas prácticas ayudan a mantener tus scripts organizados incluso cuando crezcan en complejidad.

2.10 Resumen

En este capítulo aprendiste los conceptos fundamentales necesarios para escribir código claro y correcto en Python. Viste la importancia de la sangría como parte esencial de la estructura del lenguaje, cómo usar comentarios para documentar tu código, cómo manejar espacios y saltos de línea, y cómo trabajar con comillas y caracteres especiales dentro de cadenas de texto.

También revisaste las reglas básicas para nombrar variables, la sensibilidad de Python a mayúsculas y minúsculas, las palabras reservadas del lenguaje y el uso inicial de funciones como `print()` e `input()`. Además, comprendiste cómo Python ejecuta el código línea por línea, cómo cambian las variables durante el flujo del programa y por qué los errores forman parte normal del proceso de aprendizaje.

Finalmente, conociste buenas prácticas basadas en **PEP 8** y una primera introducción a clases y objetos, una idea clave para entender cómo Python representa valores como números, textos y listas.

Con estos conocimientos, ya tienes una base sólida para continuar hacia temas más avanzados, como tipos de datos, estructuras de control y la creación de tus propias funciones.

Capítulo 3 - Variables y tipos de datos

En los capítulos anteriores aprendiste la sintaxis básica de Python y cómo estructurar un programa correctamente.

Ahora es momento de entender qué es lo que realmente manipulamos cuando programamos: los datos.

En este capítulo exploraremos cómo Python representa la información a través de variables y distintos tipos de datos. Verás cómo almacenar valores, cómo se comportan internamente y qué implicaciones tiene su uso en la práctica.

Más allá de memorizar tipos o definiciones, el objetivo es que desarrolles un modelo mental claro sobre cómo funcionan los datos en Python. Este entendimiento será clave para todo lo que viene después, especialmente cuando trabajemos con automatización real, donde los datos no siempre son simples ni están bien estructurados.

Este capítulo es conceptual. No se trata todavía de hacer cosas complejas, sino de comprender bien la base sobre la cual construiremos todo lo demás.



¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Comprender cómo Python representa y almacena los datos.
- Trabajar con variables y distintos tipos de datos básicos.
- Diferenciar entre listas, tuplas, sets y diccionarios.
- Acceder y manipular elementos mediante índices y slicing.
- Utilizar estructuras anidadas con listas y diccionarios complejos.
- Aplicar conversiones de tipos y operaciones matemáticas básicas.
- Construir cadenas dinámicas utilizando f-strings.
- Comprender conceptos como iterables, mutabilidad e inmutabilidad.
- Utilizar operadores como in, == e is correctamente.
- Entender qué son los objetos hashables y por qué son importantes.

3.1 Variables

Una variable en programación es un espacio de almacenamiento donde se puede guardar un valor o dato.

Las variables permiten trabajar con datos de forma flexible, evitando repetir valores directamente en el código.

En Python, puedes declarar una variable simplemente asignándole un valor, usando el operador igual (=). La forma general es:

```
nombre_variable = valor
```

Por ejemplo:

```
cpu_usage = 15.1
```

En este código, la variable `cpu_usage` almacena el valor **15.1**.

A partir de este momento, puedes usar esa variable en el código en lugar de escribir **15.1** cada vez que lo necesites.

Las variables en Python no requieren una declaración explícita del tipo de dato. Python es un lenguaje dinámicamente tipado, lo que significa que el tipo de dato se determina automáticamente según el valor asignado.

Sin variables, los programas serían extremadamente limitados y difíciles de mantener. Imagina un script donde una dirección IP, un hostname o un valor de temperatura tuviera que escribirse manualmente cada vez que se necesita.

Si el valor cambia, habría que modificarlo en múltiples partes del código.

En automatización de redes, las variables se utilizan constantemente para almacenar:

- Hostnames.
- Direcciones IP.
- Estados de interfaces.
- Métricas de CPU y memoria.
- VLANs.
- Resultados de comandos.
- Datos obtenidos desde APIs o SNMP.

Prácticamente todo lo que procesa un programa termina almacenándose temporalmente en variables.

3.2 Tipos de datos básicos

En Python, existen varios tipos de datos que puedes utilizar para almacenar información. A continuación, veremos algunos de los más comunes.

3.2.1 Enteros (`int`)

Un entero o `int` es un número sin decimales. En Python, los números enteros pueden ser positivos, negativos o cero.

```
interfaces_up = 24
```

Puedes realizar operaciones matemáticas básicas con enteros, como sumas, restas, multiplicaciones y divisiones.

```
interfaces_giga = 6
interfaces_tengiga = 7
print(interfaces_giga + interfaces_tengiga)
```

Resultado:

```
13
```

En automatización de redes, los enteros se utilizan constantemente para representar cantidades, VLANs, puertos, métricas, IDs o contadores obtenidos desde dispositivos y sistemas de monitoreo.

3.2.2 Flotantes (`float`)

Un `float` es un número con decimales. A diferencia de los enteros, los flotantes pueden tener una parte decimal, lo que los hace útiles para representar valores como precios, temperaturas o métricas de rendimiento.

```
temperature = 23.5
```

Las operaciones matemáticas con flotantes son similares a las que se realizan con enteros, pero pueden incluir decimales.

```
temperature_sensor1 = 26.7
temperature_sensor2 = 37.6
temperature_mean = (temperature_sensor1 + temperature_sensor2) / 2
print(temperature_mean)
```

Resultado:

```
32.15
```

Los valores flotantes son comunes en métricas de rendimiento, temperatura, utilización de CPU, latencia, throughput y otras mediciones obtenidas desde equipos de red.

3.2.3 Cadenas de texto (strings)

Una cadena de texto (**string**) es una secuencia de caracteres. En Python, los **strings** se pueden definir utilizando comillas simples o dobles.

```
# Usando comillas simples:  
alarms_message = 'No hay equipos alarmados!'  
print(alarms_message)
```

Resultado:

```
No hay equipos alarmados!
```

Obtenemos el mismo resultado si usamos comillas dobles en lugar de simples:

```
# Usando comillas dobles:  
alarms_message = "No hay equipos alarmados!"  
print(alarms_message)
```

Resultado:

```
No hay equipos alarmados!
```

Las cadenas de texto son útiles cuando necesitas trabajar con texto, como letras, números, palabras o frases.

Para concatenar cadenas, se utiliza el operador +:

```
hostname = 'R1-GYE'  
  
# Para concatenar strings usamos +:  
message = 'El router ' + hostname + ' está operativo!!'  
print(message)
```

En este ejemplo estamos concatenando tres strings:

- 'El router '
- 'R1-GYE' (valor almacenado en la variable `hostname`)
- ' está operativo!!'

El resultado en pantalla es:

```
El router R1-GYE está operativo!!
```

En redes, las cadenas de texto se utilizan constantemente para representar hostnames, direcciones IP, estados de interfaces, comandos CLI, logs y mensajes de monitoreo.

📌 Índices en un string

Puedes acceder a los elementos de un string utilizando un índice.

Un índice es la posición de un elemento dentro de una secuencia. En Python, los índices empiezan desde 0. Por ejemplo, en el siguiente string:

```
hostname = 'R1-GYE'
```

Las posiciones son:

R	1	-	G	Y	E
0	1	2	3	4	5

Para acceder a un elemento, se utilizan corchetes []:

```
elemento = cadena_de_texto[indice]
```

Por ejemplo:

```
print(hostname[0]) # Resultado: R
print(hostname[1]) # Resultado: 1
print(hostname[2]) # Resultado: -
```

Es decir:

- `hostname[0]` devuelve R
- `hostname[1]` devuelve 1
- `hostname[2]` devuelve -

También es posible utilizar índices negativos. En este caso, Python empieza a contar desde el final del string:

R	1	-	G	Y	E
-6	-5	-4	-3	-2	-1

Por ejemplo:

```
print(hostname[-1]) # Resultado: E
print(hostname[-2]) # Resultado: Y
print(hostname[-3]) # Resultado: G
```

⚠️ Nota importante: Si intentas acceder a un índice que no existe (ya sea positivo o negativo), Python generará un error de tipo `IndexError`.

```
hostname = 'R1-GYE'
print(hostname[100])
```

Resultado:

```
Traceback (most recent call last):
  File "test.py", line 2, in <module>
    print(hostname[100])
    ~~~~~^~~~~~
IndexError: string index out of range
```

⚠ **Nota importante:** Los strings no soportan modificación de sus elementos. Si intentas modificar uno, utilizando índices, vas a generar un error de tipo **TypeError**:

```
hostname = 'R1-GYE'
hostname[1] = '2'
```

Resultado:

```
Traceback (most recent call last):
  File "test.py", line 2, in <module>
    hostname[1] = '2'
    ~~~~~^~~~~~
TypeError: 'str' object does not support item assignment
```

3.2.4 Booleanos (bool)

Un valor booleano representa únicamente dos posibles estados:

- Verdadero (**True**)
- Falso (**False**)

En programación, estos valores se utilizan para representar condiciones lógicas, como:

- Sí / No
- Encendido / Apagado
- Correcto / Incorrecto
- Disponible / No disponible

En Python, los valores booleanos se representan utilizando **True** y **False**:

```
router_down = True
router_up = False
```

Se usan principalmente en:

- Condicionales (**if/else**)
- Resultados de comparaciones
- Funciones que indican éxito o fallo

Muchas operaciones en Python no producen números ni texto, sino valores booleanos. Esto ocurre, por ejemplo, cuando comparamos dos valores:


```
x = 0
if x == 0: # Esta comparación produce True
    print('x tiene un valor igual a 0')
```

Los valores booleanos son fundamentales en automatización, ya que permiten validar condiciones y controlar el flujo de ejecución de los programas.

3.2.5 Ausencia de valor (None)

None es un tipo especial que se utiliza para representar la ausencia de un valor. No significa un texto vacío ni el número cero. Significa literalmente: no hay valor:

```
value = None
```

En automatización de redes, **None** aparece con frecuencia cuando:

- Una función no pudo obtener información.
- Una búsqueda no encontró resultados.
- Un dato aún no ha sido definido.

 **Nota importante:** **None** no es lo mismo que vacío. Esta distinción es muy importante:

```
text = ''
number = 0
value = None
```

Aunque estos valores son diferentes entre sí, muchas veces se utilizan para representar estados *vacíos* o ausencia de información:

Valor de variable	Explicación
' '	Existe texto, pero está vacío.
0	Existe número. Vale cero.
[]	Existe lista. No tiene elementos.
None	No hay valor.

None representa la ausencia real de un valor. No significa texto vacío, lista vacía ni el número cero.

Para verificar si una variable es **None**, se usa el operador **is**:

```
if value is None:
    print('No hay valor disponible')
```

 Nota sobre el operador `is` y el operador `==`

En Python existen dos formas comunes de comparar valores:

- `==` compara si dos valores son iguales.
- `is` compara si dos variables apuntan al mismo objeto en memoria.

```
a = [1, 2]
b = [1, 2]
print(a == b)  # True
print(a is b)  # False
```

Las listas tienen el mismo contenido, pero son objetos diferentes en memoria.

Por esta razón, cuando se trabaja con **None**, la recomendación en Python es usar `is`:

```
if value is None:
    print('No hay valor disponible')
```

Esto verifica si la variable realmente hace referencia al objeto especial **None**.

3.3 Colecciones

Python incluye estructuras que permiten agrupar múltiples valores dentro de una sola variable. Estos tipos de datos se denominan colecciones.

A diferencia de los tipos simples (como `int`, `float` o `str`), las colecciones permiten agrupar información relacionada y trabajar con ella de forma estructurada.

Python incluye varias colecciones integradas, cada una con características distintas:

- **Listas:** colecciones ordenadas que pueden cambiar su contenido (mutables).
- **Tuplas:** colecciones ordenadas cuyo contenido no puede cambiar (inmutables).
- **Sets:** colecciones desordenadas sin elementos duplicados.
- **Diccionarios:** colecciones de pares clave/valor.

Elegir la colección adecuada es una de las decisiones más importantes al programar, especialmente en automatización, donde trabajamos constantemente con listas de equipos, interfaces, VLANs o direcciones IP.

3.3.1 Listas (`lists`)

Una lista es una colección ordenada de elementos. Estos pueden ser de distintos tipos, aunque en contextos de automatización se recomienda que todos representen el mismo tipo de información para mantener consistencia y facilitar su procesamiento.

Las listas en Python se definen utilizando corchetes `[]`, y sus elementos están separados por comas:

```
temperatures = [25.4, 32.8, 54.9, 15.2, 20.4]
hostnames = ['R1-GYE', 'R1-UIO', 'R3-CUE']

# Los elementos no necesariamente deben ser del mismo tipo:
example_list = [34.1, 27.8, 'R2-GYE', 'R2-UIO']
```

Las listas son extremadamente útiles cuando necesitas almacenar múltiples valores en una sola variable.

Puedes acceder a sus elementos utilizando índices (empezando desde 0), de forma similar a como lo hicimos anteriormente con los strings.

Sin embargo, a diferencia de los strings, las listas permiten modificar el valor correspondiente a cada índice.

```
hostnames = ['R1-GYE', 'R1-UIO', 'R1-CUE']

# Para acceder al primer elemento, usamos el índice 0:
print(hostnames[0]) # R1-GYE

# Para acceder al último elemento usamos el índice -1:
print(hostnames[-1]) # R1-CUE

# Cambiar el valor del elemento 3 (índice 2)
hostnames[2] = 'R1-LOJ'

print(hostnames) # ['R1-GYE', 'R1-UIO', 'R1-LOJ']
```

En automatización de redes, las listas se utilizan constantemente para almacenar hostnames, direcciones IP, VLANs, interfaces, alarmas o resultados obtenidos desde dispositivos.

3.3.2 Tuplas (tuples)

Una tupla es una colección ordenada e inmutable (es decir, no se puede alterar). Se define de manera similar a las listas, pero usando paréntesis:

```
router_coordinates = (-2.170997, -79.922359)
```

A diferencia de las listas:

- No pueden modificarse.
- Son más rápidas.
- Se usan cuando un valor no debe cambiar.

En redes suelen utilizarse para representar coordenadas geográficas, pares de valores o configuraciones que no deberían modificarse accidentalmente durante la ejecución del programa.

3.3.3 Conjuntos (sets)

Un set es una colección **desordenada** de elementos únicos.

Esto significa que no permite duplicados, lo cual es extremadamente útil en automatización de redes cuando quieres:

- Eliminar interfaces repetidas.
- Comparar listas de VLANs.
- Detectar IPs duplicadas.
- Encontrar qué elementos están en un grupo, pero no en otro.

Los **sets** se definen usando llaves `{ }`. Los elementos del **set** se separan con comas:

```
routers = {'R1-GYE', 'R2-GYE', 'R1-UIO', 'R1-CUE'}
```

Es importante recalcar la palabra **desordenada**. Si imprimes el contenido del **set**, Python no garantiza que los elementos se muestren en el mismo orden en que fueron declarados:

```
print(routers)
```

Resultado posible:

```
{'R1-CUE', 'R1-GYE', 'R2-GYE', 'R1-UIO'}
```

En otra ejecución, o incluso en otro entorno, el orden podría verse diferente:

```
{'R1-GYE', 'R2-GYE', 'R1-CUE', 'R1-UIO'}
```

Por esta razón, no debes usar un `set` cuando el orden de los elementos sea importante. Para esos casos, normalmente conviene usar una lista.

También es común utilizar primero un `set` para eliminar elementos duplicados y luego convertirlo nuevamente a una lista. Más adelante veremos cómo realizar este tipo de conversiones entre tipos de datos utilizando **casting**.

A diferencia de las listas, no podemos acceder a los elementos de un `set` utilizando índices, porque los elementos de un set no están ordenados como en una lista:

```
vlans = {10, 20, 30, 40}
vlans[0]
```

Al ejecutar el código, Python genera un error de tipo **TypeError**:

```
Traceback (most recent call last):
  File "test.py", line 3, in <module>
    vlans[0]
    ~~~~~^^^
TypeError: 'set' object is not subscriptable
```

🔴 Operaciones de conjuntos (muy útiles en redes)

Las operaciones con sets son extremadamente útiles para comparar información entre dispositivos, inventarios, VLANs, rutas o configuraciones.

- **Unión**

La unión combina todos los elementos de ambos conjuntos y elimina automáticamente los duplicados. Esto es útil, por ejemplo, cuando quieres obtener todas las VLANs utilizadas en diferentes grupos o dispositivos.

Se utiliza el operador `|`.

```
vlans_clients = {100, 200, 300}
vlans_admin = {10, 20, 30}

print(vlans_clients | vlans_admin)
# Resultado: {100, 20, 200, 10, 300, 30}
```

- **Intersección**

La intersección devuelve únicamente los elementos que existen en ambos conjuntos. Esto es extremadamente útil para comparar configuraciones y encontrar coincidencias entre dispositivos o servicios.

Se utiliza el operador `&`:

```
vlans_isp1 = {10, 200, 300}
vlans_isp2 = {10, 20, 30}

print(vlans_isp1 & vlans_isp2)
# Resultado: {10}
```

- **Diferencia**

La diferencia devuelve los elementos que existen en el primer conjunto, pero no en el segundo. Esto es útil para detectar configuraciones faltantes o diferencias entre dispositivos.

Se utiliza el operador `-`:

```
vlans_isp1 = {10, 200, 300}
vlans_isp2 = {10, 20, 30}

print(vlans_isp1 - vlans_isp2)
# Resultado: {200, 300}
```

3.3.4 Diccionarios (dict)

Un diccionario es un tipo de colección que almacena pares de claves (**keys**) y valores (**values**).

La idea es similar a un diccionario real:

- La palabra que buscas sería la clave.
- La definición asociada sería el valor.

Por ejemplo:

Clave	Valor
hostname	R1-GYE
ip	10.1.1.1

Este tipo de dato es útil cuando necesitas asociar información relacionada, como el nombre de un dispositivo y su dirección IP.

Los diccionarios se definen utilizando llaves `{}` y sus elementos están separados por comas. Cada elemento consta de una clave y un valor, separados por dos puntos.

```
router = {'hostname': 'R1-GYE', 'ip': '10.1.1.1'}
```

En este ejemplo:

- `'hostname'` e `'ip'` son las claves.
- `'R1-GYE'` y `'10.1.1.1'` son los valores asociados a esas claves.

Puedes acceder a los valores de un diccionario utilizando su clave y corchetes `[]`:

```
# Acceder al valor de la clave hostname:
hostname = router['hostname']
print(hostname) # R1-GYE

# Acceder al valor de la clave ip:
ip = router['ip']
print(ip) # 10.1.1.1
```

Los diccionarios permiten agregar, modificar o eliminar elementos de manera eficiente. Esto hace que sean extremadamente útiles cuando la información de un dispositivo puede cambiar o crecer dinámicamente durante la ejecución del programa:

```
router = {'hostname': 'R2_UIO'}

# Agregar nueva clave y valor:
router['uplink_interface'] = 'Gi0/1'

# Modificar un valor existente:
router['hostname'] = 'R1-GYE'

print(router)
```

Resultado:

```
{'hostname': 'R1-GYE', 'uplink_interface': 'Gi0/1'}
```

Podemos ver que el diccionario fue modificado y ahora contiene un nuevo par clave/valor.

Con diccionarios podemos representar información mucho más completa sobre un dispositivo, agrupando múltiples datos relacionados dentro de una sola estructura:

```
router = {
    'hostname': 'R1-GYE',
    'ip': '10.1.1.1',
    'sys_model': 'C810',
    'vendor': 'Cisco',
    'city': 'Guayaquil',
    'province': 'Guayas',
    'country': 'Ecuador'
}
```

En automatización de redes, los diccionarios son una de las estructuras más utilizadas para representar información de dispositivos, interfaces, métricas, configuraciones y respuestas de APIs. Gran parte de los datos que veremos más adelante en el libro estarán organizados utilizando diccionarios y estructuras similares.

3.3.5 Estructuras anidadas: listas y diccionarios complejos

En Python es muy común trabajar con estructuras de datos que contienen otras estructuras dentro de ellas.

Por ejemplo:

- Listas que contienen diccionarios.
- Diccionarios que contienen listas.
- Diccionarios dentro de otros diccionarios.

A este tipo de estructuras normalmente se las denomina estructuras anidadas.

En automatización de redes, este tipo de datos aparece constantemente:

- Inventarios de dispositivos.

- Respuestas JSON de APIs.
- Información SNMP.
- Configuraciones procesadas.
- Resultados de herramientas de monitoreo.

Por ejemplo, considera la siguiente estructura:

```
devices = [  
    {  
        'hostname': 'R1-GYE',  
        'interfaces': [  
            {'name': 'Gi0/1', 'status': 'UP'},  
            {'name': 'Gi0/2', 'status': 'DOWN'}  
        ]  
    },  
    {  
        'hostname': 'R1-UIO',  
        'interfaces': [  
            {'name': 'Gi0/3', 'status': 'DOWN'},  
            {'name': 'Gi0/4', 'status': 'UP'}  
        ]  
    }  
]
```

Aquí tenemos:

- Una lista llamada **devices**.
- Esta lista contiene diccionarios.
- Cada diccionario contiene las claves **hostname** e **interfaces**.
- La clave **interfaces** tiene como valor una lista con más diccionarios.

Aunque al inicio puede parecer complejo, realmente solo estamos combinando conceptos que ya conocemos:

- Listas
- Índices
- Diccionarios
- Claves

Por ejemplo, para obtener el estado de la interfaz **Gi0/2** usaríamos el siguiente código:

```
print(devices[0]['interfaces'][1]['status'])
```

Resultado:

DOWN

Veamos paso a paso lo que ocurre:

- **devices[0]**

Obtiene el primer elemento de la lista **devices**. Ese elemento es un diccionario:

```
{
    'hostname': 'R1-GYE',
    'interfaces': [
        {'name': 'Gi0/1', 'status': 'UP'},
        {'name': 'Gi0/2', 'status': 'DOWN'}
    ]
}
```

- **devices[0]['interfaces']**

Obtiene el valor asociado a la clave **interfaces** del primer diccionario. El valor asociado a esa clave es una lista:

```
[
    {'name': 'Gi0/1', 'status': 'UP'},
    {'name': 'Gi0/2', 'status': 'DOWN'}
]
```

- **devices[0]['interfaces'][1]**

Obtiene el segundo elemento de la lista asociada a la clave **interfaces**. Ese elemento es un diccionario:

```
{'name': 'Gi0/2', 'status': 'DOWN'}
```

- **devices[0]['interfaces'][1]['status']**

Obtiene el valor asociado a la clave **status** de ese diccionario. El valor final obtenido es:

```
DOWN
```

Este tipo de acceso es extremadamente común en Python moderno, especialmente al trabajar con APIs REST y datos JSON.

Más adelante en el libro veremos muchas estructuras similares, por lo que es importante acostumbrarse a leerlas paso a paso y entender cómo navegar entre listas y diccionarios anidados.

3.4 Operaciones sobre variables

Una vez que hemos aprendido sobre los diferentes tipos de datos en Python, es importante saber cómo realizar operaciones con ellos. Algunas operaciones básicas incluyen:

3.4.1 Conversión de tipos (casting)

En Python, casting significa convertir un dato de un tipo a otro: de texto a entero, de entero a texto, de lista a set, etc.

Esto es fundamental cuando procesas información que proviene de archivos, SNMP, APIs o configuraciones de dispositivos, ya que casi siempre los valores llegan como texto.

Las funciones más comunes para conversión son:

- `int()`: Convierte a entero.
- `float()`: Convierte a número decimal.
- `str()`: Convierte a cadena de texto.
- `list()`: Convierte a lista.
- `set()`: Convierte a conjunto.
- `dict()`: Convierte a diccionario (solo en casos específicos).

Ejemplos aplicados al mundo de redes

- Convertir un valor SNMP (que llega como `string`) a entero para poder compararlo con un valor umbral (límite definido):

```
cpu = '85'           # cpu es un string
cpu = int(cpu)       # Lo convertimos a entero
if cpu > 80:
    print('¡Alto consumo de CPU!')
```

- Convertir una VLAN a `string` para construir nombres:

```
vlan = 200
vlan_name = 'VLAN_' + str(vlan)
print(vlan_name)    # Resultado: VLAN_200
```

- Convertir una lista a `set` para eliminar duplicados:

```
vlangs = [10, 20, 20, 30, 30, 30]
vlangs_unique = set(vlangs)
print(vlangs_unique) # Resultado: {10, 20, 30}
```

- Convertir un `set` a lista para ordenarlo:

```
vlangs = {30, 10, 20}
vlangs_sorted = list(vlangs)
vlangs_sorted.sort()
print(vlangs_sorted) # [10, 20, 30]
```

⚠ **Nota importante:** Si intentas convertir un valor inválido, Python generará un error:

```
int('abc')
```

Resultado:

```
Traceback (most recent call last):
  File "C:test.py", line 1, in <module>
    int('abc')
ValueError: invalid literal for int() with base 10: 'abc'
```

[Más adelante](#) aprenderás a manejar excepciones con `try/except` para evitar que tu programa falle.

3.4.2 Operaciones matemáticas

Puedes realizar operaciones matemáticas con variables que contienen números enteros o flotantes (números con decimales):

```
a, b = 6, 7

print(a + b) # Suma: 13

print(a - b) # Resta: -1

print(a * b) # Multiplicación: 42

print(a / b) # División: 0.8571428571428571

print(a // b) # División entera (cociente): 0

print(a % b) # Módulo (resto de la división): 6

print(a ** 2) # Potencia: 36

print(a ** 0.5) # Raíz cuadrada (con potencia): 2.449489742783178
```

Estas operaciones son fundamentales en programación y se utilizan constantemente en escenarios reales, por ejemplo, para convertir unidades, calcular porcentajes, analizar métricas, procesar tráfico de red o trabajar con tiempos y estadísticas.

También es común utilizar operadores abreviados para actualizar variables numéricas.

Por ejemplo:

```
counter = 10
counter += 1

print(counter)
# Resultado: 11
```

La línea:

```
counter += 1
```

equivale a:

```
counter = counter + 1
```

Esta sintaxis es muy utilizada en programación porque hace el código más corto y fácil de leer:

Forma tradicional	Forma abreviada
<code>x = x + 5</code>	<code>x += 5</code>
<code>x = x - 3</code>	<code>x -= 3</code>
<code>x = x * 2</code>	<code>x *= 2</code>
<code>x = x / 4</code>	<code>x /= 4</code>

3.4.3 Cadenas con formato (f-string)

Una cadena con formato (**f-string**) es la forma más moderna, rápida y legible de incluir variables o expresiones dentro de una cadena de texto en Python. Se introdujeron en la versión 3.6 de Python y se han convertido en el estándar de la industria.

Para crear una, simplemente pones una letra **f** (o **F**) antes de las comillas de tu texto y encierras entre llaves **{ }** lo que quieras insertar. Esto permite reemplazar automáticamente las variables por sus valores dentro de la cadena de texto.

La estructura general de un f-string es:

```
f'Texto {variable} más texto {otra_variable}'
```

Se pueden usar comillas simples o dobles, al igual que con cualquier otro **string**. No hay límite práctico para la cantidad de texto, variables o expresiones que se pueden utilizar dentro de un f-string.

Si no utilizamos f-strings, Python interpreta el contenido como texto literal. Las variables escritas entre llaves **{ }** no son reemplazadas por sus valores:

```
hostname, port = 'R1-GYE', 22
print('Conectando a {hostname} por el puerto {port}...')
```

Resultado:

```
Conectando a {hostname} por el puerto {port}...
```

Utilizando f-strings, Python sí reemplaza automáticamente las variables por sus valores reales:

```
print(f'Conectando a {hostname} por el puerto {port}...')
```

Resultado:

```
Conectando a R1-GYE por el puerto 22...
```

Los f-strings son una de las herramientas más utilizadas en Python moderno. A lo largo del libro los utilizaremos constantemente para construir mensajes, logs, comandos, URLs y muchos otros textos dinámicos de forma clara y sencilla.

✦ Alineación y ancho fijo con f-strings

Los f-strings permiten no solo insertar variables, sino también controlar el formato del texto. La estructura general es:

```
f' {valor:formato} '
```

Dentro de las llaves {}:

- Antes de los dos puntos (:) va el valor o variable que queremos mostrar.
- Después de los dos puntos va el formato que queremos aplicar.

Por ejemplo:

```
print(f"{'Hostname':<15}")
```

En este caso:

- 'Hostname' es el valor que queremos imprimir (si es un **string** debe ir entre comillas dobles o simples).
- : indica que vamos a aplicar un formato.
- < significa alinear a la izquierda.
- 15 indica que se reservarán 15 caracteres de ancho.

Es decir, Python imprimirá 'Hostname' ocupando un espacio total de 15 posiciones.

Podemos utilizar este tipo de formato para mostrar información alineada y más fácil de leer:

```
print(f"{'Hostname':<20} {'IP':<15} {'Fabricante':<15}")
print(f"{'R1-GYE':<20} {'10.1.1.1':<15} {'Cisco':<15}")
print(f"{'R1-UIO':<20} {'10.2.1.1':<15} {'Huawei':<15}")
print(f"{'R1-CUE':<20} {'10.3.1.1':<15} {'Juniper':<15}")
```

Resultado:

Hostname	IP	Fabricante
R1-GYE	10.1.1.1	Cisco
R1-UIO	10.2.1.1	Huawei

R1-CUE	10.3.1.1	Juniper
--------	----------	---------

Este formato también se puede utilizar junto con variables:

```
hostname1, hostname2, hostname3 = 'R1-GYE', 'R1-UIO', 'R1-CUE'
ip1, ip2, ip3 = '10.1.1.1', '10.2.1.1', '10.3.1.1'
vendor1 = 'Cisco'
vendor2 = 'Huawei'
vendor3 = 'Juniper'

print(f"{'Hostname':<20} {'IP':<15} {'Fabricante':<15}")
print(f"{'hostname1':<20} {'ip1':<15} {'vendor1':<15}")
print(f"{'hostname2':<20} {'ip2':<15} {'vendor2':<15}")
print(f"{'hostname3':<20} {'ip3':<15} {'vendor3':<15}")
```

Resultado:

Hostname	IP	Fabricante
R1-GYE	10.1.1.1	Cisco
R1-UIO	10.2.1.1	Huawei
R1-CUE	10.3.1.1	Juniper

Esto permite mostrar información alineada y mucho más fácil de leer en consola.

Opciones de alineación

Podemos alinear el texto a la izquierda, a la derecha o centrado, utilizando diferentes operadores:

- < alinea a la izquierda.
- > alinea a la derecha.
- ^ centra el texto.

Ejemplo:

```
# Alineación a la izquierda
print(f"{'Hostname':<20} {'IP':<15} {'Fabricante':<15}")
print(f"{'R1-GYE':<20} {'10.1.1.1':<15} {'Cisco':<15}")

print()

# Alineación a la derecha
print(f"{'Hostname':>20} {'IP':>15} {'Fabricante':>15}")
print(f"{'R1-GYE':>20} {'10.1.1.1':>15} {'Cisco':>15}")

print()

# Alineación centrada
print(f"{'Hostname':^20} {'IP':^15} {'Fabricante':^15}")
print(f"{'R1-GYE':^20} {'10.1.1.1':^15} {'Cisco':^15}")
```

Resultado:

```

Hostname      IP      Fabricante
R1-GYE        10.1.1.1  Cisco

      Hostname      IP      Fabricante
      R1-GYE        10.1.1.1  Cisco

      Hostname      IP      Fabricante
      R1-GYE        10.1.1.1  Cisco

```

Este tipo de formato es muy útil en automatización cuando necesitas presentar resultados estructurados en pantalla sin utilizar librerías externas.

3.4.4 Repetición de strings y listas con el operador *

El operador * no solo se utiliza para multiplicaciones matemáticas.

En Python, también puede utilizarse con strings y listas para repetir su contenido múltiples veces.

Ejemplo con strings:

```
print('==-' * 5)
```

Resultado:

```
==--==--==--
```

Esto es útil, por ejemplo, para generar separadores visuales en reportes o salidas de consola.

Ejemplo con listas:

```
interfaces = ['Gi0/0', 'Gi0/1']
print(interfaces * 2)
```

Resultado:

```
['Gi0/0', 'Gi0/1', 'Gi0/0', 'Gi0/1']
```

En este caso, Python repite el contenido de la lista dos veces.

3.5 Secuencias

En Python, algunos tipos de datos almacenan elementos en un **orden definido**. Es decir, el orden de los elementos no cambia.

A estos tipos se los puede tratar como secuencias. Entre los más comunes están `str`, `list` y `tuple`.

Podemos ejecutar operaciones muy útiles con las secuencias:

- Validar la longitud.
- Acceder por índice.
- Aplicar slicing.
- Usar el operador `in`.
- Desempaquetar variables (unpacking).

3.5.1 Validar la longitud

La función `len()` permite obtener la longitud de una secuencia.

La forma general de la función es:

```
len(coleccion)
```

Si utilizamos `len()` con una lista, vamos a obtener la cantidad de elementos:

```
routers = ['R1-GYE', 'R1-UIO', 'R2-GYE']  
print(len(routers))  
# Resultado: 3
```

Si lo utilizamos con un diccionario, `len()` devuelve la cantidad de pares clave/valor:

```
router = {'hostname': 'R1-GYE', 'ip': '10.1.1.1'}  
print(len(router))  
# Resultado: 2
```

También puede utilizarse con strings. En este caso, `len()` retorna la cantidad de caracteres:

```
hostname = 'R1-GYE'  
print(len(hostname))  
# Resultado: 6
```

3.5.2 Acceder por índice

Anteriormente ya vimos cómo utilizar índices en strings y listas. Aunque el concepto pueda parecer repetitivo, es importante reforzarlo porque el acceso por índice es una de las operaciones más utilizadas en Python.

Ahora ampliaremos ese concepto para trabajar de manera más general con todas las secuencias.

Como ya definimos al inicio de la sección, las secuencias son estructuras que mantienen un orden estricto en sus elementos, como:

- Strings
- Listas
- Tuplas

Cada posición dentro de una secuencia tiene un número asignado llamado índice.

 **Nota importante:** No todas las colecciones permiten acceso por índice. Por ejemplo, los **sets** no tienen un orden definido, por lo que no es posible acceder a sus elementos utilizando índices.

En Python, el indexado funciona de dos maneras:

- **Índices positivos:** Empiezan desde 0 (primer elemento) y avanzan hacia la derecha.
- **Índices negativos:** Empiezan desde -1 (último elemento) y avanzan hacia la izquierda.

Para acceder a un elemento mediante su índice, utilizamos corchetes []:

```
variable[índice]
```

Acceso en strings

En un string, podemos acceder a caracteres específicos:

```
interface_status = ('GigabitEthernet0/1 is down, '
                    'line protocol is down (notconnect)')
print(f'String: {interface_status}')

# El primer carácter (índice 0)
first_char = interface_status[0]
print(f'Primer carácter: "{first_char}"')

# El último carácter (índice -1)
last_char = interface_status[-1]
print(f'Último carácter: "{last_char}"')
```

Resultado:

```
String: GigabitEthernet0/1 is down, line protocol is down (notconnect)
Primer carácter: "G"
Último carácter: ")"
```

Recuerda que, con los strings, solo podemos leer los elementos, no modificarlos individualmente, o se generará un error de tipo **TypeError**.

Acceso en listas

En una lista, podemos acceder a cualquier elemento:

```
routers = ['R1-GYE', 'R2-GYE', 'R1-UIO', 'R2-UIO']
print(f'Lista: {routers}')
```

```
# El segundo elemento (índice 1)
second_router = routers[1]

print(f'Segundo router: {second_router}')
```

Resultado:

```
Lista: ['R1-GYE', 'R2-GYE', 'R1-UIO', 'R2-UIO']
Segundo router: R2-GYE
```

Las listas, a diferencia de los strings, sí soportan modificación de elementos individuales mediante índice.

Acceso en listas anidadas

A veces, una lista puede contener otras listas. Para acceder a un dato interno, usamos corchetes consecutivos:

```
# Una lista de listas que contiene [Hostname, IP]
devices = [['R1', '10.1.1.1'], ['R2', '10.1.1.2']]

# Acceder a la IP del primer router (R1)
# [0] elige el primer par, [1] elige la IP dentro de ese par
print(devices[0][1])
```

Resultado:

```
10.1.1.1
```

Acceso en tuplas

En una tupla, podemos también acceder a cualquier elemento:

```
tcp_ports = (22, 23, 80, 8080)

print(f'Tupla: {tcp_ports}')

# El último elemento (índice -1)
print(f'Último elemento: {tcp_ports[-1]}')
```

Resultado:

```
Tupla: (22, 23, 80, 8080)
Último elemento: 8080
```

Las tuplas, por definición, no se pueden modificar luego de haber sido creadas.

⚠ La excepción `IndexError`

Es importante recordar que si intentas acceder a un índice que no existe (por ejemplo, el índice 5 en una lista de solo 3 elementos), Python detendrá el programa con un error:

```
vlans = [10, 20, 30]
print(vlans[5])
```

Resultado:

```
Traceback (most recent call last):
  File "test.py", line 3, in <module>
    print(vlans[5])
    ~~~~~^^^
IndexError: list index out of range
```

Para evitar este tipo de error, siempre verifica el tamaño de tu colección con la función `len()` antes de intentar acceder a índices:

```
vlans = [10, 20, 30]
index = 5

if index < len(vlans):
    print(vlans[index])
else:
    print(f'No se puede acceder al índice {index} '
          f'porque la colección tiene solo {len(vlans)} '
          f'elementos.')
```

Resultado:

```
No se puede acceder al índice 5 porque la colección tiene solo 3
elementos.
```

3.5.3 Slicing

En Python, muchas estructuras de datos permiten acceder no solo a un elemento específico, sino también a **un subconjunto de elementos**. A esta operación se le conoce como **slicing**.

El slicing permite seleccionar una parte de una secuencia utilizando la siguiente sintaxis:

```
secuencia[inicio:fin:incremento]
```

Donde:

- **inicio**: Índice donde comienza la selección. Si no se especifica, se usa 0 por defecto.
- **fin**: Índice donde termina la selección (sin incluir ese elemento). Si no se especifica, se usa la longitud de la secuencia por defecto.
- **incremento**: Indica cada cuántos elementos avanzar dentro de la secuencia. Si no se especifica, se usa 1 por defecto.

En la mayoría de los casos se utilizan únicamente los parámetros **inicio** y **fin**.

El slicing funciona con varios tipos de datos en Python, entre ellos:

- strings
- listas
- tuplas

Esto lo convierte en una herramienta muy útil para procesar información en scripts de automatización.

Ejemplo con strings

```
ip_address = '192.168.1.100'

# Todos los caracteres
print(ip_address[:])

# Desde el índice 4 hasta el final
print(ip_address[4:])

# Desde el inicio hasta el índice 3 (sin incluirlo)
print(ip_address[:3])

# Desde el índice 4 hasta el 7 (cuarto octeto)
print(ip_address[4:7])
```

Resultado:

```
192.168.1.100
168.1.100
192
168
```

En este caso estamos seleccionando diferentes partes de la dirección IP utilizando posiciones dentro de la cadena.

Este tipo de operación es común cuando se necesita **extraer partes específicas de un texto**, como segmentos de una dirección IP, identificadores dentro de un hostname o fragmentos de una salida de comandos.

Ejemplo con listas

El slicing también puede utilizarse con listas:

```
routers = ['R1-GYE', 'R2-GYE', 'R1-UIO', 'R1-CUE']

# Desde el primer elemento hasta el índice 2, sin incluirlo
print(routers[:2])

# Desde el elemento 1 hasta el índice 3, sin incluirlo
print(routers[1:3])
```

```
# Del índice 1 al 7, no importa si no existe:
print(routers[1:7])

# Toda la lista, al revés
print(routers[::-1])
```

Resultado:

```
['R1-GYE', 'R2-GYE']
['R2-GYE', 'R1-UIO']
['R2-GYE', 'R1-UIO', 'R1-CUE']
['R1-CUE', 'R1-UIO', 'R2-GYE', 'R1-GYE']
```

Esto permite trabajar con **subconjuntos de datos**, algo muy útil cuando se procesan listas grandes de dispositivos, interfaces o direcciones IP.

✚ Uso del incremento

El tercer parámetro permite indicar el **paso** entre elementos.

```
routers = ['R1-GYE', 'R2-GYE', 'R1-UIO', 'R1-CUE']
print(routers[:2])
```

Resultado:

```
['R1-GYE', 'R1-UIO']
```

En este ejemplo se selecciona **cada dos elementos** de la lista.

✚ **Nota práctica:** No es necesario memorizar todas las combinaciones posibles de slicing.

Lo importante es entender que Python permite **trabajar con fragmentos de una secuencia de forma sencilla**, lo cual resulta muy útil al procesar datos provenientes de configuraciones de red, archivos o resultados de comandos.

3.5.4 El operador `in`

Ya hemos utilizado `in` cuando hemos usado el lazo `for`. Sin embargo, aquí lo definimos formalmente.

El operador `in` es un operador de pertenencia.

Se utiliza para comprobar si un elemento está contenido dentro de una colección o si una subcadena aparece dentro de una cadena de texto.

Este operador devuelve un valor booleano:

- **True** si el elemento está presente.
- **False** si el elemento no está presente.

La forma general de usarlo es:

```
valor in colección
```

Utilizando el operador `in` con listas

```
routers = ['R1-GYE', 'R1-UIO', 'R2-GYE']  
print('R1-GYE' in routers)
```

Resultado:

```
True
```

Python verifica si el valor `'R1-GYE'` se encuentra dentro de la lista `routers`.

Utilizando el operador `in` con sets

Es muy parecido al utilizado con listas:

```
vlangs = {10, 20, 30}  
print(20 in vlangs)  
print(40 in vlangs)
```

Resultado:

```
True  
False
```

Utilizando el operador `in` con strings

El operador `in` también puede utilizarse con cadenas de texto:

```
hostname = 'R1-GYE'  
if 'GYE' in hostname:  
    print('El equipo pertenece a Guayaquil')
```

Resultado:

```
El equipo pertenece a Guayaquil
```

En este caso, Python verifica si la cadena `'GYE'` aparece dentro del texto almacenado en `hostname`.

▶ Utilizando el operador `in` con diccionarios

En un diccionario, `in` verifica si la clave existe, no si el valor existe.

```
router = {'hostname': 'R1-GYE', 'ip': '10.1.1.1'}

print('hostname' in router)
print('R1-GYE' in router)
print('ip' in router)
print('10.1.1.1' in router)
```

Resultado:

```
True
False
True
False
```

▶ Invirtiendo la lógica con `not in`

Para verificar pertenencia utilizamos `in`. Pero si necesitamos validar si un elemento **no pertenece** a la colección, usamos `not` antes de `in`:

```
valor not in colección
```

Ejemplo:

```
vlan = {10, 20, 30}
print(40 not in vlan)
print(40 in vlan)
```

Resultado:

```
True
False
```

🔴 Uso en automatización de redes

El operador `in` es muy útil cuando necesitamos:

- Verificar si un dispositivo está en una lista de inventario.
- Buscar un patrón dentro de un hostname.
- Comprobar si una VLAN o interfaz aparece en un conjunto de datos.

A lo largo del libro veremos este operador con frecuencia al trabajar con listas de dispositivos, configuraciones y resultados de comandos.

3.5.5 Desempaquetamiento de variables (unpacking)

En Python es posible asignar múltiples valores a varias variables en una sola instrucción. A esta técnica se le conoce como desempaquetamiento de variables (**unpacking**).

Por ejemplo:

```
x, y, z = 1, 2, 3

print(x)
print(y)
print(z)
```

Resultado:

```
1
2
3
```

En este caso:

- **x** toma el valor de 1.
- **y** tomar el valor de 2.
- **z** toma el valor de 3.

Aunque no lo parezca a simple vista, Python interpreta los valores del lado derecho como una colección temporal y asigna automáticamente cada elemento a su variable correspondiente.

La idea es simple: Python toma los elementos de una colección y los asigna automáticamente a múltiples variables.

La forma general es la siguiente:

```
variable1, variable2 = valores
```

Python asigna el primer elemento de la colección a la primera variable, el segundo elemento a la segunda variable, y así sucesivamente.

Ejemplo con tuplas

Un caso muy común ocurre con tuplas:

```
router = ('R1-GYE', '10.0.0.1')
hostname, ip = router
print(hostname)
print(ip)
```

Resultado:

```
R1-GYE
10.0.0.1
```


En este ejemplo:

- **hostname** recibe el primer elemento de la tupla.
- **ip** recibe el segundo elemento.

Este mecanismo hace que el código sea **más limpio y fácil de leer**, evitando acceder a los valores manualmente mediante índices.

Por ejemplo, sin **unpacking** tendríamos que escribir:

```
hostname = router[0]
ip = router[1]
```

Ejemplo con listas

También podemos usar unpacking con **listas**:

```
data = ['R1-GYE', '10.0.0.1']
hostname, ip = data

print(hostname)
print(ip)
```

Resultado:

```
R1-GYE
10.0.0.1
```

Mientras el número de variables coincida con el número de elementos de la colección, Python realizará la asignación automáticamente.

Ejemplo con diccionarios

También es muy común usar unpacking cuando recorremos diccionarios utilizando **items()**.

El método **items()** de los diccionarios devuelve pares **clave-valor**:

```
router = {'hostname': 'R1-GYE', 'ip': '192.168.1.1'}

for key, value in router.items():
    print(f'La clave "{key}" tiene el valor: {value}')
```

Resultado:

```
La clave "hostname" tiene el valor: R1-GYE
La clave "ip" tiene el valor: 192.168.1.1
```

En cada iteración del ciclo **for**, Python desempaqueta automáticamente el par **clave-valor** en las variables **key** y **value**.

Este patrón es muy común cuando se procesan inventarios de dispositivos, configuraciones o datos estructurados.

⚠ **Nota importante:** Para que el unpacking funcione correctamente, el número de variables debe coincidir con el número de elementos de la colección.

Por ejemplo, el siguiente código generará un error:

```
router = ('R1-GYE', '10.0.0.1')  
  
hostname, ip, vendor = router
```

Python producirá una excepción porque hay **más variables que valores disponibles**:

```
Traceback (most recent call last):  
  File "C:test.py", line 4, in <module>  
    hostname, ip, vendor = router  
    ^^^^^^^^^^^^^^^^^^^^^  
ValueError: not enough values to unpack (expected 3, got 2)
```

📌 En automatización de redes

Utilizamos unpacking con frecuencia cuando:

- Recorremos diccionarios de inventarios.
- Procesamos resultados estructurados.
- Trabajamos con funciones que devuelven múltiples valores.

A lo largo del libro verás este patrón en varios scripts, ya que permite escribir código **más claro y expresivo**.

3.6 Iterables

En Python, un iterable es cualquier objeto que puede recorrerse elemento por elemento, normalmente usando un lazo como `for`.

A este proceso se le conoce como **iterar**: es decir, recorrer cada elemento de una colección uno por uno.

Iterables más comunes

Los tipos más comunes de iterables son:

- Listas (`list`).
- Cadenas de caracteres (`str`).
- Rangos (`range`).
- Diccionarios (`dict`).
- Conjuntos (`set`).

[Más adelante](#) explicaremos en detalle la función `range()`.

Ejemplo sencillo del uso de un iterable:

```
for hostname in ['R1-GYE', 'R2-UIO']:
    print(hostname)
```

Al ejecutar el código, se imprimirá lo siguiente en pantalla:

```
R1-GYE
R2-UIO
```

¿Qué es un generador?

Un generador es un iterable que no almacena todos los valores en memoria, sino que los produce uno a uno cuando se necesitan.

En Python, los generadores se crean comúnmente mediante expresiones generadoras o funciones que utilizan la palabra clave `yield`.

Algunos objetos del lenguaje, como `range()`, también producen valores bajo demanda, aunque técnicamente no son generadores.

Ejemplo simple:

```
numbers = (n * n for n in range(5))
for n in numbers:
    print(n)
```

 **Nota importante:** Algunos iterables, especialmente generadores e iteradores, producen los elementos uno a uno y se consumen a medida que se recorren.

Por esta razón, después de iterarlos completamente, normalmente ya no tendrán más elementos disponibles a menos que se creen nuevamente.

En muchos casos, también es posible convertirlos a una lista utilizando `list()` para mantener todos los elementos disponibles en memoria.

Más adelante veremos ejemplos reales de este comportamiento con herramientas como `re.finditer()` y `csv.DictReader()`.

3.7 Comparación y valores booleanos

Los operadores de comparación permiten evaluar si dos valores son iguales, diferentes o si uno es mayor/menor que otro.

Estas comparaciones devuelven siempre un valor booleano.

Una comparación no modifica variables. Su resultado es simplemente **True** o **False**.

A continuación, se presentan los operadores más utilizados:

Operador	Significado	Ejemplo	Resultado
<code>==</code>	Igual a	<code>5 == 5</code>	True
<code>!=</code>	Diferente de	<code>5 != 3</code>	True
<code>></code>	Mayor que	<code>10 > 5</code>	True
<code><</code>	Menor que	<code>2 < 8</code>	True
<code>>=</code>	Mayor o igual	<code>5 >= 5</code>	True
<code><=</code>	Menor o igual	<code>3 <= 2</code>	False

 **Nota Importante:** Confundir `==` con `=` es uno de los errores más comunes al empezar a programar con Python. Recuerda siempre:

- `=` asigna una variable (se usa un símbolo igual)
- `==` compara dos valores (se usa dos símbolos igual)

En automatización de redes, las comparaciones permiten validar condiciones y detectar situaciones específicas, como alto uso de CPU, interfaces caídas o diferencias entre estados operativos.

Comparar el uso de CPU de un dispositivo

Por ejemplo, podemos verificar si el uso de CPU supera un valor límite:

```
cpu_usage = 70
if cpu_usage >= 70:
    print('¡Alto uso de CPU!')
```

Comparar interfaces activas

También podemos comparar distintos estados para detectar inconsistencias:

```
admin_status = 'up'
oper_status = 'down'

if admin_status != oper_status:
    print('¡Inconsistencia detectada en la interfaz!')
```

3.8 Mutabilidad e inmutabilidad

En Python, algunos objetos pueden cambiar su contenido después de haber sido creados, mientras que otros no.

A esta característica se la conoce como mutabilidad:

- Un objeto **mutable** puede modificarse sin crear un objeto nuevo.
- Un objeto **immutable** no puede modificarse una vez creado.

Por ejemplo, las listas son mutables:

```
interfaces = ['Te0/1', 'Gi0/1']
interfaces[0] = 'Gi0/0'

print(interfaces)
```

Resultado:

```
['Gi0/0', 'Gi0/1']
```

Aquí la lista no cambia de identidad, pero su contenido sí se modificó.

La lista sigue siendo el mismo objeto en memoria; lo que cambia es su contenido.

En cambio, las tuplas son inmutables:

```
coordinates = (-2.17, -79.92)
coordinates[0] = 10
```

Esto genera un error, porque no es posible modificar un elemento de una tupla después de haberla creado:

```
Traceback (most recent call last):
  File "test.py", line 4, in <module>
    coordinates[0] = 10
    ~~~~~^
TypeError: 'tuple' object does not support item assignment
```

Los strings también son inmutables. Aunque puedes acceder a sus caracteres por índice, no puedes reemplazarlos directamente:

```
hostname = 'R1-GYE'
hostname[0] = 'S'
```

Esto también produce un error, porque una vez creada la cadena, su contenido no puede alterarse carácter por carácter:

```
Traceback (most recent call last):
  File "test.py", line 3, in <module>
    hostname[0] = 'S'
    ~~~~~^
TypeError: 'str' object does not support item assignment
```

Si necesitas *cambiar* un string, en realidad debes crear uno nuevo:

```
hostname = 'R1-GYE'
hostname = 'R2-UIO'
```

En este caso, Python no modificó la cadena original.

Lo que hizo fue crear una nueva cadena y asignarla a la variable `hostname`.

Esa distinción es fundamental porque muchos comportamientos internos de Python dependen de si un objeto puede cambiar o no después de haber sido creado:

```
# Lista: mutable
interfaces = ['Gi0/0', 'Gi0/1']
interfaces[0] = 'Te0/0'

# String: immutable
hostname = 'R1-GYE'
hostname[0] = 'S'    # Error
```

En ambos casos estamos intentando cambiar un elemento por índice:

- En la lista sí se puede, porque es mutable.
- En el string no se puede, porque es inmutable.

3.8.1 Tipos mutables e inmutables

En la siguiente tabla se detallan los tipos de datos que hemos visto, según sean mutables o no:

Categoría	Tipo de Dato	¿Se puede modificar el contenido original?
Inmutables	<code>int</code>	No. Cualquier cambio crea un objeto nuevo en memoria.
	<code>float</code>	
	<code>str</code>	
	<code>tuple</code>	
Mutables	<code>list</code>	Sí. Puedes añadir, eliminar o cambiar elementos internamente.
	<code>set</code>	
	<code>dict</code>	

En tipos como `int` y `float`, no existe una operación que modifique *una parte interna* del valor:

```
temperature = 25.1
temperature = temperature + 1
print(temperature)
```

En Python, las variables no contienen directamente el valor en sí, sino una referencia al objeto almacenado en memoria.

De forma simplificada, puedes imaginar que la variable funciona como una etiqueta o nombre que Python utiliza para acceder a un valor.

Cuando escribes `temperature = temperature + 1`, Python no modifica el valor anterior:

- Calcula un nuevo valor y crea un nuevo objeto en memoria.
- El valor `25.1` no fue modificado para convertirse en `26.1`.
- Simplemente se creó el valor `26.1` y la variable `temperature` pasó a referirse a ese nuevo valor.

3.9 Objetos hashables

En Python, algunos objetos pueden utilizarse como identificadores dentro de ciertas colecciones.

A estos objetos se les llama **objetos hashables**. Es un término en inglés que proviene de la función **hash** interna de Python.

En Python, para que un objeto pueda usarse como clave de diccionario o como elemento de un set, debe ser hashable.

En la práctica, los objetos mutables como `list`, `dict` y `set` no son hashables, porque su contenido puede cambiar.

Muchos objetos inmutables, como `int`, `str` y `tuple`, sí son hashables.

En general, los objetos usados como claves o elementos de sets deben tener un valor estable. Por eso, normalmente los tipos inmutables son los adecuados.

En resumen, si un objeto no es hashable, no puede ser utilizado en un set o como clave de un diccionario:

- No puedes tener elementos tipo `dict`, `list` o `set` dentro de un set.
- No puedes tener claves tipo `dict`, `list` o `set` en un diccionario.

Ejemplo correcto:

```
vlans = {10, 20, 30}
print(vlans)
```

Resultado:

```
{10, 20, 30}
```

Ejemplo incorrecto usando elementos no hashables:

```
management_vlans, clients_vlans = [1, 10], [250, 251, 252]

# No se pueden crear sets con elementos tipo lista:
vlans = {management_vlans, clients_vlans}
print(vlans)
```

Resultado:

```
Traceback (most recent call last):
  File "test.py", line 6, in <module>
    vlans = {management_vlans, clients_vlans}
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: unhashable type: 'list'
```

En estos casos Python genera una excepción **TypeError** por usar una variable de tipo unhashable (una lista es un objeto mutable).

Ejemplo con diccionarios:

```
router = {  
    ('R1-GYE', 'Gi0/0'): 'UP'  
}  
print(router)
```

Resultado:

```
{ ('R1-GYE', 'Gi0/0'): 'UP' }
```

En este caso, la clave es una **tupla**, que sí es un objeto hashable, por lo que podemos crear el diccionario sin generar errores.

3.10 Resumen

En este capítulo aprendiste cómo Python maneja la información a través de variables y distintos tipos de datos.

Revisaste tipos básicos como enteros, flotantes, cadenas de texto, booleanos y `None`. También entendiste cómo trabajar con colecciones como listas, tuplas, sets y diccionarios, que permiten organizar múltiples valores dentro de una sola variable.

Además, exploraste operaciones fundamentales como conversión de tipos, operaciones matemáticas, `f-strings`, acceso por índice, slicing, el operador `in` y desempaqueado de variables. Estos conceptos son esenciales para procesar datos provenientes de archivos, APIs, configuraciones y salidas de comandos.

Finalmente, estudiaste conceptos más profundos como iterables, mutabilidad, inmutabilidad y objetos hashables, que explican por qué algunos datos pueden modificarse y otros no, y por qué ciertos objetos pueden utilizarse como claves de diccionarios o elementos de sets.

Con estos conceptos, ya tienes una base sólida para entender cómo Python representa, organiza y transforma la información. Más adelante comenzarás a trabajar activamente con estos datos, utilizando funciones y métodos para manipularlos en escenarios reales de automatización.

Capítulo 4 - Operaciones y manipulación de datos

Hasta este punto ya conocemos las principales estructuras de datos de Python y cómo almacenar información en ellas. Ahora comenzaremos a utilizarlas para procesar información de forma más dinámica y útil.

Gran parte de la programación consiste precisamente en transformar datos: ordenarlos, filtrarlos, validarlos, combinar información y generar nuevos resultados a partir de ellos.

Las herramientas que veremos en este capítulo aparecen constantemente en automatización real, especialmente al trabajar con inventarios, configuraciones, direcciones IP, logs y procesamiento de información proveniente de la red.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Utilizar funciones integradas como `sorted()`, `sum()`, `min()` y `max()`.
- Comprender qué son los métodos y cómo utilizarlos correctamente.
- Manipular cadenas de texto usando métodos como `split()`, `join()` y `replace()`.
- Trabajar con listas, sets y diccionarios utilizando sus métodos más comunes.
- Comprender la diferencia entre `append()` y `extend()`.
- Encadenar métodos para transformar datos de forma más eficiente.
- Utilizar el módulo `ipaddress` para trabajar con redes y direcciones IP.
- Validar si una IP pertenece a una subred determinada.
- Procesar inventarios básicos de dispositivos utilizando listas y diccionarios.
- Generar resultados estructurados a partir de datos de red.

4.1 Funciones integradas más utilizadas

Python incluye varias **funciones integradas** que pueden utilizarse directamente sobre distintos tipos de datos.

4.1.1 La función `sorted()`

La función `sorted()` devuelve una **nueva lista ordenada** a partir de un iterable. La forma general de la función es:

```
vlans = [30, 10, 20]
print(sorted(vlans))
```

Resultado:

```
[10, 20, 30]
```

Esto es útil cuando necesitamos presentar datos de forma ordenada en pantalla o generar reportes más legibles.

Un uso muy común de `sorted()` es utilizarlo con sets para poder ordenar los elementos, que de otro modo siempre estarían desordenados:

```
vlans = {30, 10, 20}
print(sorted(vlans))
```

Resultado:

```
[10, 20, 30]
```

4.1.2 La función `sum()`

La función `sum()` permite sumar los elementos numéricos de una colección:

```
traffic_values = [120.12, 300.96, 150.05]
print(f'El tráfico total es: {sum(traffic_values)}')
```

Resultado:

```
El tráfico total es: 571.13
```

4.1.3 Las funciones `min()` y `max()`

Las funciones `min()` y `max()` permiten obtener el valor más pequeño o más grande de una colección.

```
temperatures = [25.4, 32.8, 18.1, 29.0]
```

```
print(f'La temperatura mínima es: {min(temperatures)}°C.'
      f' y la máxima: {max(temperatures)}°C')
```

Resultado:

```
La temperatura mínima es: 18.1°C. y la máxima: 32.8°C
```

4.1.4 La función `type()`

La función `type()` permite conocer el tipo al que pertenece un objeto. En automatización de redes, muchas librerías externas (como `paramiko` o `requests`) devuelven objetos complejos que no siempre son simples cadenas de texto, listas o diccionarios.

Saber exactamente qué tipo de objeto tienes entre manos es la diferencia entre un script que funciona y uno que lanza un error inesperado:

```
# Objeto de clase list
routers = ['R1-GYE', 'R2-GYE']
print(type(routers))

# Objeto de clase dict
routers = {'hostname': 'R1-GYE'}
print(type(routers))

# Objeto de clase str
routers = 'R1-GYE'
print(type(routers))

# Objeto de clase NoneType
routers = None
print(type(routers))

# Objeto de clase bool
routers = True
print(type(routers))
```

Resultado:

```
<class 'list'>
<class 'dict'>
<class 'str'>
<class 'NoneType'>
<class 'bool'>
```

4.2 Usando métodos

En Python, muchos tipos de datos incluyen herramientas integradas llamadas métodos.

Los métodos permiten realizar operaciones directamente sobre strings, listas, diccionarios y otros objetos.

Por ejemplo, podemos:

- Convertir texto a mayúsculas.
- Modificar listas.
- Eliminar espacios.
- Buscar elementos.
- Modificar estructuras de datos.

Cada tipo de dato tiene sus propios métodos, diseñados específicamente para trabajar con ese tipo de información.

En las siguientes secciones veremos cómo funcionan y cómo utilizarlos correctamente.

4.2.1 ¿Qué es un método?

Los **métodos** en Python son funciones asociadas a un tipo de dato específico. Se utilizan para realizar operaciones directamente sobre el valor almacenado en una variable.

A diferencia de las funciones, los métodos se ejecutan **con punto después de la variable**:

```
nombre_variable.método()
```

En este caso, el método se ejecuta sobre el valor contenido en la variable.

Es importante notar que **cada tipo de dato tiene sus propios métodos**. Por ejemplo, los strings, las listas o los diccionarios tienen métodos diferentes, diseñados para trabajar con ese tipo de información:

```
hostname = 'R1-GYE'

# El método lower() de un string convierte el texto a minúsculas:
print(hostname.lower()) # r1-gye
```

 **Nota importante:** No es necesario memorizar todos los métodos disponibles en Python.

Cada tipo de dato tiene muchos métodos, y es normal consultar la documentación cuando se necesita uno específico.

Lo importante es comprender la idea general:

- Los métodos son funciones asociadas a un tipo de dato.
- Permiten realizar operaciones directamente sobre los datos.
- Se utilizan mediante la siguiente sintaxis: `nombre_variable.método()`

A lo largo del libro iremos utilizando distintos métodos según el tipo de dato con el que estemos trabajando.

4.3 Métodos de strings

Las cadenas de texto o strings tienen una gran cantidad de métodos disponibles. Algunos de los más utilizados son los siguientes:

4.3.1 Los métodos `upper()`, `lower()` y `capitalize()`

Los métodos `upper()`, `lower()` y `capitalize()` nos permiten convertir a mayúsculas, minúsculas y formato tipo oración, respectivamente.

Son muy útiles al momento de validar datos y manipular texto:

```
hostname, alarm = 'rl-gye', 'ALARMA!'

# Convertir a mayúsculas:
print(f'Con el método .upper() convertimos "{hostname}" '
      f'a mayúsculas: "{hostname.upper()}"')

# Convertir a minúsculas
print(f'Con el método .lower() convertimos "{alarm}" '
      f'a minúsculas: "{alarm.lower()}"')

# Convertir a formato oración
print(f'Con el método .capitalize() convertimos "{alarm}" '
      f'a formato tipo oración: "{alarm.capitalize()}"')
```

Resultado:

```
Con el método .upper() convertimos "rl-gye" a mayúsculas: "R1-GYE".
Con el método .lower() convertimos "ALARMA!" a minúsculas: "alarma!".
Con el método .capitalize() convertimos "ALARMA!" a formato tipo
oración: "Alarma!".
```

4.3.2 Los métodos `find()` y `rfind()`

En algunos casos es necesario encontrar la posición de un carácter dentro de un string.

Para esto existen los métodos `find()` y `rfind()`:

```
text = '192.168.10.25'

print(text.find('.')) # primera aparición del carácter punto (.)
print(text.rfind('.')) # última aparición del carácter punto (.)
```

Resultado:

```
3
10
```

- `find()` devuelve la posición de la primera coincidencia.
- `rfind()` devuelve la posición de la última coincidencia.
- Si no encuentra el valor, devuelve `-1`.

Estos métodos son útiles cuando necesitas trabajar con slicing basado en posiciones.

4.3.3 Los métodos `split()` y `splitlines()`

El método `split()` se utiliza para **dividir una cadena de texto en varias partes** usando un separador.

La forma general de `split()` es:

```
texto.split(separador)
```

Python toma el string original, busca el separador indicado y devuelve una lista con las partes encontradas:

```
text = 'R1-GYE|Cisco|Up'
result = text.split('|')

print(f'El método .split(\'|\') nos permite convertir "{text}" '
      f'en una lista: {result}')
```

Resultado:

```
El método .split('|') nos permite convertir "R1-GYE|Cisco|Up" en una
lista: ['R1-GYE', 'Cisco', 'Up']
```

En este caso, la barra vertical (|) actúa como separador, por lo que Python divide el texto cada vez que la encuentra.

Este método es muy útil en automatización de redes, porque permite procesar fácilmente datos que vienen en formatos como:

- Líneas de archivos CSV.
- Salidas de comandos.
- Textos separados por comas, espacios u otros caracteres.

Por ejemplo, en un archivo CSV, el texto es separado por comas:

```
'hostname,ip,vendor,status'
```

En estos casos podemos usar `split(',')` para separar cada campo y trabajar con ellos individualmente.

Si `split()` no recibe argumentos, utiliza espacios en blanco como separador por defecto. Esto incluye espacios normales, tabs y nuevas líneas:

```
text = 'R1-GYE          Cisco\nUp'
result = text.split()

print(f'Por defecto .split() separa usando cualquier '
      f'espacio en blanco:\n{result}')
```

Resultado:

```
Por defecto .split() separa usando cualquier espacio en blanco:
['R1-GYE', 'Cisco', 'Up']
```

De manera muy similar a `split()`, el método `splitlines()` nos permite separar el texto, pero el separador es siempre el carácter de nueva línea:

```
text = 'Línea #1.\nLínea #2.\nLínea #3.'
result = text.splitlines()

print(f'El texto que queremos separar es:\n{text}')
print()
print(f'El método .splitlines() genera una lista usando '
      f'nuevas líneas como separador:\n{result}')
```

Resultado:

```
El texto que queremos separar es:
Línea #1.
Línea #2.
Línea #3.

El método .splitlines() genera una lista usando nuevas líneas como
separador:
['Línea #1.', 'Línea #2.', 'Línea #3.']
```

El anterior código también lo pudimos haber escrito sin usar el carácter `\n` específicamente, usando el formato de triple comilla:

```
text = """Línea #1.
Línea #2.
Línea #3."""

result = text.splitlines()

print(f'El texto que queremos separar es:\n{text}')
print()
print(f'El método .splitlines() genera una lista usando '
      f'nuevas líneas como separador:\n{result}')
```

Resultado:

```
El texto que queremos separar es:
Línea #1.
Línea #2.
Línea #3.

El método .splitlines() genera una lista usando nuevas líneas como
separador:
['Línea #1.', 'Línea #2.', 'Línea #3.']
```

4.3.4 El método `join()`

El método `join()` realiza la operación opuesta a `split()`.

Mientras que `split()` divide un string en una lista, `join()` **une los elementos de una colección en un solo string**, utilizando un separador.

La forma general de utilizar `join()` es:

```
separador.join(iterable)
```

En este caso:

- **separador** es el texto que se colocará entre cada elemento.
- **iterable** es la colección cuyos elementos se quieren unir (por ejemplo, una lista).

Ejemplo:

```
interfaces = ['Gi0/0', 'Gi0/1', 'Gi0/2']
result = ','.join(interfaces)
print(f'Con .join() convertimos una lista {interfaces} '
      f'en un string con separador: "{result}"')
```

Resultado:

```
Con .join() convertimos una lista ['Gi0/0', 'Gi0/1', 'Gi0/2'] en un
string con separador: "Gi0/0,Gi0/1,Gi0/2"
```

En este ejemplo, la coma (,) se utiliza como separador, por lo que Python une todos los elementos de la lista insertando una coma entre ellos.

También podemos utilizar otros separadores:

```
interfaces = ['Gi0/0', 'Gi0/1', 'Gi0/2']
print(' | '.join(interfaces))
```

Resultado:

```
Gi0/0 | Gi0/1 | Gi0/2
```

Este método es muy útil cuando necesitamos generar texto estructurado a partir de datos almacenados en listas.

Por ejemplo, para construir una línea de salida en formato CSV:

```
router_data = ['R1-GYE', '10.10.10.1', 'Cisco', 'UP']
csv_line = ','.join(router_data)
print(csv_line)
```

Resultado:

```
R1-GYE,10.10.10.1,Cisco,UP
```

Esto es común cuando generamos archivos de inventario, reportes o resultados que luego serán procesados por otras herramientas.

⚠ **Nota importante:** El método `join()` solo puede unir **elementos que sean strings**.

Si la lista contiene números u otros tipos de datos, primero deben convertirse a texto utilizando `str()`.

4.3.5 El método `strip()`

El método `strip()` se utiliza para eliminar espacios en blanco al inicio y al final de una cadena de texto.

Esto es muy útil cuando procesamos datos provenientes de archivos, APIs o salidas de comandos, donde pueden existir espacios adicionales que no forman parte real del contenido:

```
text = '    R1-GYE    '
result = text.strip()

print(f'Antes: "{text}"')
print(f'Después de usar .strip(): "{result}"')
```

Resultado:

```
Antes: "    R1-GYE    "
Después de usar strip(): "R1-GYE"
```

El método elimina espacios, tabs (`\t`) y saltos de línea (`\n`) que se encuentren al inicio o al final del texto.

También existen dos variantes:

- `lstrip()` elimina espacios solo al inicio.
- `rstrip()` elimina espacios solo al final.

Ejemplo:

```
text = '    SW1-GYE    ' # Texto con espacios a la izquierda y derecha

print(f'"{text.lstrip()}"') # Elimina espacios a la izquierda
print(f'"{text.rstrip()}"') # Elimina espacios a la derecha
```

Resultado:

```
"SW1-GYE    "
"    SW1-GYE"
```

En automatización de redes, `strip()` es muy útil al limpiar datos obtenidos desde archivos o salidas de comandos antes de procesarlos.

4.3.6 El método `replace()`

El método `replace()` permite **reemplazar una parte del texto por otra** dentro de una cadena de caracteres.

La forma general es:

```
texto.replace(valor_antiguo, valor_nuevo)
```

Ejemplo:

```
hostname = 'R1-GYE'
result = hostname.replace('GYE', 'UIO')
print(result)
```

Resultado:

```
R1-UIO
```

El método reemplaza todas las apariciones del texto indicado.

Otro ejemplo:

```
interfaces = 'Gi0/1,Gi0/2,Te0/1'
result = interfaces.replace('Gi', 'GigabitEthernet')
print(result)
```

Resultado:

```
GigabitEthernet0/1,GigabitEthernet0/2,Te0/1
```

Este método es muy útil cuando necesitamos normalizar texto o transformar nombres de interfaces, dispositivos o identificadores.

4.3.7 Los métodos `startswith()` y `endswith()`

El método `startswith()` permite verificar si una cadena **comienza con un determinado texto**.

Este método devuelve un valor booleano:

- **True** si el texto comienza con el valor indicado.
- **False** en caso contrario.

Ejemplo:

```
hostname = 'R1-GYE'  
print(hostname.startswith('R'))
```

Resultado:

```
True
```

Otro ejemplo, invirtiendo la lógica de la comparación usando `not`:

```
hostname = 'SW1-GYE'  
  
if not hostname.startswith('R'):  
    print('El dispositivo no es un router.')
```

Resultado:

```
El dispositivo no es un router.
```

Este método es muy útil en automatización de redes cuando queremos **clasificar dispositivos según su nombre**, por ejemplo, routers, switches o firewalls.

También existe el método complementario `endswith()`, que verifica si una cadena termina con un determinado texto.

Ejemplo:

```
filename = 'config.txt'  
print(filename.endswith('.txt'))
```

Resultado:

```
True
```

Como el archivo sí termina con `.txt`, se imprime `True` en la pantalla.

4.3.8 El método `isdigit()`

El método `isdigit()` permite verificar si un string contiene únicamente dígitos numéricos.

Devuelve `True` si todos los caracteres son números y `False` en caso contrario.

Ejemplo:

```
value = '123'  
print(value.isdigit())
```

Resultado:

```
True
```

Ejemplo con un valor no numérico:

```
value = 'error'  
print(value.isdigit())
```

Resultado:

```
False
```

Este método no valida números negativos ni números decimales:

```
print('-1'.isdigit())  
print('10.5'.isdigit())
```

Resultado:

```
False  
False
```

4.3.9 El método `count()`

Se utiliza para **contar cuántas veces aparece una cadena** dentro de otra cadena de texto.

La forma general es:

```
texto.count(valor)
```

Python busca el texto indicado dentro de la cadena original y devuelve un **número entero** con la cantidad de coincidencias encontradas:

```
original_text = 'Gi0/0 Gi0/1 Gi0/2 Gi0/3'  
searched_text = 'G'  
result = original_text.count(searched_text)  
print(result)
```

Resultado:

```
4
```

En este ejemplo, Python cuenta cuántas veces aparece la letra 'G' dentro del texto.

También podemos contar palabras completas o cualquier otro patrón de texto:

```
text = 'UP DOWN UP UP DOWN'  
print(text.count('DOWN'))
```

Resultado:

```
2
```

En automatización de redes, `count()` puede ser útil cuando necesitamos analizar salidas de comandos y contar cuántas veces aparece un determinado estado, interfaz o palabra clave.

Por ejemplo, para contar cuántas interfaces aparecen en una salida de configuración o cuántas veces aparece un determinado estado dentro de un resultado de diagnóstico.

 **Nota importante:** El método `count()` no busca coincidencias parciales complejas ni expresiones regulares. Simplemente cuenta apariciones exactas del texto indicado dentro de la cadena.

Para búsquedas más avanzadas utilizaremos expresiones regulares más adelante en el libro.

4.4 Métodos de listas

Las listas en Python incluyen varios métodos que permiten agregar, eliminar, ordenar o manipular elementos dentro de la colección.

Estos métodos son muy utilizados en automatización de redes cuando trabajamos con:

- Listas de dispositivos.
- Listas de interfaces.
- Direcciones IP.
- Resultados de comandos.

A continuación, veremos algunos de los métodos más utilizados.

4.4.1 El método `append()`

El método `append()` permite **agregar un elemento al final de la lista**.

La forma general es:

```
lista.append(elemento)
```

Ejemplo:

```
routers = ['R1-GYE', 'R1-UIO']  
routers.append('R2-GYE')  
  
print(routers)
```

Resultado:

```
['R1-GYE', 'R1-UIO', 'R2-GYE']
```

Este método es muy común cuando vamos construyendo una lista dinámicamente, por ejemplo, al leer dispositivos desde un archivo o al procesar resultados de una consulta.

4.4.2 El método `extend()`

El método `extend()` permite **agregar todos los elementos de otra colección al final de la lista**.

La forma general es:

```
lista.extend(otra_coleccion)
```

Ejemplo:

```
routers = ['R1-GYE', 'R1-UIO']  
new_routers = ['R2-GYE', 'R2-UIO']  
routers.extend(new_routers)  
  
print(routers)
```

Resultado:

```
['R1-GYE', 'R1-UIO', 'R2-GYE', 'R2-UIO']
```

A diferencia de `append()`, que agrega un solo elemento, `extend()` agrega **cada elemento individualmente**.

Para que veas la diferencia con `append()`:

```

routers = ['R1-GYE', 'R1-UIO']
new_routers = ['R2-GYE', 'R2-UIO']
routers.append(new_routers)

print(routers)
```

Resultado:

```
['R1-GYE', 'R1-UIO', ['R2-GYE', 'R2-UIO']]
```

El método `append()` insertó un solo elemento (una lista) al final de la lista original. Ahora la lista `routers` tiene 3 elementos: dos strings y una lista.

4.4.3 El método `remove()`

El método `remove()` permite **eliminar un elemento específico de la lista**:

```
lista.remove(valor)
```

Ejemplo:

```

routers = ['R1-GYE', 'R1-UIO', 'R2-GYE']
routers.remove('R1-UIO')

print(routers)
```

Resultado:

```
['R1-GYE', 'R2-GYE']
```

Si el valor indicado no existe en la lista, Python generará una excepción de tipo `ValueError`:

```

routers = ['R1-GYE', 'R1-UIO', 'R2-GYE']
routers.remove('R2-UIO')
```

Resultado:

```
Traceback (most recent call last):
  File "test.py", line 4, in <module>
    routers.remove('R2-UIO')
ValueError: list.remove(x): x not in list
```

Por esta razón, en algunos casos es conveniente verificar primero si el elemento existe.

```
routers = ['R1-GYE', 'R1-UIO', 'R2-GYE']

if 'R2-UIO' in routers:
    routers.remove('R2-UIO')

print(routers)
```

Resultado:

```
['R1-GYE', 'R1-UIO', 'R2-GYE']
```

4.4.4 El método pop()

El método `pop()` permite **eliminar un elemento de la lista y devolver su valor**.

Por defecto elimina el último elemento.

```
lista.pop()
```

Ejemplo:

```
routers = ['R1-GYE', 'R1-UIO', 'R2-GYE']
last_router = routers.pop()

print(f'Elemento eliminado: "{last_router}"')
print(f'Contenido de la lista: {routers}')
```

Resultado:

```
Elemento eliminado: "R2-GYE"
Contenido de la lista: ['R1-GYE', 'R1-UIO']
```

También es posible indicar el índice del elemento que se desea eliminar:

```
routers = ['R1-GYE', 'R1-UIO', 'R2-GYE']
removed_router = routers.pop(1)
print(routers)
```

Resultado:

```
['R1-GYE', 'R2-GYE']
```

4.4.5 El método `sort()`

El método `sort()` permite **ordenar los elementos de una lista**.

La forma general es:

```
lista.sort()
```

Ejemplo:

```
vlans = [30, 10, 20]
vlans.sort()
print(vlans)
```

Resultado:

```
[10, 20, 30]
```

Por defecto, el orden es ascendente.

También podemos ordenar en orden descendente usando el parámetro **`reverse`**:

```
vlans = [30, 10, 20]
vlans.sort(reverse=True)
print(vlans)
```

Resultado:

```
[30, 20, 10]
```

⚠ Nota importante: Las listas también pueden ordenarse utilizando la función `sorted()` que vimos anteriormente.

La diferencia principal es:

```
sorted(lista)    # Crea y retorna una nueva lista ordenada.
lista.sort()     # Modifica la lista original. Retorna None.
```

Ambas formas son válidas y se utilizan con frecuencia en scripts de automatización:

```
vlans = [10, 20, 30]
print(sorted(vlans))    # No afecta la lista original vlans
print(vlans.sort())    # Ordena y modifica la lista vlans
print(vlans)            # La lista queda ordenada
```

Resultado:

```
[10, 20, 30]  
None  
[10, 20, 30]
```

Observa que `sort()` modifica la lista original y retorna `None`, por lo que normalmente no se usa dentro de `print()` salvo con fines didácticos.

4.5 Métodos de sets

Los sets también incluyen métodos que permiten agregar, eliminar o combinar elementos dentro del conjunto.

Recordemos que un **set es una colección desordenada de elementos únicos**, por lo que no permite duplicados.

A continuación, veremos algunos de los métodos más utilizados.

4.5.1 El método `add()`

El método `add()` permite **agregar un elemento al set**.

La forma general es:

```
variable_tipo_set.add(elemento)
```

Ejemplo:

```
vlans = {10, 20, 30}
vlans.add(40)
print(vlans)
```

Resultado:

```
{40, 10, 20, 30}
```

Si el elemento ya existe en el set, **no se agrega nuevamente**, ya que los sets no permiten duplicados.

```
vlans = {10, 20, 30}
vlans.add(20)
print(vlans)
```

Resultado:

```
{10, 20, 30}
```

Si agregas un elemento que no es **hashable**, se genera una excepción de tipo **TypeError**:

```
vlans = {10, 20, 30}
vlans.add({'new_vlan': 40})
print(vlans)
```

Resultado:

```
Traceback (most recent call last):
  File "test.py", line 4, in <module>
    vlans.add({'new_vlan': 40})
TypeError: unhashable type: 'dict'
```

4.5.2 El método `remove()`

El método `remove()` permite **eliminar un elemento específico del set**.

La forma general es:

```
variable_tipo_set.remove(elemento)
```

Ejemplo:

```
vlans = {10, 20, 30}
vlans.remove(20)
print(vlans)
```

Resultado:

```
{10, 30}
```

Si el elemento no existe, Python generará una excepción de tipo `KeyError`:

```
vlans = {10, 20, 30}
vlans.remove(50)
```

Resultado:

```
Traceback (most recent call last):
  File "test.py", line 4, in <module>
    vlans.remove(50)
KeyError: 50
```

4.5.3 El método `discard()`

El método `discard()` también permite eliminar un elemento del set, pero **no genera un error si el elemento no existe**:

```
vlans = {10, 20, 30}
vlans.discard(50)
print(vlans)
```

Resultado:

```
{10, 20, 30}
```

Por esta razón, `discard()` suele ser preferido cuando no estamos seguros de si el elemento existe.

4.5.4 El método `update()`

El método `update()` permite **agregar múltiples elementos al set**.

La forma general es:

```
variable_tipo_set.update(colección)
```

Ejemplo:

```
vlan = {10, 20}
new_vlan = {30, 40}

vlan.update(new_vlan)
print(vlan)
```

Resultado:

```
{40, 10, 20, 30}
```

Si alguno de los elementos ya existe, simplemente se ignora.

4.6 Métodos de diccionarios

Los diccionarios incluyen varios métodos que facilitan trabajar con claves y valores. A continuación veremos algunos de los más utilizados.

4.6.1 El método `get()`

El método `get()` permite obtener el valor asociado a una clave de forma segura:

```
diccionario.get('clave', valor_por_defecto)
```

El argumento `valor_por_defecto` es opcional y se utiliza cuando la clave no existe.

Ejemplo:

```
router = {
    'hostname': 'R1-GYE',
    'ip': '10.1.1.1',
    'vendor': 'Cisco'
}
print(router.get('hostname'))
```

Resultado:

```
R1-GYE
```

Diferencia con acceso directo

Si accedemos a una clave usando corchetes y esta no existe:

```
print(router['sys_model'])
```

Se produce un error tipo `KeyError`:

```
Traceback (most recent call last):
  File "test.py", line 8, in <module>
    print(router['sys_model'])
    ~~~~~^~~~~~
KeyError: 'sys_model'
```

Esto ocurre porque Python espera que la clave exista.

Uso de `get()` cuando la clave no existe

```
print(router.get('sys_model'))
```

Resultado:

```
None
```

En este caso, `get()` no genera una excepción, sino que devuelve `None`.

También podemos definir un valor alternativo:

```
print(router.get('sys_model', 'desconocido'))
```

Resultado:

```
desconocido
```

La idea clave es que el método `get()` es especialmente útil cuando no tenemos certeza de que una clave estará presente en el diccionario, evitando errores y permitiendo definir comportamientos controlados.

4.6.2 El método `keys()`

El método `keys()` devuelve todas las claves del diccionario.

La forma general es:

```
diccionario.keys()
```

Ejemplo:

```
router = {
    'hostname': 'R1-GYE',
    'ip': '10.1.1.1',
    'vendor': 'Cisco'
}
keys = router.keys()
print(keys)
```

Resultado

```
dict_keys(['hostname', 'ip', 'vendor'])
```

Como puedes observar, `router.keys()` no devuelve una lista, sino un objeto tipo `dict_keys`. Como no es una lista, si intentas acceder a sus índices, o intentas utilizar otros métodos de listas, se va a generar una excepción de tipo `TypeError`:

```
print(keys[0])
```

Resultado:

```
Traceback (most recent call last):
  File "test.py", line 10, in <module>
    print(keys[0])
    ~~~~^^^
TypeError: 'dict_keys' object is not subscriptable
```

Si necesitas las claves en una lista, puedes utilizar casting para convertir de tipo `dict_keys` a lista:

```
keys = list(router.keys())
print(keys)
print(keys[0])
```

Resultado:

```
['hostname', 'ip', 'vendor']
hostname
```

Un uso frecuente de este método es cuando queremos **recorrer todas las claves**. Como `dict_keys` es un iterable, lo podemos usar en un lazo `for`:

```
for key in router.keys():
    print(key)
```

Resultado:

```
hostname
ip
vendor
```

4.6.3 El método `values()`

El método `values()` devuelve todos los valores almacenados en el diccionario.

La forma general es:

```
diccionario.values()
```

Su uso es muy parecido al de `keys()`:

```
router = {
    'hostname': 'R1-GYE',
    'ip': '10.1.1.1',
    'vendor': 'Cisco'
}
print(router.values())
print('-----')
for key in router.keys():
    print(key)
```

Resultado:

```
dict_values(['R1-GYE', '10.1.1.1', 'Cisco'])
-----
hostname
ip
```

```
vendor
```

De manera similar a `keys()`, el método `values()` no devuelve una lista, sino un objeto `dict_values()`, que también es un iterable y se puede usar en un lazo `for`.

4.6.4 El método `items()`

El método `items()` devuelve todos los pares clave-valor del diccionario.

La forma general es:

```
diccionario.items()
```

Ejemplo:

```
router = {
    'hostname': 'R1-GYE',
    'ip': '10.1.1.1',
    'vendor': 'Cisco'
}

print(router.items())
```

Resultado:

```
dict_items([('hostname', 'R1-GYE'), ('ip', '10.1.1.1'), ('vendor', 'Cisco')])
```

El método `.items()` devuelve un objeto tipo `dict_items()`.

Al igual que los dos métodos anteriores, es muy común iterar el resultado de `items()`, con un lazo `for`.

Como este método devuelve pares clave-valor, necesitamos aplicar desempaqueamiento para poder asignar una variable a la clave y otra al valor:

```
for key, value in router.items():
    print(f'La clave "{key}" tiene el valor: "{value}"')
```

Resultado:

```
La clave "hostname" tiene el valor: "R1-GYE"
La clave "ip" tiene el valor: "10.1.1.1"
La clave "vendor" tiene el valor: "Cisco"
```

4.6.5 El método `update()`

El método `update()` permite agregar o modificar varios valores en el diccionario.

La forma general es:

```
diccionario.update(nuevo_diccionario)
```

Ejemplo:

```
router = {  
    'hostname': 'R1-GYE'  
}  
  
router.update({'vendor': 'Cisco', 'model': 'ASR1001'})  
print(router)
```

Resultado:

```
{'hostname': 'R1-GYE', 'vendor': 'Cisco', 'model': 'ASR1001'}
```

Si una clave ya existe, su valor se actualiza.

4.6.6 El método `pop()`

El método `pop()` elimina una clave del diccionario y devuelve su valor.

La forma general es:

```
diccionario.pop(clave)
```

Ejemplo:

```
router = {  
    'hostname': 'R1-GYE',  
    'ip': '10.1.1.1',  
    'vendor': 'Cisco'  
}  
  
vendor = router.pop('vendor')  
print(vendor)  
print(router)
```

Resultado:

```
Cisco  
{'hostname': 'R1-GYE', 'ip': '10.1.1.1'}
```

4.7 Encadenando métodos

En Python es posible llamar múltiples métodos consecutivamente sobre un mismo objeto. A esto normalmente se lo conoce como encadenamiento de métodos.

Por ejemplo:

```
hostname = '  r1-gye  '.strip().upper()
print(hostname)
```

Resultado:

```
R1-GYE
```

En este ejemplo:

- `strip()` elimina los espacios al inicio y al final.
- `upper()` convierte el texto a mayúsculas.

Aunque podría escribirse así:

```
hostname = '  r1-gye  '
hostname = hostname.strip()
hostname = hostname.upper()

print(hostname)
```

En muchos casos es más común escribirlo utilizando encadenamiento de métodos:

Esto es posible porque muchos métodos en Python devuelven un nuevo objeto, permitiendo seguir llamando más métodos sobre el resultado anterior.

El encadenamiento de métodos es extremadamente común en Python moderno y permite escribir código más compacto y expresivo.

Otro ejemplo aplicado a redes:

```
interface = '  gi0/1  '
normalized_if = interface.strip().upper().replace('GI',
                                                    'GigabitEthernet')

print(normalized_if)
```

Resultado:

```
GigabitEthernet0/1
```

En este caso, estamos eliminando espacios, convirtiendo a mayúsculas y reemplazando `GI` por `GigabitEthernet`, todo en una línea de código.

4.8 Trabajar con direcciones IP usando el módulo `ipaddress`

Python incluye un módulo muy útil para trabajar con direcciones y redes IP llamado `ipaddress`.

Este módulo forma parte de la **librería estándar de Python**, por lo que **no es necesario instalarlo**. Simplemente se importa y puede utilizarse inmediatamente en cualquier script.

El módulo `ipaddress` permite representar **direcciones IP y redes como objetos de Python**, lo que facilita realizar muchas operaciones comunes en redes de forma segura y sencilla, sin tener que manipular cadenas de texto manualmente.

Entre las operaciones más frecuentes que permite realizar se encuentran:

- Validar direcciones IP.
- Trabajar con redes y subredes.
- Comprobar si una IP pertenece a una red.
- Obtener información de una red (máscara, broadcast, hosts disponibles, etc.).
- Iterar sobre las direcciones de una red.

Esto resulta especialmente útil en **automatización de redes**, donde es común procesar direcciones IP provenientes de:

- Archivos de inventario.
- Configuraciones de dispositivos.
- Bases de datos.
- Resultados de comandos ejecutados por SSH o APIs.

Al utilizar `ipaddress`, evitamos muchos errores comunes al trabajar con direcciones IP como texto.

Para utilizar el módulo, primero debemos importarlo, junto a las funciones que vamos a utilizar:

```
from ipaddress import ip_address, ip_network
```

4.8.1 Crear una dirección IP

Para crear una IP, utilizamos `ip_address()`:

```
ip = ip_address('192.168.10.5')
print(ip)
```

Resultado:

```
192.168.10.5
```

La función `ip_address()` recibe una dirección IP en formato texto y devuelve un **objeto de tipo IP** que Python puede manipular.

Si la dirección no es válida, Python generará un error automáticamente, lo que convierte a esta función en una forma sencilla de **validar direcciones IP**.

4.8.2 Crear una red

También podemos representar una red utilizando `ip_network()`:

```
network = ip_network('192.168.10.0/29')
print(network)
```

Resultado:

```
192.168.10.0/29
```

Este objeto representa toda la red y permite acceder a diferentes propiedades de la misma, como su máscara, dirección de broadcast o los hosts disponibles utilizando los métodos de este objeto:

```
# Obtener la máscara:
print(f'La máscara es {network.netmask}')

# Obtener la dirección de broadcast:
print(f'\nLa dirección de broadcast es {network.broadcast_address}')

# Obtener los hosts disponibles:
print(f'\nLos hosts disponibles de la red son:')
for host in network.hosts():
    print(f' -{host}')
```

Resultado:

```
La máscara es 255.255.255.248

La dirección de broadcast es 192.168.10.7

Los hosts disponibles de la red son:
-192.168.10.1
-192.168.10.2
-192.168.10.3
-192.168.10.4
-192.168.10.5
-192.168.10.6
```

Además de `network_address` y `broadcast_address`, los objetos de red también tienen el atributo `prefixlen`, que devuelve la longitud del prefijo CIDR:

```
network = ip_network('192.168.10.0/30')
print(network.prefixlen)
```

Resultado:

```
30
```


4.8.3 Comprobar si una IP pertenece a una red

Una operación muy común en redes es verificar si una dirección IP pertenece a determinada subred.

Esto puede hacerse fácilmente usando el operador `in`:

```
ip = ip_address('192.168.10.1')
network = ip_network('192.168.10.0/29')

print(ip in network) # True
print(ip_address('10.0.0.1') in network) # False
```

En el primer caso, la IP `192.168.10.1` sí pertenece a la red `192.168.10.0/29`, mientras que, en el segundo caso, `10.0.0.1` **no pertenece**.

Este tipo de verificación es muy útil en scripts que analizan configuraciones de routers, validan inventarios de red o filtran direcciones IP dentro de determinados rangos.

4.9 Ejercicio práctico: Analizando un inventario básico de dispositivos

Este ejemplo reúne varios de los conceptos vistos en este capítulo: variables, strings, listas, diccionarios, sets, casting y operadores de comparación. Simula una situación común en automatización de redes: procesar un inventario simple de dispositivos y extraer información útil.

Tal como aprendimos en la sección de Pensar como programador, vamos a analizar este problema con el siguiente flujo:

- Objetivo
- Lo que tenemos
- Paso a paso
- Código final

4.9.1 Objetivos

Vamos a construir un script que lea información de equipos con sus respectivas IPs, fabricantes y estado.

El script debe generar lo siguiente:

- Lista de fabricantes de todos los equipos.
- Lista de equipos alarmados

4.9.2 Lo que tenemos

En un escenario de la vida real, la lista de equipos con sus IPs, fabricantes y estado la obtendrías de un archivo de texto, a través de SNMP o de un API. En esta etapa de aprendizaje vamos a simular esta información con una lista de dispositivos en formato **string**. El formato de cada elemento de esta lista es el siguiente:

```
hostname,ip_address,vendor,status
```

En este inventario, **status = 1** significa **operativo** y **status = 0** significa **alarmado**. La coma se utiliza para separar los campos.

Es decir, la lista de equipos quedaría de la siguiente forma:

```
inventory = [
    'R1-GYE,10.10.10.1,Cisco,1',
    'R1-UIO,10.10.20.1,Juniper,0',
    'R1-CUE,10.10.30.1,Cisco,1',
    'R2-GYE,10.10.40.1,Huawei,1',
    'R2-UIO,10.10.50.1,Huawei,1'
]
```

Toma en cuenta que **inventory** es una lista de strings, pero puede representar cualquier archivo de tipo CSV porque cada elemento representa una línea con este formato.

Esta lista es la única información que tenemos y necesitamos para empezar a construir nuestro script.

4.9.3 Paso a paso

Para desarrollar el código necesario para este ejercicio, vamos a necesitar los siguientes pasos:

- **Paso 1: Extraer la información del inventario.**
Procesar cada línea del inventario en formato CSV, separando los campos y convirtiendo los datos en una lista de diccionarios para facilitar su manipulación.
- **Paso 2: Listar los fabricantes.**
Recorrer la lista de dispositivos para obtener los fabricantes únicos, eliminando duplicados mediante un set.
- **Paso 3: Listar los dispositivos alarmados.**
Filtrar los dispositivos cuyo estado indique una alarma (`status = 0`), generando una lista con los equipos que requieren atención.
- **Paso 4: Generar los resultados.**
Imprimir en pantalla la información obtenida de forma clara y organizada para el usuario.

Paso 1: Extraer la información del inventario

El inventario está constituido por varias líneas con información de equipos en cada línea. Cada línea tiene información del hostname, la dirección ip, el fabricante y el estado.

La estructura del mismo inventario nos da una buena idea de la estructura que podemos utilizar. Como tenemos una lista de líneas, vamos a crear una lista de diccionarios, donde cada diccionario guarda la información correspondiente.

```
devices = []
```

Luego tenemos que leer línea por línea del inventario. Para ello, utilizamos un lazo `for`, el cual verás con detalle en el capítulo 4. Por el momento solo necesitas saber que `for` nos permite acceder a todas las líneas de `inventory`:

```
for line in vlans_inventory:
```

Para extraer la información de cada línea, usamos el método `split()`, el cual nos permitirá separar la cadena de caracteres en elementos de una lista y asignar directamente a nuestras variables. Este proceso es similar al que utiliza Excel al “separar en columnas”:

```
hostname, ip, vendor, status = line.split(',')
```

Con estas variables ya podemos construir un diccionario por cada dispositivo:

```
device = {
    'hostname': hostname,
    'ip': ip,
    'vendor': vendor,
    'status': int(status)
}
```

Toma en cuenta que tenemos que convertir el estado a un entero, ya que por defecto `split()` convierte todo en `string`.

A continuación, agregamos este diccionario a nuestra lista de equipos:

```
devices.append(device)
```

Finalmente tenemos este código para el Paso 1:

```
# -----
# Extraer la información del inventario
# -----
# Creamos la lista de equipos
devices = []
# Leemos el inventario, línea por línea:
for line in inventory:
    # split() convierte un string en una lista.
    hostname, ip, vendor, status = line.split(',')
    device = {
        'hostname': hostname,
        'ip': ip,
        'vendor': vendor,
        'status': int(status)    # casting: convertir texto a número
    }
    devices.append(device)
```

⚙ Paso 2: Listar los fabricantes

Ahora que tenemos toda la información en una lista de diccionarios, es muy sencillo listar todos los fabricantes. Tenemos que recorrer la lista nuevamente con `for` y accedemos al elemento con clave `vendor` en cada uno de los diccionarios:

```
vendors = set()
for device in devices:
    vendors.add(device['vendor'])
```

El Código para el Paso 2 queda de la siguiente manera:

```
# -----
# Listar los fabricantes
# -----
vendors = set()
for device in devices:
    vendors.add(device['vendor'])
```

⚙ Paso 3: Listar los dispositivos alarmados

De manera similar que en el paso 2, podemos recorrer la lista de equipos y leer el valor de `status` para obtener la lista de dispositivos alarmados. Toma en cuenta que no podemos usar un `set` porque un set no puede contener diccionarios:

```
# -----
# Listar los dispositivos alarmados
# -----
# Recuerda que status = 0 se usa para equipos alarmados.
devices_down = []
for device in devices:
    if device['status'] == 0:
        devices_down.append(device)
```

⚙ Paso 4: Generar los resultados

El último paso es simplemente generar los resultados de manera legible en pantalla para el usuario. Recuerda que el carácter `\n` introduce una línea nueva en la salida que se muestra como resultado:

```
# -----
# Generar resultados
# -----
print(f'Marcas de equipos en la red: {sorted(vendors)}')
print('\nDispositivos alarmados: ')
for d in devices_down:
    print(f"    {d['hostname']} ({d['ip']}) ! ")
```

4.9.4 Código final y resultados

Ya solo necesitamos crear un archivo con todo el código que hemos desarrollado. El siguiente archivo créalo en `python_networking/part_1/code`.

 **chapter_4_example.py**

```
# -----
# Declaración del inventario
# -----
# Formato: hostname,ip,fabricante,estado
inventory = [
    'R1-GYE,10.10.10.1,Cisco,1',
    'R1-UIO,10.10.20.1,Juniper,0',
    'R1-CUE,10.10.30.1,Cisco,1',
    'R2-GYE,10.10.40.1,Huawei,1',
    'R2-UIO,10.10.50.1,Huawei,1'
]

# -----
# Extraer la información del inventario
# -----
# Creamos la lista de equipos
devices = []
# Leemos el inventario, línea por línea:
for line in inventory:
    # split() convierte un string en una lista.
    hostname, ip, vendor, status = line.split(',')
    device = {
        'hostname': hostname,
        'ip': ip,
        'vendor': vendor,
```

```

        'status': int(status)    # casting: convertir texto a número
    }
    devices.append(device)

# -----
# Listar los fabricantes
# -----
vendors = set()
for device in devices:
    vendors.add(device['vendor'])

# -----
# Listar los dispositivos alarmados
# -----
# Recuerda que status = 0 se usa para equipos alarmados.
devices_down = []
for device in devices:
    if device['status'] == 0:
        devices_down.append(device)

# -----
# Generar resultados
# -----
print(f'Marcas de equipos en la red: {sorted(vendors)}')
print('\nDispositivos alarmados: ')
for d in devices_down:
    print(f"    ! {d['hostname']} ({d['ip']})")

```

Al ejecutar el código, vas a poder apreciar en pantalla lo siguiente:

```

Marcas de equipos en la red: ['Cisco', 'Huawei', 'Juniper']

Dispositivos alarmados:
    ! R1-UIO (10.10.20.1)

```

4.10 Resumen

En este capítulo aprendiste a trabajar activamente con los datos en Python utilizando funciones integradas y métodos específicos de cada tipo.

Exploraste cómo manipular cadenas de texto, listas, conjuntos y diccionarios, aplicando operaciones comunes que permiten transformar y analizar información de forma eficiente. También viste cómo utilizar herramientas del lenguaje para simplificar tareas repetitivas y hacer tu código más claro y expresivo.

Además, diste un primer paso hacia la aplicación en redes al trabajar con el módulo `ipaddress`, lo que te permite manejar direcciones IP de forma estructurada y confiable.

Finalmente, aplicaste todos estos conceptos en un ejercicio práctico, consolidando la transición desde la teoría hacia el uso real de Python para resolver problemas.

Con este capítulo, ya no solo entiendes los datos, sino que sabes cómo utilizarlos. Esto te prepara directamente para el siguiente paso: controlar el flujo del programa y tomar decisiones con estructuras de control.

4.11 Fin de la muestra gratuita

¡Gracias por llegar hasta aquí!

Esta muestra extendida incluye los primeros 4 capítulos completos del libro:

Python para Redes: Programación y automatización con SNMP, SSH, APIs y más desde cero.

A partir del Capítulo 5, el contenido continúa avanzando hacia temas como:

- Estructuras de control
- Funciones
- Manejo de errores
- Expresiones regulares (**regex**)
- Clases y objetos
- Archivos
- SSH y SFTP
- SNMP
- APIs
- Concurrencia
- Proyectos reales completos
- Y mucho más

Si esta muestra te resultó útil o interesante, y deseas acceder al libro completo, puedes hacerlo desde Hotmart:

 **Hotmart:**

<https://hotmart.com/es/club/daniel-francisco-llivipuma-pozo/products>

 **Repositorio oficial:**

https://github.com/aniluca-publishing/python_para_redes

 **LinkedIn:**

<https://www.linkedin.com/in/dllivipuma>

¡Gracias por formar parte de este proyecto!

Capítulo 5 - Estructuras de control

Hasta este punto hemos trabajado principalmente con datos y operaciones sobre ellos. Ahora comenzaremos a controlar cómo se comporta el programa utilizando condiciones, repeticiones y distintas estructuras de flujo.

Las estructuras de control permiten que un script tome decisiones, procese grandes cantidades de información automáticamente y reaccione de manera diferente según cada situación.

Este tipo de lógica aparece constantemente en automatización de redes: validar configuraciones, recorrer inventarios, detectar errores, filtrar información y ejecutar acciones únicamente cuando se cumplen determinadas condiciones.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Utilizar condicionales **if**, **elif** y **else** para tomar decisiones.
- Combinar condiciones usando operadores lógicos como **and**, **or** y **not**.
- Comprender el concepto de valores **truthy** y **falsey** en Python.
- Utilizar lazos **for** y **while** para repetir tareas automáticamente.
- Generar secuencias numéricas utilizando **range()**.
- Aplicar **list**, **set** y **dict comprehensions** para transformar datos.
- Controlar el flujo de un programa usando **break**, **continue** y **pass**.
- Comprender cómo funciona el bloque **for/else** en Python.
- Procesar inventarios y validar configuraciones mediante estructuras de control.
- Detectar inconsistencias y automatizar validaciones básicas de red.

5.1 Condicionales: `if`, `else`, `elif`

5.2 Operadores lógicos: and, or, not

5.3 Lazos: `for` y `while`

5.4 Comprehensions (comprensiones)

5.5 Controlando el flujo del programa

5.6 Ejercicio práctico: Validando VLANs y detectando inconsistencias

5.7 Resumen

En este capítulo aprendiste a controlar el flujo de ejecución en Python mediante estructuras de control, herramientas esenciales para cualquier tarea de automatización en redes. Vimos cómo usar condicionales (`if`, `elif`, `else`) para tomar decisiones basadas en el estado de los equipos, y cómo utilizar lazos (`for` y `while`) para repetir acciones de manera eficiente.

También aprendiste a modificar el comportamiento de un bucle mediante las palabras clave `break`, `continue` y `pass`, así como el uso del bloque `for/else`, una característica muy útil para validar condiciones cuando un bucle finaliza sin interrupciones.

Finalmente, aplicaste todos estos conceptos en un ejemplo práctico donde procesaste un inventario, construiste estructuras de datos, obtuviste VLANs únicas, validaste VLANs críticas y encontraste inconsistencias utilizando sets.

Estas herramientas te permitirán crear automatizaciones más robustas, detectar errores de forma eficiente y preparar el camino hacia scripts más avanzados orientados a dispositivos reales.

Capítulo 6 - Funciones

A medida que un programa crece, organizar todo el código en un solo bloque se vuelve difícil de mantener y reutilizar. Las funciones permiten dividir la lógica en bloques más claros, reutilizables y fáciles de administrar.

En automatización de redes, prácticamente cualquier tarea repetitiva termina implementándose mediante funciones: validar información, conectarse a equipos, procesar inventarios, consultar APIs o generar respaldos.

Por esta razón, las funciones representan uno de los pilares más importantes para construir scripts y automatizaciones reales.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Definir y utilizar funciones para organizar y reutilizar código.
- Pasar argumentos y configurar valores por defecto en funciones.
- Comprender cómo funciona la palabra clave **return**.
- Retornar múltiples valores utilizando tuplas y desempaqueamiento.
- Utilizar ***args** y ****kwargs** para crear funciones flexibles.
- Comprender el alcance de variables locales y globales.
- Crear y utilizar funciones anidadas.
- Documentar funciones utilizando docstrings.
- Comprender el propósito del bloque **name == 'main'**.
- Reestructurar scripts utilizando funciones para construir automatizaciones más claras y mantenibles.

6.1 ¿Qué es una función?

6.2 ¿Cómo definir una función?

6.3 Pasando argumentos a las funciones

6.4 Valores de retorno

6.5 Argumentos variables: `*args` y `**kwargs`

6.6 Funciones recursivas

6.7 Docstrings: documentando tus funciones

6.8 Alcance de variables (*scope*)

6.9 El bloque `__name__ == '__main__'`

6.10 Ejercicio práctico: Analizando un inventario básico de dispositivos, utilizando funciones

6.11 Ejercicio práctico: Validando VLANs y detectando inconsistencias, utilizando funciones

6.12 Resumen

En este capítulo aprendiste a definir funciones, pasar argumentos, configurar valores por defecto y retornar valores para reutilizarlos en otras partes del programa.

También viste cómo devolver múltiples valores, utilizar argumentos variables con `*args` y `**kwargs`, documentar funciones con docstrings y comprender el alcance de las variables dentro y fuera de una función.

Además, aprendiste el propósito del bloque `__name__ == '__main__'`, una estructura muy común en scripts reales de Python.

Finalmente, aplicaste estos conceptos reorganizando ejercicios de inventario y validación de VLANs mediante funciones, logrando código más claro, modular y fácil de mantener.

Capítulo 7 - Herramientas esenciales en Python

Hasta este punto ya contamos con las bases fundamentales del lenguaje. Ahora comenzaremos a trabajar con herramientas y módulos que aparecen constantemente en scripts reales de automatización.

Muchas de estas utilidades forman parte de la librería estándar de Python y permiten resolver tareas comunes como procesar archivos, validar rutas, ejecutar comandos del sistema, medir tiempos, comparar configuraciones o trabajar con expresiones regulares.

Este capítulo funciona como una caja de herramientas práctica que utilizaremos repetidamente a lo largo del resto del libro.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Instalar y gestionar librerías utilizando **pip**.
- Comprender qué son los entornos virtuales y para qué sirven.
- Utilizar funciones útiles como **filter()**, **enumerate()**, **zip()**, **any()** y **all()**.
- Crear funciones simples utilizando **lambda**.
- Introducirte al uso de expresiones regulares (**regex**).
- Leer y escribir archivos de texto y CSV.
- Trabajar con rutas, directorios y validación de archivos utilizando el módulo **os**.
- Comparar configuraciones y detectar cambios utilizando **difflib**.
- Medir tiempos de ejecución y generar timestamps con **time** y **datetime**.
- Ejecutar comandos del sistema y realizar validaciones básicas de red utilizando **subprocess** y **socket**.

7.1 Gestionando paquetes con `pip`

7.2 Procesamiento de datos con Python

7.3 Introducción a las expresiones regulares (`regex`)

7.4 Interactuar con archivos y el sistema operativo

7.5 Manejo básico de errores (`try/except`)

7.6 Trabajar con archivos CSV (datos estructurados)

7.7 Manejo de tiempo y fechas

7.8 Ejecutar comandos y comunicarse por la red

7.9 Ejercicio práctico: Analizador simple de interfaces

7.10 Resumen

En este capítulo incorporaste una serie de herramientas esenciales que te ayudarán a escribir scripts más limpios, seguros y profesionales, especialmente antes de pasar a ejemplos de automatización más avanzados.

Aprendiste a usar funciones incorporadas de Python que facilitan tareas comunes en redes, como ordenar datos (`sorted()`), numerar elementos (`enumerate()`), unir listas paralelas (`zip()`), validar condiciones de forma compacta (`any()` y `all()`), y obtener estadísticas rápidas con funciones como `max()`, `sum()` y `round()`.

También viste por qué es importante manejar errores con `try/except` y cómo leer o escribir archivos de forma sencilla usando `open()`.

A esto sumamos una pequeña introducción a expresiones regulares (`regex`), una herramienta extremadamente poderosa para validar y extraer información de textos como configuraciones, líneas de interfaces o salidas de comandos de red.

Finalmente, pusiste en práctica todos estos conceptos construyendo un analizador simple de interfaces, capaz de:

- Validar nombres con `regex`
- Manejar errores reales en el inventario
- Ordenar interfaces por tráfico
- Detectar interfaces en estado DOWN
- Calcular estadísticas
- Mostrar reportes claros

Este capítulo cierra la parte fundamental del lenguaje y te prepara para empezar a trabajar con módulos más avanzados, expresiones regulares completas, análisis de configuraciones y proyectos reales de automatización de redes.

A partir de aquí, tus scripts dejarán de ser ejercicios básicos y comenzarán a parecerse a herramientas que realmente usarías en un entorno de operación y mantenimiento de red.

Capítulo 8 - Manejo de Errores

En automatización de redes, los errores son inevitables: archivos inexistentes, datos inválidos, conversiones fallidas, equipos que no responden o servicios inaccesibles.

Por esta razón, un script no debería terminar abruptamente ante el primer problema encontrado. En muchos casos, lo correcto es detectar el error, registrarlo y continuar procesando el resto de la información.

Python utiliza **excepciones** para representar este tipo de situaciones y proporcionar mecanismos que permitan controlarlas de forma más segura y estructurada.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Comprender qué son las excepciones y por qué ocurren en Python.
- Utilizar bloques **try** y **except** para manejar errores de forma controlada.
- Capturar excepciones específicas como **ValueError**, **KeyError** y **FileNotFoundError**.
- Utilizar **else** y **finally** dentro del manejo de excepciones.
- Interpretar **tracebacks** y mensajes de error comunes.
- Generar excepciones manualmente utilizando **raise**.
- Crear excepciones personalizadas para validar escenarios específicos.
- Diseñar scripts más robustos y tolerantes a fallos.
- Manejar errores comunes en inventarios, archivos y procesamiento de datos.
- Comprender la importancia del manejo de errores en automatización y redes.

8.1 ¿Por qué es importante manejar los errores?

8.2 Cómo interpretar los errores en la consola (tracebacks)

8.3 Debugging de tracebacks complejos

8.4 Estructura de manejo de errores en Python

8.5 La instrucción `raise`

8.6 Creación de excepciones personalizadas

8.7 Ejercicio práctico: Procesador robusto de inventarios

8.8 Resumen

En este capítulo aprendiste a manejar errores usando:

- `try` y `except`.
- `else` y `finally`.
- `raise` para validar datos.
- Excepciones personalizadas.
- Patrones de manejo robusto en inventarios reales.

Manejar errores no significa esconderlos, sino **entenderlos, controlarlos y decidir cómo reaccionar ante ellos**.

Cuando estés aprendiendo, deja que el error ocurra y léelo.

Solo después decide si debes capturarlo con `try/except`.

Un buen manejo de errores transforma un script básico en una herramienta confiable. A partir de aquí, podrás crear automatizaciones más seguras, especialmente cuando trabajes con conexiones SSH, archivos grandes, APIs o dispositivos que a veces responden mal o simplemente no responden.

Capítulo 9 - Clases y Objetos en Python

La programación orientada a objetos suele parecer compleja e intimidante al inicio, principalmente por la cantidad de nuevos términos que introduce: clases, objetos, atributos, métodos, herencia, entre otros.

Sin embargo, en realidad ya hemos trabajado con objetos desde los primeros capítulos. Listas, diccionarios y strings son objetos creados a partir de clases incorporadas de Python, y métodos como `.append()`, `.split()`, `.upper()` o `.keys()` forman parte de ellos.

La diferencia ahora es que comenzaremos a crear nuestros propios tipos de objetos para organizar mejor la información y construir programas más estructurados y reutilizables.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Comprender qué son las clases y los objetos en Python.
- Diferenciar atributos, métodos y estados internos de un objeto.
- Comprender el propósito de `self` y del constructor `init`.
- Crear clases propias para modelar dispositivos y estructuras de red.
- Utilizar herencia para reutilizar y extender comportamiento.
- Comprender cómo funciona la sobreescritura de métodos.
- Utilizar `super()` para reutilizar lógica de clases padre.
- Validar tipos de objetos utilizando `isinstance()`.
- Organizar inventarios y dispositivos usando programación orientada a objetos.
- Construir código más ordenado, reutilizable y fácil de extender.

9.1 Programación orientada a objetos (POO)

9.2 Clases y objetos

9.3 Herencia

9.4 La función `isinstance()`

9.5 ¿Qué es importante entender en este capítulo?

9.6 Ejercicio práctico: Aplicando POO a redes

9.7 Resumen

La programación orientada a objetos (POO) en Python organiza el código mediante clases (plantillas para objetos) y objetos (instancias de clases), permitiendo modelar entidades con atributos y métodos.

- **Clases y Objetos:** Una clase define atributos y métodos, mientras que un objeto es una instancia con sus propios valores.
- **Atributos y Métodos:** Los atributos son datos y los métodos son funciones dentro de la clase que describen el comportamiento de los objetos.
- **El constructor `__init__`:** Método especial que se ejecuta al crear un objeto, inicializando sus atributos.
- **Herencia:** Permite a una clase hija extender o modificar el comportamiento de una clase padre.
- **`super()`:** Permite reutilizar lógica de la clase padre desde una clase hija.
- **`isinstance()`:** Permite verificar si un objeto pertenece a una clase determinada.

Ventajas de usar clases:

Las clases permiten organizar el código, reutilizar lógica mediante herencia y mantener programas más claros y escalables. Además, facilitan modelar elementos del mundo real y proteger datos internos mediante encapsulación.

Capítulo 10 - Leer y escribir archivos

Los archivos son una parte fundamental de prácticamente cualquier proceso de automatización. Permiten almacenar información, generar evidencia y conservar resultados que posteriormente pueden ser analizados, auditados o reutilizados.

En redes, es común trabajar constantemente con:

- Inventarios.
- Respaldos de configuraciones.
- Logs.
- Reportes.
- Resultados de monitoreo.

Un respaldo que no se guarda en un archivo no existe. Un error que no queda registrado difícilmente podrá analizarse después.

En capítulos anteriores ya utilizamos `open()` de forma básica. Ahora formalizaremos ese conocimiento y comenzaremos a trabajar con patrones reales utilizados en automatización y operaciones de red.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Comprender cómo funciona el manejo de archivos en Python.
- Abrir archivos utilizando distintos modos como `'r'`, `'w'`, `'a'`, `'rb'` y `'wb'`.
- Leer archivos línea por línea, completos o como listas.
- Escribir y agregar información en archivos de forma segura.
- Comprender la diferencia entre rutas relativas y absolutas.
- Trabajar correctamente con `encoding='utf-8'`.
- Manipular archivos binarios como imágenes o respaldos.
- Manejar errores comunes relacionados con archivos y permisos.
- Aplicar buenas prácticas para automatización y generación de logs.
- Construir un simulador básico de respaldos SSH utilizando archivos, logs y timestamps.

10.1 Introducción al manejo de archivos

10.2 Rutas de archivos: relativas y absolutas

10.3 Leer archivos

10.4 Escribir en archivos

10.5 Leer y escribir archivos binarios

10.6 Manejo de errores al trabajar con archivos

10.7 Buenas prácticas al trabajar con archivos

10.8 Ejercicio práctico: Simulador de respaldo SSH

10.9 Resumen

En este capítulo aprendiste a trabajar con archivos de forma segura, una habilidad esencial en la automatización de redes. Viste cómo:

- Abrir archivos en distintos modos usando `open()`.
- Leer contenido línea por línea, en bloque o como lista.
- Escribir o agregar información usando `'w'` y `'a'`.
- Manipular archivos binarios (imágenes, respaldos, configuraciones).
- Manejar errores comunes con `try/except`.
- Aplicar buenas prácticas reales que evitarán pérdida de datos.

Completaste también tu primer registrador de eventos (`logger`), una herramienta imprescindible para cualquier script serio que procese inventarios, conexiones SSH, SNMP o APIs.

A partir de este punto, ya cuentas con todas las herramientas necesarias para persistir información, generar evidencia y construir automatizaciones confiables.

En la PARTE II estas mismas técnicas se combinarán con conexiones reales a dispositivos de red.

Resumen de la PARTE I

Con los capítulos de esta parte ya cuentas con una base sólida de Python orientada a automatización y redes:

- Variables y tipos de datos.
- Strings, listas, diccionarios y métodos.
- Condicionales y lazos.
- Funciones.
- Manejo de errores.
- Expresiones regulares (**regex**).
- Clases y objetos.
- Lectura y escritura de archivos.

A partir del siguiente bloque del libro (PARTE II), aprenderemos a utilizar Python para tareas reales de redes.

Antes de continuar

Antes de avanzar a la automatización real, es importante que puedas manejar con comodidad algunos conceptos básicos de Python.

No necesitas dominarlos al 100%, pero sí entenderlos lo suficiente como para leer, entender y modificar código.

A continuación, se proponen ejercicios prácticos basados en los conceptos más utilizados en el resto del libro.

Si puedes resolver estos ejercicios sin dificultad, estás listo para continuar.

Si alguno te resulta complicado, no te preocupes. Revisa nuevamente el capítulo correspondiente y vuelve a intentarlo. Estos conceptos aparecerán constantemente en los siguientes capítulos.

1. Ejercicios de Strings y métodos básicos

2. Ejercicios de Listas y comprensión de listas

3. Ejercicios de Diccionarios y comprensión de diccionarios

4. Ejercicios de Iteración

5. Ejercicios de Condicionales

6. Ejercicios de Funciones

7. Ejercicios de Slicing

8. Ejercicios de Manejo básico de errores

PARTE II:

Aplicando Python a problemas reales de redes

Antes de continuar

A partir de esta **PARTE II**, el enfoque del libro cambia.

En los capítulos anteriores construiste las bases necesarias del lenguaje: tipos de datos, estructuras de control, funciones, manejo de errores y organización del código. A partir de aquí, estas herramientas **ya no se explican desde cero**, sino que se utilizan de forma directa para resolver problemas reales de redes.

Si en algún momento sientes que un ejemplo avanza demasiado rápido, no significa que el tema sea demasiado complejo ni que estés fallando. Significa simplemente que alguna base necesita reforzarse.

En ese caso, te recomiendo volver a los capítulos correspondientes de la **PARTE I** y revisarlos con calma. Este libro está diseñado para leerse de forma flexible: avanzar, retroceder y volver a avanzar cuando sea necesario.

La **PARTE II** no vuelve a enseñar desde cero la sintaxis básica de Python, sino que te enseña **cómo aplicar lo que ya sabes** en escenarios reales de automatización de redes.

Si ya te sientes cómodo con listas, diccionarios, bucles, funciones, archivos y manejo básico de errores, estás listo para continuar.

Capítulo 11 - Analizando configuraciones de red con expresiones regulares (**regex**)

Las configuraciones de red están llenas de patrones: interfaces con nombres similares, direcciones IP, máscaras, descripciones, rutas, VLANs, vecinos OSPF, entradas ARP y más.

A simple vista, muchas veces los patrones no son evidentes, especialmente cuando trabajamos con configuraciones extensas o grandes volúmenes de texto. Sin embargo, si observas con detalle, comenzarás a identificar estructuras repetitivas.

Las expresiones regulares, conocidas como **regex**, son una herramienta diseñada precisamente para eso: detectar, extraer y manipular patrones de texto de manera eficiente.

En redes esto es especialmente útil porque:

- Muchas veces solo necesitas extraer información puntual sin procesar todo el archivo.
- Python permite transformar resultados en listas, diccionarios u objetos que luego puedes guardar y procesar.
- La automatización moderna usa análisis de texto en casi todos los flujos: validación, auditorías, respaldos, sistemas de monitoreo, plantillas, control de calidad y más.

Si en capítulos anteriores viste pequeñas **regex** como parte de funciones, aquí entenderás los fundamentos necesarios para utilizarlas en tus scripts.

Este capítulo abre la **PARTE II** porque es el puente entre Python básico y *Python aplicado a redes*.

Regex en Python

En Python, las expresiones regulares se trabajan principalmente mediante el módulo **re**, que forma parte de la biblioteca estándar del lenguaje. Este módulo proporciona las herramientas necesarias para buscar, extraer, validar y modificar patrones de texto utilizando expresiones regulares.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Comprender cómo funcionan las expresiones regulares en Python.
- Identificar patrones reales dentro de configuraciones de red.
- Utilizar los metacaracteres más importantes en automatización.
- Trabajar con funciones como `match()`, `search()`, `findall()`, y `sub()`.
- Analizar y extraer información de:
 - Direcciones IP.
 - Interfaces.
 - VLANs.
 - Bloques de configuración.

11.1 Comprendiendo expresiones regulares (regex)

11.2 Metacaracteres esenciales en `regex`

11.3 La clase Match

11.4 Herramientas esenciales de `regex` en Python

11.5 Uso de flags

11.6 Grupos sin captura (?:)

11.7 Expresiones regulares comunes en redes

11.8 Aplicando `regex` a redes

11.9 Ejercicio práctico: Extraer VLANs de un comando "`show vlan`"

11.10 Ejercicio práctico: Extraer estado de interfaces de un `show ip interface brief`

11.11 Nota avanzada: compilar patrones con `re.compile` (eficiencia y diseño avanzado)

11.12 Resumen

En este capítulo aprendiste los fundamentos indispensables para analizar texto técnico usando expresiones regulares. Ahora sabes:

- Cómo funcionan los patrones básicos: `\d`, `\s`, `\S`, `+`, `[]`, `()`, `|`, etc.
- Cómo usar los métodos esenciales de Python: `search()`, `match()`, `findall()`, `finditer()` y `sub()`.
- Cómo pensar una regex paso a paso: reconocer patrón, descomponer y traducir.
- Cómo capturar grupos, trabajar con `group()` y `groups()`.
- Cómo usar flags como `MULTILINE` e `IGNORECASE`.
- Cómo aplicar regex a escenarios reales de redes:
 - Extraer direcciones IP.
 - Identificar VLANs.
 - Procesar interfaces.
 - Procesar bloques de configuración.
 - Interpretar salidas de `show ip interface brief`.
- Cómo usar flags adicionales como `DOTALL` y `VERBOSE` cuando el texto incluye saltos de línea o patrones complejos.

A partir de aquí, tendrás la capacidad de transformar cualquier archivo de configuración o salida de comandos en datos estructurados que Python puede procesar, comparar, almacenar o graficar.

Regex es la llave que abre la automatización en redes, y la usarás en varios capítulos posteriores del libro.

Capítulo 12 - Pruebas de conectividad: ping y traceroute

Verificar conectividad es una de las tareas más comunes en operación y diagnóstico de redes. Herramientas como ping y traceroute permiten validar disponibilidad, tiempos de respuesta y rutas hacia un destino.

En este capítulo trabajaremos con **subprocess**, un módulo de la librería estándar de Python que permite ejecutar comandos directamente desde el sistema operativo y procesar sus resultados desde nuestros scripts.

El objetivo será construir funciones reutilizables que nos permitan integrar pruebas de conectividad dentro de automatizaciones y herramientas operativas más completas.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Ejecutar pruebas de conectividad desde Python utilizando comandos del sistema operativo.
- Detectar automáticamente si el programa se está ejecutando en Windows, Linux o macOS.
- Ejecutar comandos **ping** y **traceroute** usando `subprocess.run()`.
- Procesar y analizar la salida de comandos del sistema operativo.
- Utilizar expresiones regulares para extraer información relevante de **ping** y **traceroute**.
- Crear módulos reutilizables para proyectos reales de automatización de redes.

12.1 Ping con Python

12.2 Traceroute con Python

12.3 Resumen

En este capítulo construiste un módulo completo y reutilizable para validar conectividad de red desde Python. Aprendiste a ejecutar ping y traceroute utilizando funcionalidades estándar como **subprocess** y aplicaste expresiones regulares para interpretar correctamente la salida de cada comando en distintos sistemas operativos.

En la sección de ping construiste herramientas capaces de:

- Detectar el sistema operativo.
- Ejecutar pings con parámetros adecuados para Windows, Linux y macOS.
- Extraer la pérdida de paquetes y el RTT promedio usando regex.
- Unificar toda esa información en un diccionario estándar, listo para usar.

En traceroute analizaste las diferencias reales entre plataformas, aprendiste a interpretar líneas con uno o varios hosts por salto, manejaste timeouts y clasificaste los resultados en pérdidas, hosts involucrados y RTT. Finalmente construiste una función robusta que opera igual en Windows y Linux, y que entrega un formato homogéneo para cualquier proyecto.

Con este capítulo completas un primer paquete de conectividad en Python, totalmente modular y apto para integrarlo en scripts operativos, dashboards, APIs internas o rutinas de diagnóstico automático. Más adelante comenzarás a explorar auditorías de seguridad básicas, escaneo de puertos y descubrimiento de servicios, construyendo sobre la base sólida que ya estableciste aquí.

Capítulo 13 - Auditorías de seguridad: escaneo de puertos

El escaneo de puertos es una de las técnicas más antiguas y útiles en auditorías de red. Saber qué puertos están abiertos en un dispositivo permite inferir qué servicios podrían estar en ejecución, validar configuraciones de firewall, detectar exposiciones innecesarias y preparar diagnósticos más profundos.

Aunque existen herramientas especializadas mucho más avanzadas, como **nmap**, construir nuestro propio escáner tiene una gran ventaja pedagógica y práctica: comprender cómo funciona realmente una conexión TCP y disponer de un módulo ligero que podamos integrar, modificar o complementar fácilmente dentro de proyectos de automatización, monitoreo o validación operativa.

En este capítulo construiremos nuestro propio escáner de puertos TCP utilizando únicamente la librería estándar de Python.

Python no incluye una función nativa para *probar puertos*, pero sí nos ofrece las primitivas necesarias para hacerlo de forma limpia y controlada a través del módulo **socket**.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Comprender cómo funciona un escaneo básico de puertos TCP.
- Utilizar **socket** y **connect_ex()** para probar conectividad.
- Diferenciar puertos abiertos, cerrados y filtrados.
- Manejar errores comunes de red y timeouts.
- Medir tiempos de conexión.
- Construir funciones reutilizables para auditorías y validaciones de red.
- Generar resultados estructurados usando listas y diccionarios.

13.1 Escáner de puertos con Python

13.2 Resumen

En este capítulo construiste una herramienta fundamental para cualquier auditoría de red: un escáner de puertos implementado únicamente con librerías estándar de Python y totalmente reutilizable en tus futuros proyectos.

Primero aprendiste cómo realizar un escaneo de puertos rápido y liviano utilizando `socket` y `connect_ex()`. Esta técnica te permite identificar si un puerto está:

- Abierto
- Cerrado
- Filtrado por un firewall

Además, añadiste métricas de tiempo que ayudan a interpretar la respuesta de la red y facilitan diagnósticos operativos.

Este escáner, aunque simple, es suficiente para validaciones rápidas, auditorías de inventario, verificación de servicios e incluso detección ligera de problemas de seguridad.

Más adelante darás un paso más profundo: aprenderás a conectarte a dispositivos de red usando SSH y SFTP, ejecutar comandos, transferir archivos y automatizar flujos operativos completos.

Capítulo 14 - SSH y SFTP: conectando a dispositivos de red

SSH es uno de los protocolos más utilizados en administración y automatización de redes. Servidores, routers, switches, firewalls y muchos otros dispositivos permiten ser gestionados remotamente mediante este protocolo.

Además de la ejecución remota de comandos, en entornos operativos también es muy común transferir archivos entre equipos para respaldos, recopilación de logs, exportación de configuraciones y procesamiento de información.

En este capítulo trabajaremos con SSH para ejecutar comandos remotamente y con SFTP para explorar directorios, listar archivos y realizar transferencias seguras desde Python.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Comprender cómo funcionan SSH y SFTP en automatización de redes.
- Conectarte por SSH a dispositivos y servidores usando Python.
- Ejecutar comandos remotos y procesar `stdout`, `stderr` y `exit_code`.
- Manejar errores comunes de conexión, autenticación y DNS.
- Implementar reintentos automáticos ante fallos transitorios.
- Listar y descargar archivos remotamente mediante SFTP.
- Explorar directorios y subdirectorios de forma recursiva.
- Filtrar archivos usando sufijos o coincidencias parciales.
- Construir funciones reutilizables para automatización operativa.

14.1 SSH con Python

14.2 SFTP con Python

14.3 Resumen

En este capítulo aprendiste a conectarte por SSH y SFTP desde Python usando Paramiko, una librería ampliamente utilizada en automatización de redes.

Con SSH construiste una función reutilizable para conectarte a dispositivos remotos, ejecutar comandos, capturar stdout, stderr y exit_code, medir el tiempo de ejecución, manejar errores comunes y aplicar reintentos ante fallos transitorios.

También viste una limitación importante: no todos los equipos de red se comportan igual cuando ejecutas comandos por SSH. Algunos dispositivos pueden devolver banners, prompts, ecos del comando o códigos de salida poco confiables. Por eso, el resultado debe interpretarse con criterio operativo y no solamente como texto plano.

Luego trabajaste con SFTP para listar, filtrar y descargar archivos desde servidores o dispositivos remotos. Construiste una función flexible capaz de recorrer directorios, explorar subdirectorios, aplicar filtros por sufijo o contenido del nombre y descargar archivos cuando sea necesario.

Con estas funciones tienes una base sólida para automatizar tareas reales como respaldos de configuración, recolección de logs, validaciones operativas, auditorías básicas y generación de reportes.

Capítulo 15 - Obtención y modificación de datos con SNMP

El protocolo SNMP ha sido, durante décadas, una de las bases del monitoreo de redes. Desde routers y switches hasta servidores, equipos de radio, firewalls y equipos de acceso, muchos dispositivos pueden exponer información operativa mediante este protocolo.

Con SNMP puedes consultar el estado de interfaces, versiones de software, uso de CPU, temperatura, tiempo de operación (uptime), tráfico y miles de parámetros más, sin necesidad de iniciar sesión en el dispositivo.

Algo importante a considerar es que SNMP existe en tres versiones principales:

- **v1:** Obsoleto en la práctica, pero aún presente en equipos antiguos
- **v2c:** Rápido, eficiente, el más utilizado para monitoreo actual
- **v3:** Autenticación y cifrado, usado cuando la seguridad es crítica

El módulo que desarrollaremos va a soportar **todas las versiones**. Las llamadas a las funciones serán prácticamente iguales; solo cambiarás la versión y, en caso de SNMPv3, un par de parámetros adicionales.

En este capítulo usaremos nuevamente `subprocess.run()` para ejecutar comandos del sistema. Puedes repasar lo aprendido en [Ejecutar comandos del sistema](#).

Si estás utilizando Windows, debes instalar el paquete Net-SNMP. Puedes descargarlo desde el sitio oficial:

<https://net-snmp.sourceforge.io/download.html>

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Comprender cómo funciona SNMP en monitoreo y automatización de redes.
- Diferenciar las versiones SNMPv1, SNMPv2c y SNMPv3.
- Comprender qué son los OIDs y cómo se representan los datos en SNMP.
- Ejecutar consultas SNMP GET desde Python.
- Recorrer tablas completas utilizando SNMP WALK y BULKWALK.
- Modificar parámetros remotos utilizando SNMP SET.
- Procesar y validar la salida de comandos SNMP.
- Manejar errores comunes de autenticación, conectividad y timeouts.
- Construir funciones reutilizables para monitoreo e inventarios.
- Automatizar consultas SNMP sobre distintos dispositivos de red.

15.1 OIDs numéricos y OIDs con nombre

15.2 Concepto de índices en SNMP

15.3 Relación de índices entre distintos OIDs

15.4 Formato de comandos SNMP

15.5 SNMP GET con Python

15.6 SNMP WALK / BULKWALK con Python

15.7 SNMP SET con Python

15.8 Decisiones de diseño en este capítulo

15.9 Resumen

En este capítulo aprendiste a utilizar SNMP desde Python para automatizar consultas y modificaciones en equipos de red. Con SNMP GET obtuviste valores puntuales, como el nombre del dispositivo o el estado de una interfaz. Con SNMP WALK y BULKWALK recorriste tablas completas para extraer listas de interfaces, contadores o cualquier conjunto de datos disponible. Finalmente, con SNMP SET aprendiste a modificar parámetros permitidos, como descripciones o configuraciones básicas.

Todas las funciones desarrolladas soportan SNMP v1, v2c y v3, manejan timeouts y errores, y devuelven respuestas limpias en formato diccionario para facilitar el procesamiento.

A partir de aquí ya puedes integrar estas funciones dentro de scripts más grandes, construir colectores, validar equipos automáticamente o crear herramientas de diagnóstico. En la PARTE III verás cómo combinarlas en proyectos más completos que calculan tasas de tráfico, verifican disponibilidad o generan reportes para cientos de dispositivos.

Con estas bases, Python se convierte en una herramienta práctica y poderosa para administrar redes utilizando SNMP.

Capítulo 16 - Introducción práctica a APIs

En redes modernas, gran parte de la automatización ocurre mediante APIs. Equipos, plataformas de monitoreo, servicios en la nube y muchas otras herramientas exponen interfaces que permiten consultar información, enviar datos y automatizar procesos de forma programática.

Las APIs permiten integrar scripts con servicios externos sin depender únicamente de la CLI, convirtiéndose en uno de los pilares más importantes de la automatización moderna.

En este capítulo trabajaremos principalmente con APIs REST y datos en formato JSON, construyendo funciones reutilizables orientadas a escenarios reales de redes.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Comprender qué es una API y cómo se utiliza en automatización.
- Trabajar con APIs REST y datos en formato JSON.
- Enviar parámetros y procesar respuestas desde Python.
- Consumir APIs públicas útiles para redes.
- Comprender qué son NETCONF y RESTCONF.
- Construir funciones reutilizables para interactuar con APIs.
- Integrar servicios externos dentro de automatizaciones reales.
- Construir un analizador básico de IPs públicas utilizando APIs.

16.1 ¿Qué es una API?

16.2 ¿Qué es JSON y por qué ya sabes usarlo?

16.3 Consumir una API desde Python (lo más básico)

16.4 APIs útiles para administración de redes

16.5 NETCONF, RESTCONF y las APIs que configuran dispositivos de red

16.6 Ejercicio práctico: Analizador de URLs e IPs públicas

16.7 Ejercicio práctico con token: Consulta básica a AbuseIPDB

16.8 Resumen

En este capítulo aprendiste qué es realmente una API y por qué se ha convertido en uno de los pilares fundamentales de la automatización moderna. Las APIs permiten que los programas hablen entre sí de una forma estructurada, segura y predecible. Prácticamente todo lo que usamos en redes —sistemas de monitoreo, nubes, dashboards, automatización de configuraciones e incluso plataformas internas— expone sus datos mediante APIs.

Comenzaste entendiendo qué es una API REST y cómo funciona el patrón más común: haces una petición HTTP, recibes un JSON y luego Python procesa ese JSON como un diccionario. Con la librería `requests` viste que consumir una API es tan directo como enviar un GET y leer los campos de respuesta.

Luego exploraste APIs públicas muy útiles para redes: obtención de IP pública, información de geolocalización, resolución DNS desde servicios externos y hasta identificación del fabricante de una MAC. Esto te permitió ver cómo cada servicio devuelve datos distintos y cómo puedes interpretar sus claves fácilmente desde Python.

En la segunda parte del capítulo conociste NETCONF, RESTCONF y las APIs especializadas que exponen los equipos de red modernos. Viste que estas APIs usan modelos YANG y permiten automatización real: modificar configuraciones, validar estados y aplicar cambios de forma estructurada. Aunque estos protocolos no se cubren en detalle aquí, ya entiendes cómo se relacionan con las APIs REST y por qué dominar este capítulo es requisito fundamental para avanzar en automatización.

Finalmente integraste dos ejercicios prácticos: primero, un analizador de URLs e IPs públicas; luego, una consulta autenticada para validar la reputación de una IP usando AbuseIPDB.

El primer script detecta el host, resuelve sus IPs con DNS, consulta una API para obtener país, ciudad, continente, ASN y organización, y muestra los datos en una tabla limpia. El segundo script obtiene una IP pública, consulta una API autenticada mediante token y clasifica el nivel de riesgo según la reputación reportada.

Con este capítulo ya puedes consumir APIs externas, APIs internas y sentaste las bases conceptuales necesarias para trabajar luego con NETCONF, RESTCONF y automatización avanzada de dispositivos. Esta habilidad se convertirá en una de las herramientas más poderosas en tu trabajo diario en redes.

Capítulo 17 - Notificaciones automáticas: Correo (SMTP) y Slack (Webhooks)

Hasta este punto ya sabes recolectar información, ejecutar pruebas de conectividad, leer archivos, trabajar con APIs y estructurar funciones reutilizables.

Sin embargo, en automatización real falta una pieza crítica: notificar cuando algo importante ocurre.

Detectar un problema sin notificarlo reduce drásticamente el valor operativo del script.

En escenarios reales, las notificaciones permiten:

- Alertar cuando un equipo queda sin respuesta.
- Informar cuando un enlace cambia de estado.
- Avisar cuando un valor supera un umbral crítico.
- Generar reportes automáticos.
- Integrar eventos con herramientas externas.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Enviar correos electrónicos utilizando SMTP desde Python.
- Construir mensajes automáticos de alerta y reporte.
- Utilizar webhooks para integrar scripts con Slack.
- Enviar mensajes estructurados hacia canales de monitoreo.
- Manejar errores comunes relacionados con envío de notificaciones.
- Construir funciones reutilizables para alertamiento automático.

17.1 Enviar correos con SMTP

17.2 Enviar mensajes por Slack usando Webhooks

17.3 Resumen

En este capítulo aprendiste a agregar una pieza fundamental a tus automatizaciones: la capacidad de notificar cuando ocurre algo importante.

Primero revisaste cómo funciona el envío de correos mediante SMTP. Aprendiste que un script puede conectarse a un servidor SMTP, autenticarse, construir un mensaje correctamente formateado y enviarlo a uno o varios destinatarios. También viste la diferencia entre TLS explícito usando el puerto 587 y TLS implícito usando el puerto 465, además de las consideraciones actuales de seguridad, como el uso de contraseñas de aplicación en proveedores como Gmail.

Luego construiste la función `send_email_smtp()` dentro de `notification_tools.py`. Esta función permite enviar correos desde Python de forma reutilizable, controlar el servidor SMTP, el puerto, las credenciales, el destinatario, el asunto y el contenido del mensaje. También retorna un diccionario con el estado del envío, lo cual facilita integrarla en scripts más grandes.

Después trabajaste con Slack y webhooks. Aprendiste que un webhook es una URL especial que recibe datos mediante una petición HTTP POST y ejecuta una acción, como publicar un mensaje en un canal. También configuraste una app en Slack, habilitaste Incoming Webhooks y obtuviste una URL asociada a un canal específico.

Finalmente agregaste dos funciones más al módulo `notification_tools.py`: `send_webhook()`, para enviar payloads JSON a cualquier webhook, y `send_slack_notification()`, para enviar mensajes simples a Slack usando el formato esperado por la plataforma. Con el archivo `chapter_17_slack_example.py` probaste una alerta estructurada con severidad, título, detalle y hora del evento.

Con este capítulo ya puedes enviar notificaciones por correo y por Slack desde tus scripts de automatización. Esto permite construir herramientas mucho más útiles en entornos reales de operación: alertas de equipos sin respuesta, avisos de fallas, reportes automáticos, mensajes de monitoreo y notificaciones integradas con otros sistemas.

Capítulo 18 - Usando concurrencia en Python

Hasta este punto ya construimos herramientas capaces de ejecutar ping, SSH y consultas SNMP. El problema aparece cuando la cantidad de dispositivos crece: ejecutar todo de forma secuencial termina siendo demasiado lento para entornos reales.

En automatización de redes, gran parte del tiempo de ejecución se pierde esperando respuestas de la red. La concurrencia permite aprovechar ese tiempo de espera para ejecutar múltiples tareas simultáneamente y acelerar significativamente los procesos operativos.

En este capítulo trabajaremos con `ThreadPoolExecutor` para ejecutar tareas concurrentes de forma práctica y reutilizable, aplicando el mismo patrón a ping, SSH y SNMP.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Comprender qué es concurrencia y por qué es importante en redes.
- Diferenciar concurrencia y paralelismo.
- Comprender qué es un `thread` y cómo funciona en Python.
- Utilizar `ThreadPoolExecutor` para ejecutar tareas concurrentemente.
- Construir funciones `worker` reutilizables.
- Ejecutar ping, SSH y SNMP de forma concurrente.
- Procesar resultados a medida que las tareas terminan.
- Manejar errores sin interrumpir toda la ejecución.
- Comprender cómo escoger un valor adecuado de `max_workers`.
- Construir automatizaciones más rápidas y escalables.

18.1 Qué es concurrencia, y por qué importa en redes

18.2 ¿Qué es un thread?

18.3 Conceptos clave antes de continuar

18.4 Creando un módulo para concurrencia

18.5 Por qué a veces necesitamos una función `worker`

18.6 Ping concurrente

18.7 SSH concurrente

18.8 SNMP concurrente

18.9 Qué tomar en cuenta al escoger `max_workers`

18.10 Resumen

En este capítulo aprendiste a acelerar tus automatizaciones sin cambiar tu lógica de red.

Primero entendiste qué es concurrencia y por qué en redes casi todo es I/O-bound: tu script pasa la mayor parte del tiempo esperando respuestas de ping, SSH, SNMP o APIs, no calculando.

Luego aprendiste el patrón base para concurrencia con `threads`:

- Una lista de tareas
- Una función worker que procesa una tarea
- Un **Executor** (`ThreadPoolExecutor`) que reparte esas tareas entre `threads` y recolecta resultados

Después construiste un módulo reutilizable, `concurrency_tools.py`, cuya única responsabilidad es orquestrar ejecuciones concurrentes y recolectar resultados a medida que las tareas terminan, manejando errores sin tumbar tu script.

Finalmente aplicaste el mismo patrón a casos reales de redes:

- Ping concurrente
- SSH concurrente
- SNMP concurrente

La idea clave es esta:

La concurrencia no cambia tu automatización. Cambia el rendimiento y la escala.

Si antes un script era “útil pero lento”, ahora tienes el patrón para convertirlo en una herramienta operativa, capaz de trabajar con cientos o miles de equipos con tiempos razonables.

PARTE III:

Automatización y proyectos reales de redes con Python

Introducción

En la **PARTE I** construiste una base sólida de Python, enfocada desde el inicio en problemas reales de redes. Aprendiste Python como una herramienta para interactuar con el sistema operativo, la red y los dispositivos.

En la **PARTE II** aplicaste esos fundamentos a tareas concretas: validaciones de conectividad, ejecución de comandos por SSH, consultas SNMP y automatización en paralelo. Aprendiste patrones reutilizables y buenas prácticas para escribir código claro y mantenible.

En esta **PARTE III** damos el siguiente paso.

Aquí ya no trabajamos con ejemplos aislados.

Aquí construimos **proyectos completos**, similares a los que se ejecutan en entornos de producción.

Cada capítulo de esta parte presenta un proyecto real de automatización de redes, con la misma estructura que hemos utilizado a lo largo del libro:

- Objetivo
- Lo que tenemos
- Paso a paso
- Integración final del código

Los proyectos están pensados para ser reutilizados, adaptados y ampliados según tus necesidades. No son scripts desechables, sino bases reales sobre las que puedes construir soluciones más grandes.

El objetivo principal no es enseñarte nuevas librerías por sí mismas, sino ayudarte a pensar como un programador que organiza, documenta y mantiene su código en el tiempo.

Si llegaste hasta aquí, ya tienes las herramientas necesarias.

Ahora toca usarlas como se usan en el mundo real.

En entornos reales, el problema no es que el código funcione una vez.

El problema es que funcione hoy, mañana y dentro de seis meses, cuando ya no recuerdes cada línea que escribiste.

Qué cambia en la PARTE III

Hasta ahora hemos aprendido los fundamentos y las herramientas principales para automatizar tareas de redes:

- Python
- Módulos reutilizables
- Protocolos de red
- APIs
- Concurrencia
- Notificaciones

En la **PARTE III** damos un paso más: dejamos de trabajar solamente con ejemplos individuales y empezamos a construir proyectos completos.

Esto significa que el código ya no se piensa únicamente como una secuencia de instrucciones que resuelve una tarea puntual, sino como una solución que debe poder entenderse, configurarse, ejecutarse, revisarse y mantenerse en el tiempo.

Aquí aprenderás a:

- Separar configuración del código.
- Estructurar proyectos como en entornos reales.
- Definir contratos explícitos en funciones.
- Registrar eventos correctamente con logging.
- Diseñar código reutilizable y mantenible.

No se trata solamente de crear proyectos más grandes.

Se trata de crear proyectos más claros, organizados y mantenibles.

Capítulo 19 - De script funcional a herramienta robusta

A medida que los proyectos de automatización crecen, ya no basta con que el código simplemente funcione. También es necesario que pueda mantenerse, reutilizarse y ejecutarse de forma predecible en el tiempo.

En entornos reales, esto implica manejar credenciales de forma más segura, registrar eventos correctamente, estructurar mejor el código y organizar los proyectos para que puedan crecer sin volverse difíciles de administrar.

En este capítulo comenzaremos a trabajar con conceptos y prácticas utilizadas habitualmente en herramientas profesionales de automatización y operaciones.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Utilizar variables de entorno para manejar credenciales de forma más segura.
- Comprender qué son los **type hints** y cómo utilizarlos.
- Utilizar **Optional**, **Union** y tipado de tuplas en funciones.
- Reemplazar **print()** por **logging** estructurado.
- Generar logs en consola y archivos utilizando **logging**.
- Comprender los niveles de severidad del módulo **logging**.
- Organizar proyectos utilizando módulos y paquetes Python.
- Ejecutar proyectos correctamente utilizando **python3 -m**.
- Construir una librería reutilizable para automatización de redes.
- Comprender prácticas básicas de organización y arquitectura de proyectos.

19.1 Gestión segura de credenciales usando variables de entorno

19.2 Tipado y contratos explícitos en funciones

19.3 Logging estructurado en lugar de `print()`

19.4 Estructura y organización de proyectos

19.5 Resumen

En este capítulo dimos un paso importante: pasamos de escribir código funcional a organizar herramientas más robustas y mantenibles.

Aprendimos a separar credenciales del código utilizando variables de entorno, evitando almacenar usuarios, contraseñas, comunidades SNMP o tokens directamente dentro de los archivos Python.

También incorporamos tipado en las funciones para hacer más explícito qué datos espera y qué devuelve cada componente del proyecto. Esto mejora la legibilidad, facilita el mantenimiento y ayuda a detectar errores con mayor anticipación.

Además, reemplazamos progresivamente el uso de `print()` por logging, permitiendo registrar eventos con severidad, contexto e historial tanto en consola como en archivos.

Finalmente, organizamos el código como una estructura de proyecto más profesional, separando una librería reutilizable `network_lib` de los proyectos específicos de la Parte III.

A partir de este punto, los proyectos del libro dejan de ser archivos aislados y empiezan a comportarse como herramientas estructuradas, reutilizables y preparadas para crecer.

Capítulo 20 - Respaldo automático de configuraciones de red

En entornos de red, mantener respaldos actualizados de las configuraciones es una necesidad operativa fundamental. Un cambio incorrecto, una falla de hardware o una actualización problemática pueden requerir restaurar rápidamente la configuración de un dispositivo.

En este capítulo construiremos una solución completa de respaldos automáticos utilizando Python, integrando conectividad, SSH, concurrencia, logging, manejo de archivos y comparación de configuraciones dentro de una arquitectura modular y reutilizable.

El enfoque ya no será únicamente ejecutar tareas individuales, sino diseñar una herramienta organizada y preparada para escenarios reales de operación de redes.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Construir un sistema automático de respaldo de configuraciones.
- Validar conectividad mediante ping antes de ejecutar SSH.
- Ejecutar respaldos concurrentes utilizando threads.
- Organizar respaldos y logs utilizando estructuras por fecha.
- Trabajar con rutas y archivos utilizando `pathlib`.
- Generar archivos `.cfg` y `.diff` automáticamente.
- Detectar cambios entre configuraciones utilizando `difflib`.
- Utilizar variables de entorno para manejar credenciales SSH.
- Separar lógica reutilizable y lógica específica del proyecto.
- Diseñar herramientas de automatización más robustas y mantenibles.

20.1 Objetivos

20.2 Lo que tenemos

20.3 Paso a paso

20.4 Integración final del código

20.5 Posibles mejoras y consideraciones

20.6 Resumen

En este capítulo se construyó un sistema completo de respaldo automático de configuraciones de red, integrando los conceptos y herramientas desarrollados en capítulos anteriores.

Más allá de ejecutar comandos por SSH, el enfoque estuvo en **diseñar una solución estructurada**, modular y reutilizable, similar a las utilizadas en entornos productivos.

A lo largo del capítulo se abordaron los siguientes aspectos clave:

- Planificación del flujo de respaldo, desde la validación de conectividad hasta la detección de cambios.
- Separación clara de responsabilidades mediante módulos independientes.
- Uso de concurrencia para ejecutar tareas masivas de forma eficiente.
- Organización de respaldos y diffs utilizando una estructura basada en fechas.
- Detección automática de cambios mediante comparaciones entre respaldos consecutivos.
- Aplicación de buenas prácticas de seguridad, evitando credenciales en texto plano.
- Reutilización de una librería genérica de funciones de red (`network_lib`).

El resultado no es un script aislado, sino una base sólida sobre la cual pueden construirse sistemas más complejos, como inventarios, monitoreo o plataformas de gestión.

Este capítulo marca un punto de inflexión en el libro: a partir de aquí, Python deja de ser solo una herramienta para ejecutar tareas puntuales y pasa a convertirse en un lenguaje para diseñar soluciones de automatización reales.

En el siguiente capítulo se ampliará esta arquitectura para abordar otros problemas fundamentales de operación de redes, reutilizando los mismos principios y patrones de diseño.

Capítulo 21 - Inventario automático de dispositivos de red

En entornos de red reales, mantener un inventario actualizado es una tarea fundamental para operación, monitoreo, auditoría y planificación de crecimiento.

Información como marca, modelo, versión de software, interfaces o números de serie suele estar distribuida en cientos o miles de dispositivos, por lo que obtenerla manualmente se vuelve inviable rápidamente.

En este capítulo construiremos una herramienta de inventario automático utilizando SNMP, concurrencia y una arquitectura modular reutilizable, siguiendo el mismo enfoque estructurado utilizado en los proyectos anteriores.

¿Qué aprenderás en este capítulo?

Al finalizar este capítulo podrás:

- Construir un sistema automático de inventario de dispositivos de red.
- Consultar información utilizando SNMP v1/v2c y SNMPv3.
- Obtener datos como hostname, modelo, versión y número de serie.
- Ejecutar consultas SNMP concurrentemente.
- Manejar errores y dispositivos no alcanzables.
- Construir **workers** reutilizables para automatización masiva.
- Organizar proyectos utilizando módulos y librerías reutilizables.
- Generar resultados estructurados a partir de múltiples dispositivos.
- Diseñar herramientas orientadas a escalabilidad y mantenimiento.
- Reutilizar componentes desarrollados en capítulos anteriores.

21.1 Objetivos

21.2 Lo que tenemos

21.3 Paso a paso

21.4 Integración final del código

21.5 Resumen

En este capítulo se construyó un sistema completo de inventario automático de dispositivos de red, integrando SNMP, concurrencia, logging y una estructura modular reutilizable.

Más allá de consultar OIDs por SNMP, el enfoque estuvo en diseñar una herramienta capaz de leer dispositivos desde un archivo CSV, validar cuáles responden, obtener información física del inventario y generar resultados estructurados en archivos reutilizables.

A lo largo del capítulo se abordaron los siguientes aspectos clave:

- Lectura y validación de dispositivos desde un archivo CSV.
- Consulta de información mediante SNMP v1/v2c y SNMPv3.
- Validación previa de equipos sin respuesta SNMP.
- Uso de concurrencia para ejecutar consultas masivas de forma eficiente.
- Obtención de información como fabricante, descripción física, modelo y número de serie.
- Indexación de respuestas SNMP para relacionar datos provenientes de distintos OIDs.
- Generación automática de archivos CSV con el inventario obtenido.
- Registro de eventos, errores y resultados mediante archivos de log.
- Separación de responsabilidades entre el archivo principal y las funciones auxiliares.
- Reutilización de funciones de la librería `network_lib`.

El resultado es una herramienta práctica para construir inventarios automáticos y mantener información técnica más ordenada, trazable y fácil de actualizar.

Este capítulo refuerza una idea central de la **PARTE III**: Python no solo permite ejecutar comandos o consultar datos, sino construir soluciones completas de automatización que combinan validación, procesamiento, concurrencia, logging y generación de resultados.

A partir de esta base, el mismo enfoque puede extenderse hacia sistemas más avanzados de auditoría, monitoreo, documentación automática o integración con plataformas externas.

PARTE IV:

Anexos técnicos

Introducción

Los anexos de esta sección complementan el contenido principal del libro con material de referencia, criterios prácticos y documentación resumida de las herramientas desarrolladas a lo largo de los capítulos anteriores.

El objetivo no es introducir conceptos completamente nuevos, sino proporcionar una guía rápida y reutilizable para facilitar el trabajo con automatización, desarrollo y operación de redes utilizando Python.

Anexo A - Documentación Técnica de las Funciones

Este anexo documenta la estructura técnica de las funciones desarrolladas en la **PARTE II** y **PARTE III** de este libro.

Su propósito es servir como referencia rápida para reutilizar estas funciones en proyectos reales.

Aquí se especifican:

- Qué hace la función.
- Parámetros de entrada.
- Estructura de salida.
- Significado de los estados devueltos.

No se repite la explicación teórica ni el código completo, sino únicamente el contrato técnico de cada función.

A.1 Funciones de conectividad

A.2 Funciones de seguridad

A.3 Funciones SSH/SFTP

A.4 Funciones SNMP

A.5 Funciones para interactuar con APIs

A.6 Funciones para notificaciones

A.7 Funciones para registrar logs

A.8 Funciones para concurrencia

Anexo B - Criterios y Buenas Prácticas

Este anexo resume criterios prácticos y recomendaciones utilizadas a lo largo del libro para construir scripts más claros, mantenibles y confiables.

No pretende ser una guía exhaustiva de arquitectura o desarrollo profesional, sino una base mínima de buenas prácticas aplicables incluso en automatizaciones pequeñas.

B.1 Estilo y legibilidad (PEP 8 mínimo viable)

B.2 Gestión segura de credenciales y variables de entorno

B.3 Estandarización de retornos de funciones

B.4 Logging y trazabilidad

B.5 Separación de responsabilidades y estructura de proyecto

B.6 Timeouts, reintentos y fallos transitorios

B.7 Validación de argumentos y errores previsibles

Anexo C - Ideas para proyectos futuros

Este anexo propone ideas de automatización basadas en las funciones desarrolladas a lo largo del libro.

El objetivo es mostrar cómo reutilizar las herramientas construidas para extenderlas hacia escenarios más reales, complejos o especializados.

C.1 Automatizaciones basadas en `execute_ssh()`

C.2 Automatizaciones basadas en `snmp_walk()` y `snmp_get()`

C.3 Automatizaciones basadas en `scan_port()` y `scan_ports()`

C.4 Automatizaciones con `execute_ping()` y `execute_traceroute()`

C.5 Automatizaciones basadas en `api_tools.py`

C.6 Automatizaciones basadas en `notification_tools.py`

Anexo D - Glosario Técnico

Este glosario reúne términos técnicos utilizados frecuentemente a lo largo del libro relacionados con redes, automatización y desarrollo en Python.

Su propósito es servir como referencia rápida para aclarar conceptos importantes sin necesidad de volver constantemente a capítulos anteriores.

D.1 Redes y automatización

D.2 Python y programación

Anexo E - Soluciones a los ejercicios de la PARTE I

Este anexo contiene posibles soluciones para los ejercicios propuestos en la **PARTE I** del libro.

En muchos casos existen múltiples formas válidas de resolver un problema en Python, por lo que las soluciones presentadas deben entenderse como referencias didácticas y no como únicas respuestas correctas.

E.1 Soluciones a ejercicios de Strings y métodos básicos

E.2 Soluciones a ejercicios de Listas y comprensión de listas

E.3 Soluciones a ejercicios de Diccionarios y comprensión de diccionarios

E.4 Soluciones a ejercicios de Iteración

E.5 Soluciones a ejercicios de Condicionales

E.6 Soluciones a ejercicios de Funciones

E.7 Soluciones a ejercicios de Slicing

E.8 Soluciones a ejercicios de Manejo básico de errores

Anexo F - Introducción rápida a PyCharm

Este anexo presenta una guía rápida de herramientas útiles dentro de PyCharm para facilitar el trabajo diario con Python.

El objetivo no es cubrir todas las funcionalidades del IDE, sino mostrar algunas características prácticas que pueden mejorar significativamente la experiencia de desarrollo.

F.1 Interfaz básica de PyCharm

F.2 Trabajar con archivos en PyCharm

F.3 Autocompletado y ayuda inteligente

F.4 Advertencias y detección de errores

F.5 Navegación y edición rápida

F.6 Funciones útiles adicionales de PyCharm

Continuar aprendiendo

Si este material te resultó útil y deseas acceder al contenido completo del libro, puedes adquirir la versión oficial en:

 **Hotmart:**

<https://hotmart.com/es/club/daniel-francisco-llivipuma-pozo>

 **Repositorio oficial:**

<https://github.com/aniluca-publishing/python-para-redes>

 **LinkedIn:**

<https://www.linkedin.com/in/dllivipuma>

¡Gracias por formar parte de este proyecto!

Agradecimiento

Gracias por tomarte el tiempo de leer Python para Redes .

Espero sinceramente que este libro te ayude a aprender, construir nuevas ideas y desarrollar herramientas útiles para tu crecimiento profesional.

Tu apoyo y confianza permiten continuar desarrollando contenido técnico y educativo en español para la comunidad.

Daniel Llivipuma

ANILUCA PUBLISHING